DOCUMENTACIÓN LÍNEA POR LÍNEA

Software Financiero · Frontend React/Vite y backend Node.js/Express

Generado el 03/11/2026 · 44 archivos · 4.646 líneas documentadas

Cómo leer esta guía

Cada fila corresponde a una línea del archivo original: se muestran el número, el código y una explicación individual de su propósito o conexión. Las explicaciones describen la sintaxis y el contexto del proyecto; cuando una línea continúa una expresión, se indica su relación con las líneas vecinas.

Alcance

Incluye los archivos propios JavaScript/JSX, CSS, HTML, JSON y SQL. Se excluyen node_modules, dist y otros resultados generados, archivos .env y secretos, archivos de bloqueo de paquetes, README, licencia y apuntes de pruebas manuales: no son código fuente del sistema o pueden contener datos privados.

Índice de archivos

Archivo	Líneas
Financiero/index.html	18
Financiero/package.json	21
Financiero/src/App.css	41
Financiero/src/App.jsx	87
Financiero/src/components/Bienvenida.css	97
Financiero/src/components/Bienvenida.jsx	196
Financiero/src/components/Footer.css	59
Financiero/src/components/Footer.jsx	30
Financiero/src/components/Funcionalidades.css	540
Financiero/src/components/Funcionalidades.jsx	980
Financiero/src/components/Header.css	158
Financiero/src/components/Header.jsx	82
Financiero/src/components/Hero.css	36
Financiero/src/components/Hero.jsx	38
Financiero/src/components/Login.css	80
Financiero/src/components/Login.jsx	99
Financiero/src/components/Problema.css	56
Financiero/src/components/Problema.jsx	82
Financiero/src/index.css	132

Archivo	Líneas
Financiero/src/main.jsx	13
Financiero/src/pages/Backend.jsx	79
Financiero/src/pages/Dashboard.css	426
Financiero/src/pages/Dashboard.jsx	554
Financiero/src/pages/FuncionalidadesPage.jsx	7
Financiero/src/pages/Inicio.jsx	19
Financiero/src/pages/Proyecto.jsx	7
Financiero/src/services/api.js	13
Financiero/src/services/incomeEvents.js	5
Financiero/vite.config.js	6
package.json	5
server/migrations/20261029_align_ingresos_with_app.sql	33
server/package.json	19
server/src/authSessions.js	29
server/src/config.js	4
server/src/controllers/incomesController.js	184
server/src/controllers/usersController.js	105
server/src/db.js	13
server/src/index.js	43
server/src/middleware/requireAuth.js	16
server/src/middleware/validateIncome.js	99
server/src/middleware/validateUser.js	47
server/src/routes/auth.js	51
server/src/routes/incomes.js	19
server/src/routes/users.js	18

Financiero/index.html

Plantilla inicial que Vite entrega al navegador y donde React monta la interfaz.

18 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`<!DOCTYPE html>`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
2	`<html lang="es">`	Abre el elemento <html>; sus atributos configuran contenido, accesibilidad o eventos.
3	`  <head>`	Abre el elemento <head>; sus atributos configuran contenido, accesibilidad o eventos.
4	`    <meta charset="UTF-8" />`	Abre el elemento <meta>; sus atributos configuran contenido, accesibilidad o eventos.
5	`    <meta name="viewport" content="width=device-width, initial-scale=1.0" />`	Abre el elemento <meta>; sus atributos configuran contenido, accesibilidad o eventos.
6	`    <link rel="preconnect" href="https://fonts.googleapis.com" />`	Abre el elemento <link>; sus atributos configuran contenido, accesibilidad o eventos.
7	`    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />`	Abre el elemento <link>; sus atributos configuran contenido, accesibilidad o eventos.
8	`    <link`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
9	`      href="https://fonts.googleapis.com/css2?family=Anton&family=Barlow:wght@400;500;600;700&display=swap"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
10	`      rel="stylesheet"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
11	`    />`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
12	`    <title>Software Financiero</title>`	Abre el elemento <title>; sus atributos configuran contenido, accesibilidad o eventos.
13	`  </head>`	Cierra el elemento <head> y delimita el contenido que pertenece a él.
14	`  <body>`	Abre el elemento <body>; sus atributos configuran contenido, accesibilidad o eventos.
15	`    <div id="root"></div>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
16	`    <script type="module" src="/src/main.jsx"></script>`	Abre el elemento <script>; sus atributos configuran contenido, accesibilidad o eventos.
17	`  </body>`	Cierra el elemento <body> y delimita el contenido que pertenece a él.
18	`</html>`	Cierra el elemento <html> y delimita el contenido que pertenece a él.

Financiero/package.json

Manifiesto npm: declara identidad, comandos disponibles y dependencias del módulo.

21 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`{`	Abre un objeto JSON que agrupa valores de configuración.
2	`  "name": "jsc-software-financiero",`	Configura «name»: identifica el paquete. Valor: " jsc-software-financiero".
3	`  "private": true,`	Configura «private»: impide publicar accidentalmente el paquete en npm. Valor: true.
4	`  "version": "1.0.0",`	Configura «version»: indica su versión. Valor: "1.0.0".
5	`  "type": "module",`	Configura «type»: indica el sistema de módulos de Node.js. Valor: "module".
6	`  "scripts": {`	Configura «scripts»: agrupa comandos ejecutables. Valor: {.
7	`    "dev": "vite",`	Configura «dev»: define el comando de desarrollo. Valor: "vite".
8	`    "build": "vite build",`	Configura «build»: define el comando de compilación. Valor: "vite build".
9	`    "preview": "vite preview"`	Configura «preview»: define esta opción del manifiesto. Valor: " vite preview".
10	`  },`	Cierra el objeto o sección de configuración JSON.
11	`  "dependencies": {`	Configura «dependencies»: declara bibliotecas necesarias en ejecución. Valor: {.
12	`    "axios": "^1.20.0",`	Configura «axios»: define esta opción del manifiesto. Valor: " ^1.20.0".
13	`    "react": "^18.3.1",`	Configura «react»: define esta opción del manifiesto. Valor: " ^18.3.1".
14	`    "react-dom": "^18.3.1",`	Configura «react-dom»: define esta opción del manifiesto. Valor: " ^18.3.1".
15	`    "react-router-dom": "^6.30.6"`	Configura «react-router-dom»: define esta opción del manifiesto. Valor: "^6.30.6".
16	`  },`	Cierra el objeto o sección de configuración JSON.
17	`  "devDependencies": {`	Configura «devDependencies»: declara herramientas de desarrollo o compilación. Valor: {.
18	`    "@vitejs/plugin-react": "^4.3.1",`	Configura «@vitejs/plugin-react»: define esta opción del manifiesto. Valor: "^4.3.1".
19	`    "vite": "^5.4.0"`	Configura «vite»: define esta opción del manifiesto. Valor: " ^5.4.0".
20	`  }`	Cierra el objeto o sección de configuración JSON.
21	`}`	Cierra el objeto o sección de configuración JSON.

Financiero/src/App.css

Hoja de estilos de App: define presentación, distribución, estados visuales y adaptación de pantalla.

41 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`/* Estilos generales de layout que no pertenecen a un componente específico */`	Comentario CSS; documenta el estilo, pero no cambia la apariencia renderizada.
2	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
3	`.app-shell {`	Abre las reglas visuales del selector .app-shell.
4	`  min-height: 100vh;`	La propiedad min-height establece la altura mínima; valor: 100vh.
5	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
6	`  flex-direction: column;`	La propiedad flex-direction define la dirección del flexbox; valor: column.
7	`}`	Cierra el bloque CSS actual.
8	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
9	`.app-main {`	Abre las reglas visuales del selector .app-main.
10	`  flex: 1;`	La propiedad flex define el crecimiento y tamaño flexible; valor: 1.
11	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
12	`  flex-direction: column;`	La propiedad flex-direction define la dirección del flexbox; valor: column.
13	`}`	Cierra el bloque CSS actual.
14	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
15	`.login-required {`	Abre las reglas visuales del selector .login-required.
16	`  width: min(720px, calc(100% - 3rem));`	La propiedad width establece el ancho; valor: min(720px, calc(100% - 3rem)).
17	`  margin: auto;`	La propiedad margin define el espacio exterior; valor: auto.
18	`  padding: 5rem 1.5rem;`	La propiedad padding define el espacio interior; valor: 5rem 1.5rem.
19	`  text-align: center;`	La propiedad text-align alinea el texto; valor: center.
20	`}`	Cierra el bloque CSS actual.
21	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
22	`.login-required h1 {`	Abre las reglas visuales del selector .login-required h1.
23	`  margin-bottom: 1.15rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 1.15rem.
24	`  font-size: clamp(2rem, 3.8vw, 2.8rem);`	La propiedad font-size define el tamaño de letra; valor: clamp(2rem, 3.8vw, 2.8rem).
25	`}`	Cierra el bloque CSS actual.
26	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
27	`.login-required p:not(.eyebrow) {`	Abre las reglas visuales del selector .login-required p:not(.eyebrow).
28	`  margin-bottom: 1.35rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 1.35rem.
29	`  color: var(--ink-soft);`	La propiedad color define el color del texto; valor: var(--ink-soft).
30	`  font-size: 1.1rem;`	La propiedad font-size define el tamaño de letra; valor: 1.1rem.
31	`}`	Cierra el bloque CSS actual.
32	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.

N.º	Código original	Qué hace esta línea
33	`.login-required .btn {`	Abre las reglas visuales del selector .login-required .btn.
34	`  margin-top: 0.25rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 0.25rem.
35	`  padding: 1rem 2rem;`	La propiedad padding define el espacio interior; valor: 1rem 2rem.
36	`  font-size: 1.05rem;`	La propiedad font-size define el tamaño de letra; valor: 1.05rem.
37	`}`	Cierra el bloque CSS actual.
38	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
39	`#root {`	Abre las reglas visuales del selector #root.
40	`  min-height: 100vh;`	La propiedad min-height establece la altura mínima; valor: 100vh.
41	`}`	Cierra el bloque CSS actual.

Financiero/src/App.jsx

Coordina las rutas protegidas, el inicio/cierre de sesión y los componentes globales; conecta el frontend con la API de autenticación.

87 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { useState } from 'react'`	Importa { useState } desde react y conecta este archivo con ese recurso.
2	`import { Routes, Route } from 'react-router-dom'`	Importa { Routes, Route } desde react-router-dom y conecta este archivo con ese recurso.
3	`import axios from 'axios'`	Importa axios desde axios y conecta este archivo con ese recurso.
4	`import Header from './components/Header.jsx'`	Importa Header desde ./components/Header.jsx y conecta este archivo con ese recurso.
5	`import Footer from './components/Footer.jsx'`	Importa Footer desde ./components/Footer.jsx y conecta este archivo con ese recurso.
6	`import Login from './components/Login.jsx'`	Importa Login desde ./components/Login.jsx y conecta este archivo con ese recurso.
7	`import Inicio from './pages/Inicio.jsx'`	Importa Inicio desde ./pages/Inicio.jsx y conecta este archivo con ese recurso.
8	`import Dashboard from './pages/Dashboard.jsx'`	Importa Dashboard desde ./pages/Dashboard.jsx y conecta este archivo con ese recurso.
9	`import Proyecto from './pages/Proyecto.jsx'`	Importa Proyecto desde ./pages/Proyecto.jsx y conecta este archivo con ese recurso.
10	`import FuncionalidadesPage from './pages/FuncionalidadesPage.jsx'`	Importa FuncionalidadesPage desde ./pages/FuncionalidadesPage.jsx y conecta este archivo con ese recurso.
11	`import Backend from './pages/Backend.jsx'`	Importa Backend desde ./pages/Backend.jsx y conecta este archivo con ese recurso.
12	`import './App.css'`	Importa './App.css' desde el módulo indicado y conecta este archivo con ese recurso.
13	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
14	`function App() {`	Declara la función App; el bloque siguiente contiene las instrucciones que ejecuta.
15	`  const [usuario, setUsuario] = useState(() =>`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
16	`    sessionStorage.getItem('authToken') ? sessionStorage.getItem('usuario') : null,`	Lee, guarda o elimina un dato de sesión en el almacenamiento temporal del navegador.
17	`  )`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
18	`  const [loginAbierto, setLoginAbierto] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
19	`  const API_URL = import.meta.env.VITE_API_URL`	Lee en API_URL una variable de configuración expuesta por Vite al frontend.
20	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
21	`  function iniciarSesion(nombreUsuario, token) {`	Declara la función iniciarSesion; el bloque siguiente contiene las instrucciones que ejecuta.
22	`    sessionStorage.setItem('authToken', token)`	Lee, guarda o elimina un dato de sesión en el almacenamiento temporal del navegador.
23	`    sessionStorage.setItem('usuario', nombreUsuario)`	Lee, guarda o elimina un dato de sesión en el almacenamiento temporal del navegador.
24	`    setUsuario(nombreUsuario)`	Invoca setUsuario para ejecutar la operación definida por esa función o biblioteca.
25	`    setLoginAbierto(false)`	Invoca setLoginAbierto para ejecutar la operación definida por esa función o biblioteca.
26	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
27	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
28	`  async function cerrarSesion() {`	Declara la función cerrarSesion; el bloque siguiente contiene las instrucciones que ejecuta.
29	`    const token = sessionStorage.getItem('authToken')`	Lee, guarda o elimina un dato de sesión en el almacenamiento temporal del navegador.

N.º	Código original	Qué hace esta línea
30	`if (token && API_URL) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
31	`try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
32	`await axios.post(`	Espera una petición HTTP al backend antes de continuar.
33	`` `${API_URL.replace(/\/+$/, '')}/auth/logout`, ``	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
34	`{},`	Abre un bloque de código, objeto o expresión JSX.
35	`{ headers: { Authorization: `Bearer ${token}` } },`	Abre un bloque de código, objeto o expresión JSX.
36	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
37	`} catch (error) {`	Abre el bloque de código que contiene las instrucciones siguientes.
38	`console.error('No fue posible invalidar la sesión en el servidor:', error)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
39	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
40	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
41	`sessionStorage.removeItem('authToken')`	Lee, guarda o elimina un dato de sesión en el almacenamiento temporal del navegador.
42	`sessionStorage.removeItem('usuario')`	Lee, guarda o elimina un dato de sesión en el almacenamiento temporal del navegador.
43	`setUsuario(null)`	Invoca setUsuario para ejecutar la operación definida por esa función o biblioteca.
44	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
45	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
46	`return (`	Devuelve un valor de la función o el elemento React que se renderizará.
47	`<div className="app-shell">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
48	`<Header`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
49	`usuario={usuario}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
50	`onIniciarSesion={() => setLoginAbierto(true)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
51	`onCerrarSesion={cerrarSesion}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
52	`/>`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
53	`<main className="app-main">`	Abre el elemento <main>; sus atributos configuran contenido, accesibilidad o eventos.
54	`{usuario ? (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
55	`<Routes>`	Inserta el componente React Routes y le pasa sus props.
56	`<Route path="/" element={<Dashboard />} />`	Inserta el componente React Route y le pasa sus props.
57	`<Route path="/inicio" element={<Inicio />} />`	Inserta el componente React Route y le pasa sus props.
58	`<Route path="/proyecto" element={<Proyecto />} />`	Inserta el componente React Route y le pasa sus props.
59	`<Route path="/funcionalidades" element={<FuncionalidadesPage />} />`	Inserta el componente React Route y le pasa sus props.
60	`<Route path="/api-test" element={<Backend />} />`	Inserta el componente React Route y le pasa sus props.
61	`<Route path="/backend" element={<Backend />} />`	Inserta el componente React Route y le pasa sus props.

N.º	Código original	Qué hace esta línea
62	`</Routes>`	Cierra el elemento <Routes> y delimita el contenido que pertenece a él.
63	`) : (`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
64	`<section className="login-required">`	Abre el elemento <section>; sus atributos configuran contenido, accesibilidad o eventos.
65	`<h1>Inicia sesión para continuar</h1>`	Abre el elemento <h1>; sus atributos configuran contenido, accesibilidad o eventos.
66	`<p>Usa tu cuenta de administrador para acceder al sistema financiero.</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
67	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
68	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
69	`className="btn btn-primary"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
70	`onClick={() => setLoginAbierto(true)}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
71	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
72	`Iniciar sesión`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
73	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
74	`</section>`	Cierra el elemento <section> y delimita el contenido que pertenece a él.
75	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
76	`</main>`	Cierra el elemento <main> y delimita el contenido que pertenece a él.
77	`<Login`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
78	`abierto={loginAbierto}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
79	`onCerrar={() => setLoginAbierto(false)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
80	`onIniciarSesion={iniciarSesion}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
81	`/>`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
82	`<Footer />`	Inserta el componente React Footer y le pasa sus props.
83	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
84	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
85	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
86	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
87	`export default App`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.

Financiero/src/components/Bienvenida.css

Hoja de estilos de Bienvenida: define presentación, distribución, estados visuales y adaptación de pantalla.

97 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`.bienvenida {`	Abre las reglas visuales del selector .bienvenida.
2	`  padding: 3.5rem 1.5rem;`	La propiedad padding define el espacio interior; valor: 3.5rem 1.5rem.
3	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
4	`  justify-content: center;`	La propiedad justify-content distribuye elementos en el eje principal; valor: center.
5	`}`	Cierra el bloque CSS actual.
6	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
7	`.bienvenida-cerrando {`	Abre las reglas visuales del selector .bienvenida-cerrando.
8	`  overflow: hidden;`	La propiedad overflow controla el contenido que desborda; valor: hidden.
9	`  animation: bienvenida-cerrar 350ms ease-in forwards;`	La propiedad animation asocia una animación; valor: bienvenida-cerrar 350ms ease-in forwards.
10	`}`	Cierra el bloque CSS actual.
11	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
12	`@keyframes bienvenida-cerrar {`	Declara los pasos de una animación CSS que puede asociarse a un elemento.
13	`  from {`	Abre el paso «from» de una animación CSS.
14	`    height: var(--altura-seccion);`	La propiedad height establece la altura; valor: var(--altura-seccion).
15	`    padding-top: 3.5rem;`	La propiedad padding-top establece la propiedad visual padding-top; valor: 3.5rem.
16	`    padding-bottom: 3.5rem;`	La propiedad padding-bottom establece la propiedad visual padding-bottom; valor: 3.5rem.
17	`    opacity: 1;`	La propiedad opacity define la transparencia; valor: 1.
18	`  }`	Cierra el bloque CSS actual.
19	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
20	`  to {`	Abre el paso «to» de una animación CSS.
21	`    height: 0;`	La propiedad height establece la altura; valor: 0.
22	`    padding-top: 0;`	La propiedad padding-top establece la propiedad visual padding-top; valor: 0.
23	`    padding-bottom: 0;`	La propiedad padding-bottom establece la propiedad visual padding-bottom; valor: 0.
24	`    opacity: 0;`	La propiedad opacity define la transparencia; valor: 0.
25	`  }`	Cierra el bloque CSS actual.
26	`}`	Cierra el bloque CSS actual.
27	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
28	`.bienvenida-card {`	Abre las reglas visuales del selector .bienvenida-card.
29	`  background: var(--white);`	La propiedad background define el fondo; valor: var(--white).
30	`  border-radius: var(--radius-md);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-md).
31	`  padding: 2.2rem 2rem;`	La propiedad padding define el espacio interior; valor: 2.2rem 2rem.
32	`  max-width: 560px;`	La propiedad max-width establece el ancho máximo; valor: 560px.

N.º	Código original	Qué hace esta línea
33	`  width: 100%;`	La propiedad width establece el ancho; valor: 100%.
34	`  box-shadow: 0 10px 30px rgba(18, 61, 67, 0.12);`	La propiedad box-shadow añade sombra; valor: 0 10px 30px rgba(18, 61, 67, 0.12).
35	`  text-align: center;`	La propiedad text-align alinea el texto; valor: center.
36	`}`	Cierra el bloque CSS actual.
37	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
38	`.bienvenida-card h2 {`	Abre las reglas visuales del selector .bienvenida-card h2.
39	`  margin-bottom: 1.2rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 1.2rem.
40	`  font-size: 1.5rem;`	La propiedad font-size define el tamaño de letra; valor: 1.5rem.
41	`}`	Cierra el bloque CSS actual.
42	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
43	`.form-bienvenida-field {`	Abre las reglas visuales del selector .form-bienvenida-field.
44	`  margin-bottom: 0.9rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 0.9rem.
45	`  text-align: left;`	La propiedad text-align alinea el texto; valor: left.
46	`}`	Cierra el bloque CSS actual.
47	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
48	`.form-bienvenida label {`	Abre las reglas visuales del selector .form-bienvenida label.
49	`  display: block;`	La propiedad display define cómo se distribuyen los elementos; valor: block.
50	`  text-align: left;`	La propiedad text-align alinea el texto; valor: left.
51	`  font-weight: 600;`	La propiedad font-weight define el grosor de letra; valor: 600.
52	`  color: var(--ink-soft);`	La propiedad color define el color del texto; valor: var(--ink-soft).
53	`  margin-bottom: 0.5rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 0.5rem.
54	`  font-size: 0.9rem;`	La propiedad font-size define el tamaño de letra; valor: 0.9rem.
55	`}`	Cierra el bloque CSS actual.
56	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
57	`.form-bienvenida input {`	Abre las reglas visuales del selector .form-bienvenida input.
58	`  width: 100%;`	La propiedad width establece el ancho; valor: 100%.
59	`  padding: 0.8rem 1rem;`	La propiedad padding define el espacio interior; valor: 0.8rem 1rem.
60	`  border: 1px solid #cfe0e2;`	La propiedad border define el borde; valor: 1px solid #cfe0e2.
61	`  border-radius: var(--radius-sm);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-sm).
62	`  font-family: var(--font-body);`	La propiedad font-family elige la tipografía; valor: var(--font-body).
63	`  font-size: 1rem;`	La propiedad font-size define el tamaño de letra; valor: 1rem.
64	`  background: var(--bg-light);`	La propiedad background define el fondo; valor: var(--bg-light).
65	`  color: var(--ink);`	La propiedad color define el color del texto; valor: var(--ink).
66	`}`	Cierra el bloque CSS actual.
67	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
68	`.form-row input:focus {`	Abre las reglas visuales del selector .form-row input:focus.
69	`  outline: 2px solid var(--cyan-500);`	La propiedad outline establece la propiedad visual outline; valor: 2px solid var(--cyan-500).

N.º	Código original	Qué hace esta línea
70	`  outline-offset: 1px;`	La propiedad outline-offset establece la propiedad visual outline-offset; valor: 1px.
71	`}`	Cierra el bloque CSS actual.
72	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
73	`.form-bienvenida .btn {`	Abre las reglas visuales del selector .form-bienvenida .btn.
74	`  margin-top: 0.3rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 0.3rem.
75	`}`	Cierra el bloque CSS actual.
76	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
77	`.error-msg {`	Abre las reglas visuales del selector .error-msg.
78	`  color: #c23b3b;`	La propiedad color define el color del texto; valor: #c23b3b.
79	`  font-size: 0.85rem;`	La propiedad font-size define el tamaño de letra; valor: 0.85rem.
80	`  min-height: 1.2rem;`	La propiedad min-height establece la altura mínima; valor: 1.2rem.
81	`  margin: 0.6rem 0 0;`	La propiedad margin define el espacio exterior; valor: 0.6rem 0 0.
82	`  text-align: left;`	La propiedad text-align alinea el texto; valor: left.
83	`}`	Cierra el bloque CSS actual.
84	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
85	`.mensaje-dinamico {`	Abre las reglas visuales del selector .mensaje-dinamico.
86	`  margin-top: 1.2rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 1.2rem.
87	`  font-weight: 600;`	La propiedad font-weight define el grosor de letra; valor: 600.
88	`  color: var(--teal-800);`	La propiedad color define el color del texto; valor: var(--teal-800).
89	`  font-size: 1.05rem;`	La propiedad font-size define el tamaño de letra; valor: 1.05rem.
90	`  min-height: 1.2rem;`	La propiedad min-height establece la altura mínima; valor: 1.2rem.
91	`}`	Cierra el bloque CSS actual.
92	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
93	`@media (prefers-reduced-motion: reduce) {`	Inicia una condición responsive; sus estilos se aplican cuando se cumple esta condición de pantalla.
94	`  .bienvenida-cerrando {`	Abre las reglas visuales del selector .bienvenida-cerrando.
95	`    animation-duration: 1ms;`	La propiedad animation-duration establece la propiedad visual animation-duration; valor: 1ms.
96	`  }`	Cierra el bloque CSS actual.
97	`}`	Cierra el bloque CSS actual.

Financiero/src/components/Bienvenida.jsx

Valida nombre, apellido y correo; envía el alta al endpoint /users y presenta el resultado.

196 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { useEffect, useRef, useState } from 'react'`	Importa { useEffect, useRef, useState } desde react y conecta este archivo con ese recurso.
2	`import apiClient from '../services/api.js'`	Importa apiClient desde ../services/api.js y conecta este archivo con ese recurso.
3	`import './Bienvenida.css'`	Importa './Bienvenida.css' desde el módulo indicado y conecta este archivo con ese recurso.
4	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
5	`function Bienvenida({ visible, onCerrar }) {`	Declara la función Bienvenida; el bloque siguiente contiene las instrucciones que ejecuta.
6	`  const seccionRef = useRef(null)`	Crea una referencia persistente para conservar un valor o acceder a un elemento DOM.
7	`  const nombreRef = useRef(null)`	Crea una referencia persistente para conservar un valor o acceder a un elemento DOM.
8	`  const [nombre, setNombre] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
9	`  const [apellido, setApellido] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
10	`  const [email, setEmail] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
11	`  const [error, setError] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
12	`  const [mensaje, setMensaje] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
13	`  const [guardando, setGuardando] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
14	`  const [cerrando, setCerrando] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
15	`  const [alturaSeccion, setAlturaSeccion] = useState(0)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
16	`  const temporizadorRef = useRef(null)`	Crea una referencia persistente para conservar un valor o acceder a un elemento DOM.
17	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
18	`  const API_URL = import.meta.env.VITE_API_URL`	Lee en API_URL una variable de configuración expuesta por Vite al frontend.
19	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
20	`  useEffect(() => {`	Registra un efecto React para sincronizar la interfaz con eventos o recursos externos.
21	`    return () => clearTimeout(temporizadorRef.current)`	Devuelve un valor de la función o el elemento React que se renderizará.
22	`  }, [])`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
23	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
24	`  useEffect(() => {`	Registra un efecto React para sincronizar la interfaz con eventos o recursos externos.
25	`    if (visible) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
26	`      clearTimeout(temporizadorRef.current)`	Invoca clearTimeout para ejecutar la operación definida por esa función o biblioteca.
27	`      setCerrando(false)`	Invoca setCerrando para ejecutar la operación definida por esa función o biblioteca.
28	`      setMensaje('')`	Invoca setMensaje para ejecutar la operación definida por esa función o biblioteca.
29	`      seccionRef.current?.scrollIntoView({ behavior: 'smooth' })`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.

N.º	Código original	Qué hace esta línea
30	`nombreRef.current?.focus({ preventScroll: true })`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
31	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
32	`}, [visible])`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
33	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
34	`const manejarCambio = (setter) => (evento) => {`	Define en manejarCambio una función flecha para reutilizar o ejecutar una acción.
35	`setter(evento.target.value)`	Invoca setter para ejecutar la operación definida por esa función o biblioteca.
36	`clearTimeout(temporizadorRef.current)`	Invoca clearTimeout para ejecutar la operación definida por esa función o biblioteca.
37	`setError('')`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
38	`setMensaje('')`	Invoca setMensaje para ejecutar la operación definida por esa función o biblioteca.
39	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
40	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
41	`const mostrarError = (mensajeError) => {`	Define en mostrarError una función flecha para reutilizar o ejecutar una acción.
42	`setError(mensajeError)`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
43	`clearTimeout(temporizadorRef.current)`	Invoca clearTimeout para ejecutar la operación definida por esa función o biblioteca.
44	`temporizadorRef.current = setTimeout(() => setError(''), 5000)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
45	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
46	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
47	`const manejarEnvio = async (evento) => {`	Define en manejarEnvio una función flecha para reutilizar o ejecutar una acción.
48	`evento.preventDefault()`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
49	`clearTimeout(temporizadorRef.current)`	Invoca clearTimeout para ejecutar la operación definida por esa función o biblioteca.
50	`setError('')`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
51	`setMensaje('')`	Invoca setMensaje para ejecutar la operación definida por esa función o biblioteca.
52	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
53	`const nombreLimpio = nombre.trim()`	Declara nombreLimpio y le asigna el valor o expresión mostrado para reutilizarlo.
54	`const apellidoLimpio = apellido.trim()`	Declara apellidoLimpio y le asigna el valor o expresión mostrado para reutilizarlo.
55	`const emailLimpio = email.trim()`	Declara emailLimpio y le asigna el valor o expresión mostrado para reutilizarlo.
56	`const soloLetras = /^[\p{L} ]+$/u`	Declara soloLetras y le asigna el valor o expresión mostrado para reutilizarlo.
57	`const emailValido = /^[^\s@]+@[^\s@]+\.[^\s@]+$/`	Declara emailValido y le asigna el valor o expresión mostrado para reutilizarlo.
58	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
59	`if (!nombreLimpio) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
60	`mostrarError('El campo nombre no puede ir vacío')`	Invoca mostrarError para ejecutar la operación definida por esa función o biblioteca.
61	`return`	Devuelve un valor de la función o el elemento React que se renderizará.

N.º	Código original	Qué hace esta línea
62	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
63	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
64	`if (nombreLimpio.length < 2 || nombreLimpio.length > 50 || !soloLetras.test(nombreLimpio)) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
65	`mostrarError('El nombre debe tener entre 2 y 50 caracteres y solo puede contener letras y espacios.')`	Invoca mostrarError para ejecutar la operación definida por esa función o biblioteca.
66	`return`	Devuelve un valor de la función o el elemento React que se renderizará.
67	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
68	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
69	`if (!apellidoLimpio) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
70	`mostrarError('El campo apellido no puede ir vacío')`	Invoca mostrarError para ejecutar la operación definida por esa función o biblioteca.
71	`return`	Devuelve un valor de la función o el elemento React que se renderizará.
72	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
73	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
74	`if (apellidoLimpio.length < 2 || apellidoLimpio.length > 50 || !soloLetras.test(apellidoLimpio)) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
75	`mostrarError('El apellido debe tener entre 2 y 50 caracteres y solo puede contener letras y espacios.')`	Invoca mostrarError para ejecutar la operación definida por esa función o biblioteca.
76	`return`	Devuelve un valor de la función o el elemento React que se renderizará.
77	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
78	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
79	`if (!emailLimpio) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
80	`mostrarError('El campo correo electrónico no puede ir vacío')`	Invoca mostrarError para ejecutar la operación definida por esa función o biblioteca.
81	`return`	Devuelve un valor de la función o el elemento React que se renderizará.
82	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
83	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
84	`if (emailLimpio.length > 100 || !emailValido.test(emailLimpio)) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
85	`mostrarError('Escribe un correo electrónico válido de máximo 100 caracteres.')`	Invoca mostrarError para ejecutar la operación definida por esa función o biblioteca.
86	`return`	Devuelve un valor de la función o el elemento React que se renderizará.
87	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
88	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
89	`if (!API_URL) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
90	`mostrarError('Falta configurar VITE_API_URL en el archivo .env del frontend.')`	Invoca mostrarError para ejecutar la operación definida por esa función o biblioteca.
91	`return`	Devuelve un valor de la función o el elemento React que se renderizará.
92	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.

N.º	Código original	Qué hace esta línea
93	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
94	`setGuardando(true)`	Invoca setGuardando para ejecutar la operación definida por esa función o biblioteca.
95	`try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
96	`await apiClient.post(`${API_URL.replace(/\/+$/, '')}/users`, {`	Espera una petición HTTP al backend antes de continuar.
97	`name: nombreLimpio,`	Define el valor de la propiedad «name» del objeto actual.
98	`lastName: apellidoLimpio,`	Define el valor de la propiedad «lastName» del objeto actual.
99	`email: emailLimpio,`	Define el valor de la propiedad «email» del objeto actual.
100	`})`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
101	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
102	`setNombre('')`	Invoca setNombre para ejecutar la operación definida por esa función o biblioteca.
103	`setApellido('')`	Invoca setApellido para ejecutar la operación definida por esa función o biblioteca.
104	`setEmail('')`	Invoca setEmail para ejecutar la operación definida por esa función o biblioteca.
105	`setMensaje(`Usuario guardado correctamente. ¡Bienvenido, ${nombreLimpio}!`)`	Invoca setMensaje para ejecutar la operación definida por esa función o biblioteca.
106	`temporizadorRef.current = setTimeout(() => {`	Abre el bloque de código que contiene las instrucciones siguientes.
107	`setAlturaSeccion(seccionRef.current?.getBoundingClientRect().height || 0)`	Invoca setAlturaSeccion para ejecutar la operación definida por esa función o biblioteca.
108	`setCerrando(true)`	Invoca setCerrando para ejecutar la operación definida por esa función o biblioteca.
109	`}, 1000)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
110	`} catch (errorSolicitud) {`	Abre el bloque de código que contiene las instrucciones siguientes.
111	`const mensajeError =`	Declara mensajeError y le asigna el valor o expresión mostrado para reutilizarlo.
112	`errorSolicitud.response?.data?.message || 'No fue posible conectar con el servidor.'`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
113	`mostrarError(mensajeError)`	Invoca mostrarError para ejecutar la operación definida por esa función o biblioteca.
114	`} finally {`	Abre el bloque de código que contiene las instrucciones siguientes.
115	`setGuardando(false)`	Invoca setGuardando para ejecutar la operación definida por esa función o biblioteca.
116	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
117	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
118	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
119	`const manejarFinAnimacion = (evento) => {`	Define en manejarFinAnimacion una función flecha para reutilizar o ejecutar una acción.
120	`if (evento.target === evento.currentTarget && evento.animationName === 'bienvenida-cerrar') {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
121	`onCerrar()`	Invoca onCerrar para ejecutar la operación definida por esa función o biblioteca.
122	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
123	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
124	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.

N.º	Código original	Qué hace esta línea
125	`  if (!visible) return null`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
126	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
127	`  return (`	Devuelve un valor de la función o el elemento React que se renderizará.
128	`    <section`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
129	`      className={`bienvenida${cerrando ? ' bienvenida-cerrando' : ''}`}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
130	`      id="bienvenida"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
131	`      ref={seccionRef}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
132	`      style={cerrando ? { '--altura-seccion': `${alturaSeccion}px` } : undefined}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
133	`      onAnimationEnd={manejarFinAnimacion}`	Conecta el evento onAnimationEnd del elemento visual con la función que responde a la interacción.
134	`    >`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
135	`      <div className="bienvenida-card">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
136	`        <h2>Bienvenido(a)</h2>`	Abre el elemento <h2>; sus atributos configuran contenido, accesibilidad o eventos.
137	`        <form className="form-bienvenida" onSubmit={manejarEnvio} noValidate>`	Conecta el evento onSubmit del elemento visual con la función que responde a la interacción.
138	`          <div className="form-bienvenida-field">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
139	`            <label htmlFor="nombreUsuario">Nombre:</label>`	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.
140	`            <input`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
141	`              type="text"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
142	`              id="nombreUsuario"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
143	`              name="nombreUsuario"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
144	`              ref={nombreRef}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
145	`              placeholder="Escribe tu nombre"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
146	`              value={nombre}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
147	`              maxLength={50}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
148	`              onChange={manejarCambio(setNombre)}`	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
149	`              disabled={Boolean(mensaje)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
150	`              required`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
151	`            />`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
152	`          </div>`	Cierra el elemento y delimita el contenido que pertenece a él.
153	`          <div className="form-bienvenida-field">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
154	`            <label htmlFor="apellidoUsuario">Apellido:</label>`	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.
155	`            <input`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.

N.º	Código original	Qué hace esta línea
156	`type="text"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
157	`id="apellidoUsuario"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
158	`name="apellidoUsuario"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
159	`placeholder="Escribe tu apellido"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
160	`value={apellido}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
161	`maxLength={50}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
162	`onChange={manejarCambio(setApellido)}`	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
163	`disabled={Boolean(mensaje)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
164	`required`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
165	`/>`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
166	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
167	`<div className="form-bienvenida-field">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
168	`<label htmlFor="emailUsuario">Correo electrónico:</label>`	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.
169	`<input`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
170	`type="email"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
171	`id="emailUsuario"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
172	`name="emailUsuario"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
173	`placeholder="nombre@correo.com"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
174	`value={email}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
175	`maxLength={100}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
176	`onChange={manejarCambio(setEmail)}`	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
177	`disabled={Boolean(mensaje)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
178	`required`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
179	`/>`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
180	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
181	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
182	`type="submit"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
183	`className="btn btn-primary"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
184	`disabled={guardando || cerrando || Boolean(mensaje)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
185	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
186	`{guardando ? 'Guardando...' : 'Guardar'}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.

N.º	Código original	Qué hace esta línea
187	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
188	`<p className="error-msg" aria-live="polite">{error}</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
189	`</form>`	Cierra el elemento <form> y delimita el contenido que pertenece a él.
190	`<p className="mensaje-dinamico" aria-live="polite">{mensaje}</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
191	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
192	`</section>`	Cierra el elemento <section> y delimita el contenido que pertenece a él.
193	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
194	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
195	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
196	`export default Bienvenida`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.

Financiero/src/components/Footer.css

Hoja de estilos de Footer: define presentación, distribución, estados visuales y adaptación de pantalla.

59 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`.site-footer {`	Abre las reglas visuales del selector .site-footer.
2	`  background: var(--teal-950);`	La propiedad background define el fondo; valor: var(--teal-950).
3	`  color: #cfe6e9;`	La propiedad color define el color del texto; valor: #cfe6e9.
4	`}`	Cierra el bloque CSS actual.
5	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
6	`.footer-inner {`	Abre las reglas visuales del selector .footer-inner.
7	`  max-width: 900px;`	La propiedad max-width establece el ancho máximo; valor: 900px.
8	`  margin: 0 auto;`	La propiedad margin define el espacio exterior; valor: 0 auto.
9	`  padding: 3.5rem 1.5rem 2rem;`	La propiedad padding define el espacio interior; valor: 3.5rem 1.5rem 2rem.
10	`  display: grid;`	La propiedad display define cómo se distribuyen los elementos; valor: grid.
11	`  grid-template-columns: repeat(3, minmax(0, 1fr));`	La propiedad grid-template-columns define las columnas de la cuadrícula; valor: repeat(3, minmax(0, 1fr)).
12	`  gap: 2.5rem;`	La propiedad gap define el espacio entre elementos; valor: 2.5rem.
13	`}`	Cierra el bloque CSS actual.
14	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
15	`.footer-brand .brand-name {`	Abre las reglas visuales del selector .footer-brand .brand-name.
16	`  display: block;`	La propiedad display define cómo se distribuyen los elementos; valor: block.
17	`  margin-bottom: 0.6rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 0.6rem.
18	`  font-family: var(--font-display);`	La propiedad font-family elige la tipografía; valor: var(--font-display).
19	`  color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
20	`  font-size: 1.05rem;`	La propiedad font-size define el tamaño de letra; valor: 1.05rem.
21	`  text-transform: uppercase;`	La propiedad text-transform establece la propiedad visual text-transform; valor: uppercase.
22	`}`	Cierra el bloque CSS actual.
23	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
24	`.footer-brand p {`	Abre las reglas visuales del selector .footer-brand p.
25	`  color: #9fc4c9;`	La propiedad color define el color del texto; valor: #9fc4c9.
26	`  font-size: 0.9rem;`	La propiedad font-size define el tamaño de letra; valor: 0.9rem.
27	`}`	Cierra el bloque CSS actual.
28	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
29	`.footer-col h4 {`	Abre las reglas visuales del selector .footer-col h4.
30	`  color: var(--cyan-300);`	La propiedad color define el color del texto; valor: var(--cyan-300).
31	`  font-size: 0.95rem;`	La propiedad font-size define el tamaño de letra; valor: 0.95rem.
32	`  margin-bottom: 0.6rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 0.6rem.
33	`}`	Cierra el bloque CSS actual.

N.º	Código original	Qué hace esta línea
34	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
35	.footer-col p {	Abre las reglas visuales del selector .footer-col p.
36	color: #bcdbdf;	La propiedad color define el color del texto; valor: #bcdbdf.
37	font-size: 0.9rem;	La propiedad font-size define el tamaño de letra; valor: 0.9rem.
38	margin: 0 0 0.3rem;	La propiedad margin define el espacio exterior; valor: 0 0 0.3rem.
39	}	Cierra el bloque CSS actual.
40	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
41	.footer-bottom {	Abre las reglas visuales del selector .footer-bottom.
42	border-top: 1px solid rgba(255, 255, 255, 0.1);	La propiedad border-top establece la propiedad visual border-top; valor: 1px solid rgba(255, 255, 255, 0.1).
43	text-align: center;	La propiedad text-align alinea el texto; valor: center.
44	padding: 1.2rem 1.5rem;	La propiedad padding define el espacio interior; valor: 1.2rem 1.5rem.
45	font-size: 0.85rem;	La propiedad font-size define el tamaño de letra; valor: 0.85rem.
46	color: #8fb7bc;	La propiedad color define el color del texto; valor: #8fb7bc.
47	}	Cierra el bloque CSS actual.
48	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
49	@media (max-width: 860px) {	Inicia una condición responsive; sus estilos se aplican cuando se cumple esta condición de pantalla.
50	.footer-inner {	Abre las reglas visuales del selector .footer-inner.
51	gap: 1.5rem;	La propiedad gap define el espacio entre elementos; valor: 1.5rem.
52	}	Cierra el bloque CSS actual.
53	}	Cierra el bloque CSS actual.
54	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
55	@media (max-width: 640px) {	Inicia una condición responsive; sus estilos se aplican cuando se cumple esta condición de pantalla.
56	.footer-inner {	Abre las reglas visuales del selector .footer-inner.
57	grid-template-columns: 1fr;	La propiedad grid-template-columns define las columnas de la cuadrícula; valor: 1fr.
58	}	Cierra el bloque CSS actual.
59	}	Cierra el bloque CSS actual.

Financiero/src/components/Footer.jsx

Componente o página React Footer: construye una parte de la interfaz y usa las props y estados del módulo.

30 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import './Footer.css'`	Importa './Footer.css' desde el módulo indicado y conecta este archivo con ese recurso.
2	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
3	`function Footer() {`	Declara la función Footer; el bloque siguiente contiene las instrucciones que ejecuta.
4	`  const anioActual = new Date().getFullYear()`	Declara anioActual y le asigna el valor o expresión mostrado para reutilizarlo.
5	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
6	`  return (`	Devuelve un valor de la función o el elemento React que se renderizará.
7	`    <footer className="site-footer">`	Abre el elemento <footer>; sus atributos configuran contenido, accesibilidad o eventos.
8	`      <div className="footer-inner">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
9	`        <div className="footer-brand">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
10	`          <span className="brand-name">SOFTWARE FINANCIERO</span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
11	`          <p>Institución Universitaria Tecnológico de Antioquia — Medellín</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
12	`        </div>`	Cierra el elemento y delimita el contenido que pertenece a él.
13	`        <div className="footer-col">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
14	`          <h4>Proyecto</h4>`	Abre el elemento <h4>; sus atributos configuran contenido, accesibilidad o eventos.
15	`          <p>SOFTWARE FINANCIERO</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
16	`          <p>Año: 2026</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
17	`        </div>`	Cierra el elemento y delimita el contenido que pertenece a él.
18	`        <div className="footer-col">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
19	`          <h4>Universidad</h4>`	Abre el elemento <h4>; sus atributos configuran contenido, accesibilidad o eventos.
20	`          <p>Institución Universitaria Tecnológico de Antioquia</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
21	`        </div>`	Cierra el elemento y delimita el contenido que pertenece a él.
22	`      </div>`	Cierra el elemento y delimita el contenido que pertenece a él.
23	`      <div className="footer-bottom">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
24	`        <p>© {anioActual} SOFTWARE FINANCIERO. Todos los derechos reservados.</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
25	`      </div>`	Cierra el elemento y delimita el contenido que pertenece a él.
26	`    </footer>`	Cierra el elemento <footer> y delimita el contenido que pertenece a él.
27	`  )`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
28	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
29	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.

N.º	Código original	Qué hace esta línea
30	`export default Footer`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.

Financiero/src/components/Funcionalidades.css

Hoja de estilos de Funcionalidades: define presentación, distribución, estados visuales y adaptación de pantalla.

540 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`.funcionalidades {`	Abre las reglas visuales del selector .funcionalidades.
2	`  background: var(--teal-800);`	La propiedad background define el fondo; valor: var(--teal-800).
3	`}`	Cierra el bloque CSS actual.
4	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
5	`.func-grid {`	Abre las reglas visuales del selector .func-grid.
6	`  display: grid;`	La propiedad display define cómo se distribuyen los elementos; valor: grid.
7	`  grid-template-columns: repeat(2, minmax(0, 1fr));`	La propiedad grid-template-columns define las columnas de la cuadrícula; valor: repeat(2, minmax(0, 1fr)).
8	`  gap: 1.5rem;`	La propiedad gap define el espacio entre elementos; valor: 1.5rem.
9	`}`	Cierra el bloque CSS actual.
10	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
11	`.func-card {`	Abre las reglas visuales del selector .func-card.
12	`  background: var(--teal-950);`	La propiedad background define el fondo; valor: var(--teal-950).
13	`  border-radius: var(--radius-md);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-md).
14	`  padding: 1.8rem 1.5rem;`	La propiedad padding define el espacio interior; valor: 1.8rem 1.5rem.
15	`  transition: transform 0.15s ease, background-color 0.15s ease;`	La propiedad transition anima cambios entre estados; valor: transform 0.15s ease, background-color 0.15s ease.
16	`  min-width: 0;`	La propiedad min-width establece el ancho mínimo; valor: 0.
17	`}`	Cierra el bloque CSS actual.
18	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
19	`.func-card:hover {`	Abre las reglas visuales del selector .func-card:hover.
20	`  transform: translateY(-4px);`	La propiedad transform transforma visualmente el elemento; valor: translateY(-4px).
21	`  background: var(--teal-700);`	La propiedad background define el fondo; valor: var(--teal-700).
22	`}`	Cierra el bloque CSS actual.
23	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
24	`.func-icon {`	Abre las reglas visuales del selector .func-icon.
25	`  width: 42px;`	La propiedad width establece el ancho; valor: 42px.
26	`  height: 42px;`	La propiedad height establece la altura; valor: 42px.
27	`  border-radius: 50%;`	La propiedad border-radius redondea las esquinas; valor: 50%.
28	`  background: var(--cyan-500);`	La propiedad background define el fondo; valor: var(--cyan-500).
29	`  color: var(--teal-950);`	La propiedad color define el color del texto; valor: var(--teal-950).
30	`  font-family: var(--font-display);`	La propiedad font-family elige la tipografía; valor: var(--font-display).
31	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
32	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
33	`  justify-content: center;`	La propiedad justify-content distribuye elementos en el eje principal; valor: center.

N.º	Código original	Qué hace esta línea
34	`  font-size: 1.1rem;`	La propiedad font-size define el tamaño de letra; valor: 1.1rem.
35	`  margin-bottom: 1rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 1rem.
36	`}`	Cierra el bloque CSS actual.
37	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
38	`.func-card h3 {`	Abre las reglas visuales del selector .func-card h3.
39	`  color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
40	`  font-size: 1.15rem;`	La propiedad font-size define el tamaño de letra; valor: 1.15rem.
41	`  margin-bottom: 0.6rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 0.6rem.
42	`}`	Cierra el bloque CSS actual.
43	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
44	`.func-card p {`	Abre las reglas visuales del selector .func-card p.
45	`  color: #bfe2e6;`	La propiedad color define el color del texto; valor: #bfe2e6.
46	`  font-size: 0.92rem;`	La propiedad font-size define el tamaño de letra; valor: 0.92rem.
47	`  margin: 0;`	La propiedad margin define el espacio exterior; valor: 0.
48	`}`	Cierra el bloque CSS actual.
49	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
50	`.func-card-action {`	Abre las reglas visuales del selector .func-card-action.
51	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
52	`  flex-direction: column;`	La propiedad flex-direction define la dirección del flexbox; valor: column.
53	`  align-items: flex-start;`	La propiedad align-items alinea elementos en el eje transversal; valor: flex-start.
54	`  width: 100%;`	La propiedad width establece el ancho; valor: 100%.
55	`  border: 0;`	La propiedad border define el borde; valor: 0.
56	`  color: inherit;`	La propiedad color define el color del texto; valor: inherit.
57	`  font-family: inherit;`	La propiedad font-family elige la tipografía; valor: inherit.
58	`  text-align: left;`	La propiedad text-align alinea el texto; valor: left.
59	`  cursor: pointer;`	La propiedad cursor define el cursor del puntero; valor: pointer.
60	`}`	Cierra el bloque CSS actual.
61	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
62	`.func-card-action:focus-visible,`	Completa una regla CSS que participa en la presentación visual.
63	`.user-action-close:focus-visible {`	Abre las reglas visuales del selector .user-action-close:focus-visible.
64	`  outline: 3px solid var(--cyan-300);`	La propiedad outline establece la propiedad visual outline; valor: 3px solid var(--cyan-300).
65	`  outline-offset: 3px;`	La propiedad outline-offset establece la propiedad visual outline-offset; valor: 3px.
66	`}`	Cierra el bloque CSS actual.
67	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
68	`.income-toggle-label {`	Abre las reglas visuales del selector .income-toggle-label.
69	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
70	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.

N.º	Código original	Qué hace esta línea
71	`  gap: 0.45rem;`	La propiedad gap define el espacio entre elementos; valor: 0.45rem.
72	`  margin-top: 1.15rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 1.15rem.
73	`  color: var(--cyan-300);`	La propiedad color define el color del texto; valor: var(--cyan-300).
74	`  font-size: 0.85rem;`	La propiedad font-size define el tamaño de letra; valor: 0.85rem.
75	`  font-weight: 700;`	La propiedad font-weight define el grosor de letra; valor: 700.
76	`}`	Cierra el bloque CSS actual.
77	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
78	`.income-toggle-arrow {`	Abre las reglas visuales del selector .income-toggle-arrow.
79	`  display: inline-block;`	La propiedad display define cómo se distribuyen los elementos; valor: inline-block.
80	`  transition: transform 0.2s ease;`	La propiedad transition anima cambios entre estados; valor: transform 0.2s ease.
81	`}`	Cierra el bloque CSS actual.
82	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
83	`.income-toggle-arrow.open {`	Abre las reglas visuales del selector .income-toggle-arrow.open.
84	`  transform: rotate(180deg);`	La propiedad transform transforma visualmente el elemento; valor: rotate(180deg).
85	`}`	Cierra el bloque CSS actual.
86	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
87	`.func-card-title {`	Abre las reglas visuales del selector .func-card-title.
88	`  color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
89	`  font-family: var(--font-display);`	La propiedad font-family elige la tipografía; valor: var(--font-display).
90	`  font-size: 1.15rem;`	La propiedad font-size define el tamaño de letra; valor: 1.15rem.
91	`  margin-bottom: 0.6rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 0.6rem.
92	`  text-transform: uppercase;`	La propiedad text-transform establece la propiedad visual text-transform; valor: uppercase.
93	`}`	Cierra el bloque CSS actual.
94	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
95	`.func-card-description {`	Abre las reglas visuales del selector .func-card-description.
96	`  color: #bfe2e6;`	La propiedad color define el color del texto; valor: #bfe2e6.
97	`  font-size: 0.92rem;`	La propiedad font-size define el tamaño de letra; valor: 0.92rem.
98	`}`	Cierra el bloque CSS actual.
99	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
100	`.user-action-panel {`	Abre las reglas visuales del selector .user-action-panel.
101	`  margin-top: 2rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 2rem.
102	`  padding: 1.5rem;`	La propiedad padding define el espacio interior; valor: 1.5rem.
103	`  background: var(--teal-950);`	La propiedad background define el fondo; valor: var(--teal-950).
104	`  border-radius: var(--radius-md);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-md).
105	`  color: #bfe2e6;`	La propiedad color define el color del texto; valor: #bfe2e6.
106	`}`	Cierra el bloque CSS actual.

N.º	Código original	Qué hace esta línea
107	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
108	`.user-action-panel-abriendo {`	Abre las reglas visuales del selector .user-action-panel-abriendo.
109	`  overflow: hidden;`	La propiedad overflow controla el contenido que desborda; valor: hidden.
110	`  animation: user-action-panel-open 350ms ease-out both;`	La propiedad animation asocia una animación; valor: user-action-panel-open 350ms ease-out both.
111	`}`	Cierra el bloque CSS actual.
112	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
113	`.user-action-panel-cerrando {`	Abre las reglas visuales del selector .user-action-panel-cerrando.
114	`  overflow: hidden;`	La propiedad overflow controla el contenido que desborda; valor: hidden.
115	`  animation: user-action-panel-close 350ms ease-in forwards;`	La propiedad animation asocia una animación; valor: user-action-panel-close 350ms ease-in forwards.
116	`}`	Cierra el bloque CSS actual.
117	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
118	`@keyframes user-action-panel-open {`	Declara los pasos de una animación CSS que puede asociarse a un elemento.
119	`  from {`	Abre el paso «from» de una animación CSS.
120	`    height: 0;`	La propiedad height establece la altura; valor: 0.
121	`    margin-top: 0;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 0.
122	`    padding-top: 0;`	La propiedad padding-top establece la propiedad visual padding-top; valor: 0.
123	`    padding-bottom: 0;`	La propiedad padding-bottom establece la propiedad visual padding-bottom; valor: 0.
124	`    opacity: 0;`	La propiedad opacity define la transparencia; valor: 0.
125	`  }`	Cierra el bloque CSS actual.
126	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
127	`  to {`	Abre el paso «to» de una animación CSS.
128	`    height: var(--altura-panel);`	La propiedad height establece la altura; valor: var(--altura-panel).
129	`    margin-top: 2rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 2rem.
130	`    padding: 1.5rem;`	La propiedad padding define el espacio interior; valor: 1.5rem.
131	`    opacity: 1;`	La propiedad opacity define la transparencia; valor: 1.
132	`  }`	Cierra el bloque CSS actual.
133	`}`	Cierra el bloque CSS actual.
134	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
135	`@keyframes user-action-panel-close {`	Declara los pasos de una animación CSS que puede asociarse a un elemento.
136	`  from {`	Abre el paso «from» de una animación CSS.
137	`    height: var(--altura-panel);`	La propiedad height establece la altura; valor: var(--altura-panel).
138	`    margin-top: 2rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 2rem.
139	`    padding: 1.5rem;`	La propiedad padding define el espacio interior; valor: 1.5rem.
140	`    opacity: 1;`	La propiedad opacity define la transparencia; valor: 1.
141	`  }`	Cierra el bloque CSS actual.
142	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.

N.º	Código original	Qué hace esta línea
143	`to {`	Abre el paso «to» de una animación CSS.
144	`height: 0;`	La propiedad height establece la altura; valor: 0.
145	`margin-top: 0;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 0.
146	`padding-top: 0;`	La propiedad padding-top establece la propiedad visual padding-top; valor: 0.
147	`padding-bottom: 0;`	La propiedad padding-bottom establece la propiedad visual padding-bottom; valor: 0.
148	`opacity: 0;`	La propiedad opacity define la transparencia; valor: 0.
149	`}`	Cierra el bloque CSS actual.
150	`}`	Cierra el bloque CSS actual.
151	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
152	`.user-action-heading {`	Abre las reglas visuales del selector .user-action-heading.
153	`display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
154	`justify-content: space-between;`	La propiedad justify-content distribuye elementos en el eje principal; valor: space-between.
155	`align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
156	`gap: 1rem;`	La propiedad gap define el espacio entre elementos; valor: 1rem.
157	`margin-bottom: 1rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 1rem.
158	`}`	Cierra el bloque CSS actual.
159	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
160	`.user-action-heading h3 {`	Abre las reglas visuales del selector .user-action-heading h3.
161	`color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
162	`}`	Cierra el bloque CSS actual.
163	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
164	`.user-action-close {`	Abre las reglas visuales del selector .user-action-close.
165	`color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
166	`background: transparent;`	La propiedad background define el fondo; valor: transparent.
167	`border: 1px solid rgba(255, 255, 255, 0.45);`	La propiedad border define el borde; valor: 1px solid rgba(255, 255, 255, 0.45).
168	`border-radius: var(--radius-sm);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-sm).
169	`font-size: 1.4rem;`	La propiedad font-size define el tamaño de letra; valor: 1.4rem.
170	`line-height: 1;`	La propiedad line-height define la altura de línea; valor: 1.
171	`cursor: pointer;`	La propiedad cursor define el cursor del puntero; valor: pointer.
172	`padding: 0.2rem 0.55rem;`	La propiedad padding define el espacio interior; valor: 0.2rem 0.55rem.
173	`}`	Cierra el bloque CSS actual.
174	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
175	`.user-action-form {`	Abre las reglas visuales del selector .user-action-form.
176	`display: grid;`	La propiedad display define cómo se distribuyen los elementos; valor: grid.
177	`gap: 0.65rem;`	La propiedad gap define el espacio entre elementos; valor: 0.65rem.
178	`max-width: 560px;`	La propiedad max-width establece el ancho máximo; valor: 560px.

N.º	Código original	Qué hace esta línea
179	`}`	Cierra el bloque CSS actual.
180	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
181	`.user-action-form label {`	Abre las reglas visuales del selector .user-action-form label.
182	`  color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
183	`  font-weight: 600;`	La propiedad font-weight define el grosor de letra; valor: 600.
184	`}`	Cierra el bloque CSS actual.
185	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
186	`.user-action-form input,`	Completa una regla CSS que participa en la presentación visual.
187	`.user-action-form select {`	Abre las reglas visuales del selector .user-action-form select.
188	`  width: 100%;`	La propiedad width establece el ancho; valor: 100%.
189	`  min-width: 0;`	La propiedad min-width establece el ancho mínimo; valor: 0.
190	`  padding: 0.7rem 0.85rem;`	La propiedad padding define el espacio interior; valor: 0.7rem 0.85rem.
191	`  border: 1px solid #9fc4c9;`	La propiedad border define el borde; valor: 1px solid #9fc4c9.
192	`  border-radius: var(--radius-sm);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-sm).
193	`  background: var(--bg-light);`	La propiedad background define el fondo; valor: var(--bg-light).
194	`  color: var(--ink);`	La propiedad color define el color del texto; valor: var(--ink).
195	`  font: inherit;`	La propiedad font establece la propiedad visual font; valor: inherit.
196	`}`	Cierra el bloque CSS actual.
197	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
198	`.user-action-form .btn {`	Abre las reglas visuales del selector .user-action-form .btn.
199	`  justify-self: start;`	La propiedad justify-self establece la propiedad visual justify-self; valor: start.
200	`  margin-top: 0.5rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 0.5rem.
201	`}`	Cierra el bloque CSS actual.
202	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
203	`.income-registration {`	Abre las reglas visuales del selector .income-registration.
204	`  margin-top: 2rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 2rem.
205	`  padding: clamp(1.5rem, 4vw, 3.5rem);`	La propiedad padding define el espacio interior; valor: clamp(1.5rem, 4vw, 3.5rem).
206	`  border-radius: var(--radius-md);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-md).
207	`  background: #eef3f4;`	La propiedad background define el fondo; valor: #eef3f4.
208	`  color: #101719;`	La propiedad color define el color del texto; valor: #101719.
209	`}`	Cierra el bloque CSS actual.
210	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
211	`.income-registration-cerrando {`	Abre las reglas visuales del selector . income-registration-cerrando.
212	`  overflow: hidden;`	La propiedad overflow controla el contenido que desborda; valor: hidden.
213	`  animation: income-panel-close 350ms ease-in forwards;`	La propiedad animation asocia una animación; valor: income-panel-close 350ms ease-in forwards.
214	`}`	Cierra el bloque CSS actual.

N.º	Código original	Qué hace esta línea
215	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
216	.income-registration-abriendo {	Abre las reglas visuales del selector .income-registration-abriendo.
217	overflow: hidden;	La propiedad overflow controla el contenido que desborda; valor: hidden.
218	animation: income-panel-open 350ms ease-out both;	La propiedad animation asocia una animación; valor: income-panel-open 350ms ease-out both.
219	}	Cierra el bloque CSS actual.
220	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
221	@keyframes income-panel-open {	Declara los pasos de una animación CSS que puede asociarse a un elemento.
222	from {	Abre el paso «from» de una animación CSS.
223	height: 0;	La propiedad height establece la altura; valor: 0.
224	margin-top: 0;	La propiedad margin-top establece la propiedad visual margin-top; valor: 0.
225	padding-top: 0;	La propiedad padding-top establece la propiedad visual padding-top; valor: 0.
226	padding-bottom: 0;	La propiedad padding-bottom establece la propiedad visual padding-bottom; valor: 0.
227	opacity: 0;	La propiedad opacity define la transparencia; valor: 0.
228	}	Cierra el bloque CSS actual.
229	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
230	to {	Abre el paso «to» de una animación CSS.
231	height: var(--altura-income-panel);	La propiedad height establece la altura; valor: var(--altura-income-panel).
232	margin-top: 2rem;	La propiedad margin-top establece la propiedad visual margin-top; valor: 2rem.
233	padding: clamp(1.5rem, 4vw, 3.5rem);	La propiedad padding define el espacio interior; valor: clamp(1.5rem, 4vw, 3.5rem).
234	opacity: 1;	La propiedad opacity define la transparencia; valor: 1.
235	}	Cierra el bloque CSS actual.
236	}	Cierra el bloque CSS actual.
237	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
238	@keyframes income-panel-close {	Declara los pasos de una animación CSS que puede asociarse a un elemento.
239	from {	Abre el paso «from» de una animación CSS.
240	height: var(--altura-income-panel);	La propiedad height establece la altura; valor: var(--altura-income-panel).
241	margin-top: 2rem;	La propiedad margin-top establece la propiedad visual margin-top; valor: 2rem.
242	opacity: 1;	La propiedad opacity define la transparencia; valor: 1.
243	}	Cierra el bloque CSS actual.
244	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
245	to {	Abre el paso «to» de una animación CSS.
246	height: 0;	La propiedad height establece la altura; valor: 0.
247	margin-top: 0;	La propiedad margin-top establece la propiedad visual margin-top; valor: 0.
248	padding-top: 0;	La propiedad padding-top establece la propiedad visual padding-top; valor: 0.
249	padding-bottom: 0;	La propiedad padding-bottom establece la propiedad visual padding-bottom; valor: 0.

N.º	Código original	Qué hace esta línea
250	`    opacity: 0;`	La propiedad opacity define la transparencia; valor: 0.
251	`  }`	Cierra el bloque CSS actual.
252	`}`	Cierra el bloque CSS actual.
253	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
254	`.income-registration h3 {`	Abre las reglas visuales del selector .income-registration h3.
255	`  margin-bottom: 2.5rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 2.5rem.
256	`  color: #177f8a;`	La propiedad color define el color del texto; valor: #177f8a.
257	`  font-family: var(--font-body);`	La propiedad font-family elige la tipografía; valor: var(--font-body).
258	`  font-size: clamp(1.35rem, 3vw, 1.7rem);`	La propiedad font-size define el tamaño de letra; valor: clamp(1.35rem, 3vw, 1.7rem).
259	`  font-weight: 700;`	La propiedad font-weight define el grosor de letra; valor: 700.
260	`  text-align: center;`	La propiedad text-align alinea el texto; valor: center.
261	`  text-transform: uppercase;`	La propiedad text-transform establece la propiedad visual text-transform; valor: uppercase.
262	`}`	Cierra el bloque CSS actual.
263	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
264	`.income-registration-form {`	Abre las reglas visuales del selector .income-registration-form.
265	`  max-width: 720px;`	La propiedad max-width establece el ancho máximo; valor: 720px.
266	`  margin: 0 auto;`	La propiedad margin define el espacio exterior; valor: 0 auto.
267	`}`	Cierra el bloque CSS actual.
268	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
269	`.income-registration-fields {`	Abre las reglas visuales del selector .income-registration-fields.
270	`  display: grid;`	La propiedad display define cómo se distribuyen los elementos; valor: grid.
271	`  grid-template-columns: minmax(0, 1fr) minmax(0, 286px);`	La propiedad grid-template-columns define las columnas de la cuadrícula; valor: minmax(0, 1fr) minmax(0, 286px).
272	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
273	`  column-gap: 2rem;`	La propiedad column-gap establece la propiedad visual column-gap; valor: 2rem.
274	`  row-gap: 1.25rem;`	La propiedad row-gap establece la propiedad visual row-gap; valor: 1.25rem.
275	`}`	Cierra el bloque CSS actual.
276	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
277	`.income-registration-fields label {`	Abre las reglas visuales del selector .income-registration-fields label.
278	`  font-size: 1.1rem;`	La propiedad font-size define el tamaño de letra; valor: 1.1rem.
279	`  font-weight: 700;`	La propiedad font-weight define el grosor de letra; valor: 700.
280	`  text-transform: uppercase;`	La propiedad text-transform establece la propiedad visual text-transform; valor: uppercase.
281	`}`	Cierra el bloque CSS actual.
282	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
283	`.income-registration-fields input,`	Completa una regla CSS que participa en la presentación visual.
284	`.income-registration-fields select {`	Abre las reglas visuales del selector .income-registration-fields select.
285	`  width: 100%;`	La propiedad width establece el ancho; valor: 100%.

N.º	Código original	Qué hace esta línea
286	`  min-width: 0;`	La propiedad min-width establece el ancho mínimo; valor: 0.
287	`  height: 46px;`	La propiedad height establece la altura; valor: 46px.
288	`  padding: 0.6rem 1rem;`	La propiedad padding define el espacio interior; valor: 0.6rem 1rem.
289	`  border: 0;`	La propiedad border define el borde; valor: 0.
290	`  border-radius: 16px;`	La propiedad border-radius redondea las esquinas; valor: 16px.
291	`  outline-color: #177f8a;`	La propiedad outline-color establece la propiedad visual outline-color; valor: #177f8a.
292	`  background: #d9d9d9;`	La propiedad background define el fondo; valor: #d9d9d9.
293	`  color: #101719;`	La propiedad color define el color del texto; valor: #101719.
294	`  font: inherit;`	La propiedad font establece la propiedad visual font; valor: inherit.
295	`}`	Cierra el bloque CSS actual.
296	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
297	`.income-registration-fields input:disabled {`	Abre las reglas visuales del selector .income-registration-fields input:disabled.
298	`  cursor: not-allowed;`	La propiedad cursor define el cursor del puntero; valor: not-allowed.
299	`  opacity: 0.65;`	La propiedad opacity define la transparencia; valor: 0.65.
300	`}`	Cierra el bloque CSS actual.
301	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
302	`.income-registration-actions {`	Abre las reglas visuales del selector .income-registration-actions.
303	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
304	`  justify-content: center;`	La propiedad justify-content distribuye elementos en el eje principal; valor: center.
305	`  gap: 2rem;`	La propiedad gap define el espacio entre elementos; valor: 2rem.
306	`  margin-top: 3rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 3rem.
307	`}`	Cierra el bloque CSS actual.
308	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
309	`.income-registration-actions button {`	Abre las reglas visuales del selector .income-registration-actions button.
310	`  min-width: 132px;`	La propiedad min-width establece el ancho mínimo; valor: 132px.
311	`  padding: 0.75rem 1rem;`	La propiedad padding define el espacio interior; valor: 0.75rem 1rem.
312	`  border: 0;`	La propiedad border define el borde; valor: 0.
313	`  border-radius: 16px;`	La propiedad border-radius redondea las esquinas; valor: 16px.
314	`  color: #fff;`	La propiedad color define el color del texto; valor: #fff.
315	`  font-family: var(--font-body);`	La propiedad font-family elige la tipografía; valor: var(--font-body).
316	`  font-size: 1rem;`	La propiedad font-size define el tamaño de letra; valor: 1rem.
317	`  font-weight: 700;`	La propiedad font-weight define el grosor de letra; valor: 700.
318	`  text-transform: uppercase;`	La propiedad text-transform establece la propiedad visual text-transform; valor: uppercase.
319	`  cursor: pointer;`	La propiedad cursor define el cursor del puntero; valor: pointer.
320	`  transition: filter 0.15s ease, transform 0.15s ease;`	La propiedad transition anima cambios entre estados; valor: filter 0.15s ease, transform 0.15s ease.
321	`}`	Cierra el bloque CSS actual.
322	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.

N.º	Código original	Qué hace esta línea
323	`.income-registration-actions button:hover {`	Abre las reglas visuales del selector .income-registration-actions button:hover.
324	`  filter: brightness(0.94);`	La propiedad filter establece la propiedad visual filter; valor: brightness(0.94).
325	`  transform: translateY(-2px);`	La propiedad transform transforma visualmente el elemento; valor: translateY(-2px).
326	`}`	Cierra el bloque CSS actual.
327	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
328	`.income-cancel-button {`	Abre las reglas visuales del selector .income-cancel-button.
329	`  background: #3bb5d0;`	La propiedad background define el fondo; valor: #3bb5d0.
330	`}`	Cierra el bloque CSS actual.
331	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
332	`.income-save-button {`	Abre las reglas visuales del selector .income-save-button.
333	`  background: #118590;`	La propiedad background define el fondo; valor: #118590.
334	`}`	Cierra el bloque CSS actual.
335	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
336	`.income-registration-message {`	Abre las reglas visuales del selector . income-registration-message.
337	`  margin: 1rem 0 0;`	La propiedad margin define el espacio exterior; valor: 1rem 0 0.
338	`  color: #12626c;`	La propiedad color define el color del texto; valor: #12626c.
339	`  text-align: center;`	La propiedad text-align alinea el texto; valor: center.
340	`}`	Cierra el bloque CSS actual.
341	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
342	`.income-registration-error {`	Abre las reglas visuales del selector .income-registration-error.
343	`  margin: 1rem 0 0;`	La propiedad margin define el espacio exterior; valor: 1rem 0 0.
344	`  color: #b42318;`	La propiedad color define el color del texto; valor: #b42318.
345	`  font-weight: 600;`	La propiedad font-weight define el grosor de letra; valor: 600.
346	`}`	Cierra el bloque CSS actual.
347	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
348	`.income-list-toggle {`	Abre las reglas visuales del selector .income-list-toggle.
349	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
350	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
351	`  justify-content: space-between;`	La propiedad justify-content distribuye elementos en el eje principal; valor: space-between.
352	`  gap: 1rem;`	La propiedad gap define el espacio entre elementos; valor: 1rem.
353	`  width: 100%;`	La propiedad width establece el ancho; valor: 100%.
354	`  margin-top: 1.5rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 1.5rem.
355	`  padding: 1rem 1.1rem;`	La propiedad padding define el espacio interior; valor: 1rem 1.1rem.
356	`  border: 1px solid #177f8a;`	La propiedad border define el borde; valor: 1px solid #177f8a.
357	`  border-radius: var(--radius-sm);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-sm).
358	`  color: #12626c;`	La propiedad color define el color del texto; valor: #12626c.
359	`  background: #e1eff0;`	La propiedad background define el fondo; valor: #e1eff0.

N.º	Código original	Qué hace esta línea
360	`  font-family: var(--font-body);`	La propiedad font-family elige la tipografía; valor: var(--font-body).
361	`  font-size: 1rem;`	La propiedad font-size define el tamaño de letra; valor: 1rem.
362	`  font-weight: 700;`	La propiedad font-weight define el grosor de letra; valor: 700.
363	`  text-align: left;`	La propiedad text-align alinea el texto; valor: left.
364	`  cursor: pointer;`	La propiedad cursor define el cursor del puntero; valor: pointer.
365	`}`	Cierra el bloque CSS actual.
366	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
367	`.income-list-toggle:hover {`	Abre las reglas visuales del selector .income-list-toggle:hover.
368	`  background: #d3e8ea;`	La propiedad background define el fondo; valor: #d3e8ea.
369	`}`	Cierra el bloque CSS actual.
370	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
371	`.income-list-toggle:focus-visible {`	Abre las reglas visuales del selector .income-list-toggle:focus-visible.
372	`  outline: 2px solid #177f8a;`	La propiedad outline establece la propiedad visual outline; valor: 2px solid #177f8a.
373	`  outline-offset: 3px;`	La propiedad outline-offset establece la propiedad visual outline-offset; valor: 3px.
374	`}`	Cierra el bloque CSS actual.
375	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
376	`.income-list {`	Abre las reglas visuales del selector .income-list.
377	`  margin-top: 1.5rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 1.5rem.
378	`  padding-top: 2rem;`	La propiedad padding-top establece la propiedad visual padding-top; valor: 2rem.
379	`  border-top: 1px solid rgba(18, 98, 108, 0.2);`	La propiedad border-top establece la propiedad visual border-top; valor: 1px solid rgba(18, 98, 108, 0.2).
380	`}`	Cierra el bloque CSS actual.
381	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
382	`.income-list h3 {`	Abre las reglas visuales del selector .income-list h3.
383	`  margin-bottom: 1.25rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 1.25rem.
384	`  color: #177f8a;`	La propiedad color define el color del texto; valor: #177f8a.
385	`  font-family: var(--font-body);`	La propiedad font-family elige la tipografía; valor: var(--font-body).
386	`  font-size: 1.25rem;`	La propiedad font-size define el tamaño de letra; valor: 1.25rem.
387	`  font-weight: 700;`	La propiedad font-weight define el grosor de letra; valor: 700.
388	`  text-align: left;`	La propiedad text-align alinea el texto; valor: left.
389	`}`	Cierra el bloque CSS actual.
390	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
391	`.income-table-wrapper {`	Abre las reglas visuales del selector .income-table-wrapper.
392	`  overflow-x: auto;`	La propiedad overflow-x establece la propiedad visual overflow-x; valor: auto.
393	`}`	Cierra el bloque CSS actual.
394	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
395	`.income-table {`	Abre las reglas visuales del selector .income-table.
396	`  width: 100%;`	La propiedad width establece el ancho; valor: 100%.

N.º	Código original	Qué hace esta línea
397	`  border-collapse: collapse;`	La propiedad border-collapse establece la propiedad visual border-collapse; valor: collapse.
398	`  color: #243b3f;`	La propiedad color define el color del texto; valor: #243b3f.
399	`  text-align: left;`	La propiedad text-align alinea el texto; valor: left.
400	`}`	Cierra el bloque CSS actual.
401	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
402	`.income-table th,`	Completa una regla CSS que participa en la presentación visual.
403	`.income-table td {`	Abre las reglas visuales del selector .income-table td.
404	`  padding: 0.8rem;`	La propiedad padding define el espacio interior; valor: 0.8rem.
405	`  border-bottom: 1px solid #cbdcde;`	La propiedad border-bottom establece la propiedad visual border-bottom; valor: 1px solid #cbdcde.
406	`  vertical-align: top;`	La propiedad vertical-align establece la propiedad visual vertical-align; valor: top.
407	`}`	Cierra el bloque CSS actual.
408	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
409	`.income-table th {`	Abre las reglas visuales del selector .income-table th.
410	`  color: #12626c;`	La propiedad color define el color del texto; valor: #12626c.
411	`  white-space: nowrap;`	La propiedad white-space controla espacios y saltos de línea; valor: nowrap.
412	`}`	Cierra el bloque CSS actual.
413	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
414	`.income-row-actions {`	Abre las reglas visuales del selector .income-row-actions.
415	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
416	`  gap: 0.5rem;`	La propiedad gap define el espacio entre elementos; valor: 0.5rem.
417	`}`	Cierra el bloque CSS actual.
418	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
419	`.income-row-actions button {`	Abre las reglas visuales del selector .income-row-actions button.
420	`  padding: 0.4rem 0.65rem;`	La propiedad padding define el espacio interior; valor: 0.4rem 0.65rem.
421	`  border: 1px solid #177f8a;`	La propiedad border define el borde; valor: 1px solid #177f8a.
422	`  border-radius: var(--radius-sm);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-sm).
423	`  color: #12626c;`	La propiedad color define el color del texto; valor: #12626c.
424	`  background: transparent;`	La propiedad background define el fondo; valor: transparent.
425	`  font-family: var(--font-body);`	La propiedad font-family elige la tipografía; valor: var(--font-body).
426	`  font-weight: 700;`	La propiedad font-weight define el grosor de letra; valor: 700.
427	`  cursor: pointer;`	La propiedad cursor define el cursor del puntero; valor: pointer.
428	`}`	Cierra el bloque CSS actual.
429	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
430	`.income-row-actions button:hover {`	Abre las reglas visuales del selector .income-row-actions button:hover.
431	`  color: #fff;`	La propiedad color define el color del texto; valor: #fff.
432	`  background: #177f8a;`	La propiedad background define el fondo; valor: #177f8a.
433	`}`	Cierra el bloque CSS actual.

N.º	Código original	Qué hace esta línea
434	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
435	`.income-row-actions .income-delete-button {`	Abre las reglas visuales del selector .income-row-actions .income-delete-button.
436	`  border-color: #b42318;`	La propiedad border-color establece la propiedad visual border-color; valor: #b42318.
437	`  color: #b42318;`	La propiedad color define el color del texto; valor: #b42318.
438	`}`	Cierra el bloque CSS actual.
439	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
440	`.income-row-actions .income-delete-button:hover {`	Abre las reglas visuales del selector .income-row-actions .income-delete-button:hover.
441	`  color: #fff;`	La propiedad color define el color del texto; valor: #fff.
442	`  background: #b42318;`	La propiedad background define el fondo; valor: #b42318.
443	`}`	Cierra el bloque CSS actual.
444	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
445	`.income-row-actions button:focus-visible {`	Abre las reglas visuales del selector .income-row-actions button:focus-visible.
446	`  outline: 2px solid #177f8a;`	La propiedad outline establece la propiedad visual outline; valor: 2px solid #177f8a.
447	`  outline-offset: 2px;`	La propiedad outline-offset establece la propiedad visual outline-offset; valor: 2px.
448	`}`	Cierra el bloque CSS actual.
449	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
450	`@media (prefers-reduced-motion: reduce) {`	Inicia una condición responsive; sus estilos se aplican cuando se cumple esta condición de pantalla.
451	`  .user-action-panel-abriendo,`	Completa una regla CSS que participa en la presentación visual.
452	`  .user-action-panel-cerrando,`	Completa una regla CSS que participa en la presentación visual.
453	`  .income-registration-abriendo,`	Completa una regla CSS que participa en la presentación visual.
454	`  .income-registration-cerrando {`	Abre las reglas visuales del selector .income-registration-cerrando.
455	`    animation-duration: 1ms;`	La propiedad animation-duration establece la propiedad visual animation-duration; valor: 1ms.
456	`  }`	Cierra el bloque CSS actual.
457	`}`	Cierra el bloque CSS actual.
458	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
459	`.user-action-error {`	Abre las reglas visuales del selector .user-action-error.
460	`  color: #ffaaaa;`	La propiedad color define el color del texto; valor: #ffaaaa.
461	`}`	Cierra el bloque CSS actual.
462	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
463	`.user-action-success {`	Abre las reglas visuales del selector .user-action-success.
464	`  color: var(--cyan-300);`	La propiedad color define el color del texto; valor: var(--cyan-300).
465	`}`	Cierra el bloque CSS actual.
466	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
467	`.user-table-wrapper {`	Abre las reglas visuales del selector .user-table-wrapper.
468	`  overflow-x: auto;`	La propiedad overflow-x establece la propiedad visual overflow-x; valor: auto.
469	`}`	Cierra el bloque CSS actual.

N.º	Código original	Qué hace esta línea
470	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
471	`.user-table {`	Abre las reglas visuales del selector .user-table.
472	`  width: 100%;`	La propiedad width establece el ancho; valor: 100%.
473	`  border-collapse: collapse;`	La propiedad border-collapse establece la propiedad visual border-collapse; valor: collapse.
474	`  color: #bfe2e6;`	La propiedad color define el color del texto; valor: #bfe2e6.
475	`  text-align: left;`	La propiedad text-align alinea el texto; valor: left.
476	`}`	Cierra el bloque CSS actual.
477	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
478	`.user-table th,`	Completa una regla CSS que participa en la presentación visual.
479	`.user-table td {`	Abre las reglas visuales del selector .user-table td.
480	`  padding: 0.75rem;`	La propiedad padding define el espacio interior; valor: 0.75rem.
481	`  border-bottom: 1px solid rgba(255, 255, 255, 0.16);`	La propiedad border-bottom establece la propiedad visual border-bottom; valor: 1px solid rgba(255, 255, 255, 0.16).
482	`}`	Cierra el bloque CSS actual.
483	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
484	`.user-table th {`	Abre las reglas visuales del selector .user-table th.
485	`  color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
486	`}`	Cierra el bloque CSS actual.
487	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
488	`.delete-confirm-dialog {`	Abre las reglas visuales del selector .delete-confirm-dialog.
489	`  width: max-content;`	La propiedad width establece el ancho; valor: max-content.
490	`  max-width: calc(100vw - 2rem);`	La propiedad max-width establece el ancho máximo; valor: calc(100vw - 2rem).
491	`}`	Cierra el bloque CSS actual.
492	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
493	`.delete-confirm-dialog p {`	Abre las reglas visuales del selector .delete-confirm-dialog p.
494	`  white-space: nowrap;`	La propiedad white-space controla espacios y saltos de línea; valor: nowrap.
495	`}`	Cierra el bloque CSS actual.
496	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
497	`.delete-confirm-actions {`	Abre las reglas visuales del selector .delete-confirm-actions.
498	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
499	`  justify-content: flex-end;`	La propiedad justify-content distribuye elementos en el eje principal; valor: flex-end.
500	`  gap: 0.5rem;`	La propiedad gap define el espacio entre elementos; valor: 0.5rem.
501	`}`	Cierra el bloque CSS actual.
502	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
503	`.delete-confirm-dialog button {`	Abre las reglas visuales del selector .delete-confirm-dialog button.
504	`  font-family: var(--font-body);`	La propiedad font-family elige la tipografía; valor: var(--font-body).
505	`}`	Cierra el bloque CSS actual.
506	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.

N.º	Código original	Qué hace esta línea
507	`@media (max-width: 640px) {`	Inicia una condición responsive; sus estilos se aplican cuando se cumple esta condición de pantalla.
508	`.func-grid {`	Abre las reglas visuales del selector .func-grid.
509	`grid-template-columns: 1fr;`	La propiedad grid-template-columns define las columnas de la cuadrícula; valor: 1fr.
510	`}`	Cierra el bloque CSS actual.
511	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
512	`.income-registration-fields {`	Abre las reglas visuales del selector .income-registration-fields.
513	`grid-template-columns: 1fr;`	La propiedad grid-template-columns define las columnas de la cuadrícula; valor: 1fr.
514	`row-gap: 0.5rem;`	La propiedad row-gap establece la propiedad visual row-gap; valor: 0.5rem.
515	`}`	Cierra el bloque CSS actual.
516	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
517	`.income-registration-fields label:not(:first-child) {`	Abre las reglas visuales del selector .income-registration-fields label:not(:first-child).
518	`margin-top: 0.75rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 0.75rem.
519	`}`	Cierra el bloque CSS actual.
520	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
521	`.income-registration-actions {`	Abre las reglas visuales del selector .income-registration-actions.
522	`gap: 0.75rem;`	La propiedad gap define el espacio entre elementos; valor: 0.75rem.
523	`margin-top: 2rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 2rem.
524	`}`	Cierra el bloque CSS actual.
525	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
526	`.income-registration-actions button {`	Abre las reglas visuales del selector .income-registration-actions button.
527	`min-width: 0;`	La propiedad min-width establece el ancho mínimo; valor: 0.
528	`flex: 1;`	La propiedad flex define el crecimiento y tamaño flexible; valor: 1.
529	`}`	Cierra el bloque CSS actual.
530	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
531	`@media (prefers-reduced-motion: reduce) {`	Inicia una condición responsive; sus estilos se aplican cuando se cumple esta condición de pantalla.
532	`.user-action-panel-cerrando {`	Abre las reglas visuales del selector .user-action-panel-cerrando.
533	`animation-duration: 1ms;`	La propiedad animation-duration establece la propiedad visual animation-duration; valor: 1ms.
534	`}`	Cierra el bloque CSS actual.
535	`}`	Cierra el bloque CSS actual.
536	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
537	`.delete-confirm-dialog p {`	Abre las reglas visuales del selector .delete-confirm-dialog p.
538	`white-space: normal;`	La propiedad white-space controla espacios y saltos de línea; valor: normal.
539	`}`	Cierra el bloque CSS actual.
540	`}`	Cierra el bloque CSS actual.

Financiero/src/components/Funcionalidades.jsx

Gestiona usuarios e ingresos mediante los endpoints protegidos /users e /ingresos y actualiza la interfaz.

980 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { useEffect, useLayoutEffect, useRef, useState } from 'react'`	Importa { useEffect, useLayoutEffect, useRef, useState } desde react y conecta este archivo con ese recurso.
2	`import apiClient from '../services/api.js'`	Importa apiClient desde ../services/api.js y conecta este archivo con ese recurso.
3	`import { notifyIncomeDataChanged } from '../services/incomeEvents.js'`	Importa { notifyIncomeDataChanged } desde ./services/incomeEvents.js y conecta este archivo con ese recurso.
4	`import './Funcionalidades.css'`	Importa './Funcionalidades.css' desde el módulo indicado y conecta este archivo con ese recurso.
5	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
6	`const funcionalidades = [`	Inicia en funcionalidades una estructura de datos que agrupa valores relacionados.
7	`  {`	Abre un bloque de código, objeto o expresión JSX.
8	`    icono: '$',`	Define el valor de la propiedad «icono» del objeto actual.
9	`    titulo: 'Ingreso de dinero',`	Define el valor de la propiedad «titulo» del objeto actual.
10	`    descripcion: 'Registra los pagos recibidos por cliente, cuenta bancaria y método de pago.',`	Define el valor de la propiedad «descripcion» del objeto actual.
11	`  },`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
12	`]`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
13	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
14	`const accionesUsuarios = [`	Inicia en accionesUsuarios una estructura de datos que agrupa valores relacionados.
15	`  {`	Abre un bloque de código, objeto o expresión JSX.
16	`    icono: (`	Define el valor de la propiedad «icono» del objeto actual.
17	`      <svg viewBox="0 0 24 24" width="24" height="24" fill="none" aria-hidden="true">`	Abre el elemento <svg>; sus atributos configuran contenido, accesibilidad o eventos.
18	`        <circle cx="10.8" cy="10.8" r="6.8" stroke="currentColor" strokeWidth="2.2" />`	Abre el elemento <circle>; sus atributos configuran contenido, accesibilidad o eventos.
19	`        <path d="m16 16 5 5" stroke="currentColor" strokeWidth="2.2" strokeLinecap="round" />`	Abre el elemento <path>; sus atributos configuran contenido, accesibilidad o eventos.
20	`      </svg>`	Cierra el elemento <svg> y delimita el contenido que pertenece a él.
21	`    ),`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
22	`    titulo: 'Consultar usuarios',`	Define el valor de la propiedad «titulo» del objeto actual.
23	`    descripcion: 'Revisa los usuarios registrados en el sistema.',`	Define el valor de la propiedad «descripcion» del objeto actual.
24	`    accion: 'consultar',`	Define el valor de la propiedad «accion» del objeto actual.
25	`  },`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
26	`  {`	Abre un bloque de código, objeto o expresión JSX.
27	`    icono: (`	Define el valor de la propiedad «icono» del objeto actual.
28	`      <svg viewBox="0 0 24 24" width="24" height="24" fill="none" aria-hidden="true">`	Abre el elemento <svg>; sus atributos configuran contenido, accesibilidad o eventos.
29	`        <path d="M4 7h16M10 11v6m4-6v6M6 7l1 14h10l1-14M9 7V4h6v3" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round" />`	Abre el elemento <path>; sus atributos configuran contenido, accesibilidad o eventos.

N.º	Código original	Qué hace esta línea
30	`</svg>`	Cierra el elemento <svg> y delimita el contenido que pertenece a él.
31	`),`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
32	`titulo: 'Eliminar usuarios',`	Define el valor de la propiedad «titulo» del objeto actual.
33	`descripcion: 'Selecciona y elimina un usuario registrado.',`	Define el valor de la propiedad «descripcion» del objeto actual.
34	`accion: 'eliminar',`	Define el valor de la propiedad «accion» del objeto actual.
35	`},`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
36	`{`	Abre un bloque de código, objeto o expresión JSX.
37	`icono: (`	Define el valor de la propiedad «icono» del objeto actual.
38	`<svg viewBox="0 0 24 24" width="24" height="24" fill="none" aria-hidden="true">`	Abre el elemento <svg>; sus atributos configuran contenido, accesibilidad o eventos.
39	`<path d="m15 5 4 4M4 20l4.2-.9L19.2 8a2.1 2.1 0 0 0-3-3L5.2 16 4 20Z" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round" />`	Abre el elemento <path>; sus atributos configuran contenido, accesibilidad o eventos.
40	`</svg>`	Cierra el elemento <svg> y delimita el contenido que pertenece a él.
41	`),`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
42	`titulo: 'Modificar usuarios',`	Define el valor de la propiedad «titulo» del objeto actual.
43	`descripcion: 'Edita los datos de un usuario existente.',`	Define el valor de la propiedad «descripcion» del objeto actual.
44	`accion: 'modificar',`	Define el valor de la propiedad «accion» del objeto actual.
45	`},`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
46	`]`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
47	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
48	`const API_URL = import.meta.env.VITE_API_URL`	Lee en API_URL una variable de configuración expuesta por Vite al frontend.
49	`const DEMORA_CIERRE_TARJETA = 10000`	Declara DEMORA_CIERRE_TARJETA y le asigna el valor o expresión mostrado para reutilizarlo.
50	`const CUENTA_NO_APLICA = 'No aplica'`	Declara CUENTA_NO_APLICA y le asigna el valor o expresión mostrado para reutilizarlo.
51	`const ingresoVacio = {`	Inicia en ingresoVacio una estructura de datos que agrupa valores relacionados.
52	`usuarioId: '',`	Define el valor de la propiedad «usuarioId» del objeto actual.
53	`cuentaBancaria: '',`	Define el valor de la propiedad «cuentaBancaria» del objeto actual.
54	`metodoPago: '',`	Define el valor de la propiedad «metodoPago» del objeto actual.
55	`fechaPago: '',`	Define el valor de la propiedad «fechaPago» del objeto actual.
56	`valor: '',`	Define el valor de la propiedad «valor» del objeto actual.
57	`observacion: '',`	Define el valor de la propiedad «observacion» del objeto actual.
58	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
59	`const formatoMoneda = new Intl.NumberFormat('es-CO', {`	Declara formatoMoneda y le asigna el valor o expresión mostrado para reutilizarlo.
60	`style: 'currency',`	Define el valor de la propiedad «style» del objeto actual.
61	`currency: 'COP',`	Define el valor de la propiedad «currency» del objeto actual.
62	`})`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
63	`const formatoFecha = new Intl.DateTimeFormat('es-CO', {`	Declara formatoFecha y le asigna el valor o expresión mostrado para reutilizarlo.

N.º	Código original	Qué hace esta línea
64	`  day: '2-digit',`	Define el valor de la propiedad «day» del objeto actual.
65	`  month: 'short',`	Define el valor de la propiedad «month» del objeto actual.
66	`  year: 'numeric',`	Define el valor de la propiedad «year» del objeto actual.
67	`  timeZone: 'UTC',`	Define el valor de la propiedad «timeZone» del objeto actual.
68	`})`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
69	`const fechaComoDiaCalendario = (valor) => {`	Define en fechaComoDiaCalendario una función flecha para reutilizar o ejecutar una acción.
70	`  const fecha = new Date(`${String(valor).slice(0, 10)}T00:00:00Z`)`	Declara fecha y le asigna el valor o expresión mostrado para reutilizarlo.
71	`  return Number.isNaN(fecha.getTime()) ? 'Fecha inválida' : formatoFecha.format(fecha)`	Devuelve un valor de la función o el elemento React que se renderizará.
72	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
73	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
74	`function Funcionalidades() {`	Declara la función Funcionalidades; el bloque siguiente contiene las instrucciones que ejecuta.
75	`  const dialogoConfirmacionRef = useRef(null)`	Crea una referencia persistente para conservar un valor o acceder a un elemento DOM.
76	`  const dialogoIngresoRef = useRef(null)`	Crea una referencia persistente para conservar un valor o acceder a un elemento DOM.
77	`  const panelIngresoRef = useRef(null)`	Crea una referencia persistente para conservar un valor o acceder a un elemento DOM.
78	`  const panelAccionRef = useRef(null)`	Crea una referencia persistente para conservar un valor o acceder a un elemento DOM.
79	`  const temporizadorCierreRef = useRef(null)`	Crea una referencia persistente para conservar un valor o acceder a un elemento DOM.
80	`  const temporizadorCierreIngresoRef = useRef(null)`	Crea una referencia persistente para conservar un valor o acceder a un elemento DOM.
81	`  const [accionActiva, setAccionActiva] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
82	`  const [ingresoAbierto, setIngresoAbierto] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
83	`  const [abriendoIngreso, setAbriendoIngreso] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
84	`  const [cerrandoIngreso, setCerrandoIngreso] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
85	`  const [alturaIngreso, setAlturaIngreso] = useState(0)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
86	`  const [listaIngresosAbierta, setListaIngresosAbierta] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
87	`  const [mensajeIngreso, setMensajeIngreso] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
88	`  const [errorIngreso, setErrorIngreso] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
89	`  const [errorListaIngresos, setErrorListaIngresos] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
90	`  const [ingresos, setIngresos] = useState([])`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
91	`  const [datosIngreso, setDatosIngreso] = useState(ingresoVacio)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
92	`  const [ingresoEditando, setIngresoEditando] = useState(null)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
93	`  const [ingresoAEliminar, setIngresoAEliminar] = useState(null)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
94	`  const [cargandoIngresos, setCargandoIngresos] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
95	`  const [guardandoIngreso, setGuardandoIngreso] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.

N.º	Código original	Qué hace esta línea
96	`  const [eliminandoIngreso, setEliminandoIngreso] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
97	`  const [usuarios, setUsuarios] = useState([])`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
98	`  const [cargandoClientes, setCargandoClientes] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
99	`  const [errorClientes, setErrorClientes] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
100	`  const [usuarioSeleccionado, setUsuarioSeleccionado] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
101	`  const [datosUsuario, setDatosUsuario] = useState({ name: '', lastName: '', email: '' })`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
102	`  const [cargando, setCargando] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
103	`  const [mostrarConfirmacion, setMostrarConfirmacion] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
104	`  const [abriendoPanel, setAbriendoPanel] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
105	`  const [cerrandoPanel, setCerrandoPanel] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
106	`  const [alturaPanel, setAlturaPanel] = useState(0)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
107	`  const [error, setError] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
108	`  const [mensaje, setMensaje] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
109	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
110	`  useEffect(`	Registra un efecto React para sincronizar la interfaz con eventos o recursos externos.
111	`    () => () => {`	Abre el bloque de código que contiene las instrucciones siguientes.
112	`      clearTimeout(temporizadorCierreRef.current)`	Invoca clearTimeout para ejecutar la operación definida por esa función o biblioteca.
113	`      clearTimeout(temporizadorCierreIngresoRef.current)`	Invoca clearTimeout para ejecutar la operación definida por esa función o biblioteca.
114	`    },`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
115	`    [],`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
116	`  )`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
117	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
118	`  useLayoutEffect(() => {`	Registra un efecto React para sincronizar la interfaz con eventos o recursos externos.
119	`    const panel = panelIngresoRef.current`	Declara panel y le asigna el valor o expresión mostrado para reutilizarlo.
120	`    if (!ingresoAbierto || !panel) return`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
121	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
122	`    setAlturaIngreso(panel.getBoundingClientRect().height)`	Invoca setAlturaIngreso para ejecutar la operación definida por esa función o biblioteca.
123	`    setAbriendoIngreso(true)`	Invoca setAbriendoIngreso para ejecutar la operación definida por esa función o biblioteca.
124	`  }, [ingresoAbierto])`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
125	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
126	`  useLayoutEffect(() => {`	Registra un efecto React para sincronizar la interfaz con eventos o recursos externos.
127	`    const panel = panelAccionRef.current`	Declara panel y le asigna el valor o expresión mostrado para reutilizarlo.

N.º	Código original	Qué hace esta línea
128	`    if (!accionActiva || !panel) return`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
129	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
130	`    setAlturaPanel(panel.getBoundingClientRect().height)`	Invoca setAlturaPanel para ejecutar la operación definida por esa función o biblioteca.
131	`    setAbriendoPanel(true)`	Invoca setAbriendoPanel para ejecutar la operación definida por esa función o biblioteca.
132	`  }, [accionActiva])`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
133	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
134	`  useEffect(() => {`	Registra un efecto React para sincronizar la interfaz con eventos o recursos externos.
135	`    const dialogo = dialogoConfirmacionRef.current`	Declara dialogo y le asigna el valor o expresión mostrado para reutilizarlo.
136	`    if (!dialogo) return`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
137	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
138	`    if (mostrarConfirmacion && !dialogo.open) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
139	`      dialogo.showModal()`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
140	`    } else if (!mostrarConfirmacion && dialogo.open) {`	Abre el bloque de código que contiene las instrucciones siguientes.
141	`      dialogo.close()`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
142	`    }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
143	`  }, [mostrarConfirmacion])`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
144	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
145	`  useEffect(() => {`	Registra un efecto React para sincronizar la interfaz con eventos o recursos externos.
146	`    const dialogo = dialogoIngresoRef.current`	Declara dialogo y le asigna el valor o expresión mostrado para reutilizarlo.
147	`    if (!dialogo) return`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
148	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
149	`    if (ingresoAEliminar && !dialogo.open) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
150	`      dialogo.showModal()`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
151	`    } else if (!ingresoAEliminar && dialogo.open) {`	Abre el bloque de código que contiene las instrucciones siguientes.
152	`      dialogo.close()`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
153	`    }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
154	`  }, [ingresoAEliminar])`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
155	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
156	`  const obtenerIngresos = async () => {`	Define en obtenerIngresos una función flecha para reutilizar o ejecutar una acción.
157	`    if (!API_URL) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
158	`      throw new Error('Falta configurar VITE_API_URL en el archivo .env del frontend.')`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
159	`    }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.

N.º	Código original	Qué hace esta línea
160	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
161	`const respuesta = await apiClient.get(`${API_URL.replace(/\/+$/, '')}/ingresos`)`	Espera una petición HTTP al backend antes de continuar.
162	`setIngresos(respuesta.data)`	Invoca setIngresos para ejecutar la operación definida por esa función o biblioteca.
163	`return respuesta.data`	Devuelve un valor de la función o el elemento React que se renderizará.
164	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
165	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
166	`const cerrarIngresoAutomaticamente = () => {`	Define en cerrarIngresoAutomaticamente una función flecha para reutilizar o ejecutar una acción.
167	`clearTimeout(temporizadorCierreIngresoRef.current)`	Invoca clearTimeout para ejecutar la operación definida por esa función o biblioteca.
168	`temporizadorCierreIngresoRef.current = setTimeout(() => {`	Abre el bloque de código que contiene las instrucciones siguientes.
169	`const panel = panelIngresoRef.current`	Declara panel y le asigna el valor o expresión mostrado para reutilizarlo.
170	`if (!panel) return`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
171	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
172	`setListaIngresosAbierta(false)`	Invoca setListaIngresosAbierta para ejecutar la operación definida por esa función o biblioteca.
173	`setAbriendoIngreso(false)`	Invoca setAbriendoIngreso para ejecutar la operación definida por esa función o biblioteca.
174	`setAlturaIngreso(panel.getBoundingClientRect().height)`	Invoca setAlturaIngreso para ejecutar la operación definida por esa función o biblioteca.
175	`setCerrandoIngreso(true)`	Invoca setCerrandoIngreso para ejecutar la operación definida por esa función o biblioteca.
176	`}, DEMORA_CIERRE_TARJETA)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
177	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
178	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
179	`const desplegarFormularioIngreso = async () => {`	Define en desplegarFormularioIngreso una función flecha para reutilizar o ejecutar una acción.
180	`clearTimeout(temporizadorCierreIngresoRef.current)`	Invoca clearTimeout para ejecutar la operación definida por esa función o biblioteca.
181	`if (ingresoAbierto) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
182	`setListaIngresosAbierta(false)`	Invoca setListaIngresosAbierta para ejecutar la operación definida por esa función o biblioteca.
183	`if (cerrandoIngreso) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
184	`setCerrandoIngreso(false)`	Invoca setCerrandoIngreso para ejecutar la operación definida por esa función o biblioteca.
185	`setAlturaIngreso(panelIngresoRef.current?.getBoundingClientRect().height || 0)`	Invoca setAlturaIngreso para ejecutar la operación definida por esa función o biblioteca.
186	`setAbriendoIngreso(true)`	Invoca setAbriendoIngreso para ejecutar la operación definida por esa función o biblioteca.
187	`} else {`	Abre el bloque de código que contiene las instrucciones siguientes.
188	`setAbriendoIngreso(false)`	Invoca setAbriendoIngreso para ejecutar la operación definida por esa función o biblioteca.
189	`setAlturaIngreso(panelIngresoRef.current?.getBoundingClientRect().height || 0)`	Invoca setAlturaIngreso para ejecutar la operación definida por esa función o biblioteca.

N.º	Código original	Qué hace esta línea
190	`setCerrandoIngreso(true)`	Invoca setCerrandoIngreso para ejecutar la operación definida por esa función o biblioteca.
191	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
192	`return`	Devuelve un valor de la función o el elemento React que se renderizará.
193	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
194	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
195	`setErrorClientes('')`	Invoca setErrorClientes para ejecutar la operación definida por esa función o biblioteca.
196	`setCargandoClientes(true)`	Invoca setCargandoClientes para ejecutar la operación definida por esa función o biblioteca.
197	`setCerrandoIngreso(false)`	Invoca setCerrandoIngreso para ejecutar la operación definida por esa función o biblioteca.
198	`setMensajeIngreso('')`	Invoca setMensajeIngreso para ejecutar la operación definida por esa función o biblioteca.
199	`try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
200	`await obtenerUsuarios()`	Espera la resolución de una operación asíncrona.
201	`} catch (errorSolicitud) {`	Abre el bloque de código que contiene las instrucciones siguientes.
202	`setErrorClientes(`	Invoca setErrorClientes para ejecutar la operación definida por esa función o biblioteca.
203	`errorSolicitud.response?.data?.message ||`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
204	`errorSolicitud.message ||`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
205	`'No fue posible consultar los usuarios.',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
206	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
207	`} finally {`	Abre el bloque de código que contiene las instrucciones siguientes.
208	`setCargandoClientes(false)`	Invoca setCargandoClientes para ejecutar la operación definida por esa función o biblioteca.
209	`setIngresoAbierto(true)`	Invoca setIngresoAbierto para ejecutar la operación definida por esa función o biblioteca.
210	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
211	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
212	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
213	`const manejarFinCierreIngreso = (evento) => {`	Define en manejarFinCierreIngreso una función flecha para reutilizar o ejecutar una acción.
214	`if (evento.target !== evento.currentTarget) return`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
215	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
216	`if (evento.animationName === 'income-panel-open') {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
217	`setAbriendoIngreso(false)`	Invoca setAbriendoIngreso para ejecutar la operación definida por esa función o biblioteca.
218	`} else if (evento.animationName === 'income-panel-close' && cerrandoIngreso) {`	Abre el bloque de código que contiene las instrucciones siguientes.
219	`setIngresoAbierto(false)`	Invoca setIngresoAbierto para ejecutar la operación definida por esa función o biblioteca.
220	`setAbriendoIngreso(false)`	Invoca setAbriendoIngreso para ejecutar la operación definida por esa función o biblioteca.
221	`setCerrandoIngreso(false)`	Invoca setCerrandoIngreso para ejecutar la operación definida por esa función o biblioteca.

N.º	Código original	Qué hace esta línea
222	`setAlturaIngreso(0)`	Invoca setAlturaIngreso para ejecutar la operación definida por esa función o biblioteca.
223	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
224	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
225	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
226	`const desplegarListaIngresos = async () => {`	Define en desplegarListaIngresos una función flecha para reutilizar o ejecutar una acción.
227	`if (listaIngresosAbierta) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
228	`setListaIngresosAbierta(false)`	Invoca setListaIngresosAbierta para ejecutar la operación definida por esa función o biblioteca.
229	`return`	Devuelve un valor de la función o el elemento React que se renderizará.
230	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
231	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
232	`setListaIngresosAbierta(true)`	Invoca setListaIngresosAbierta para ejecutar la operación definida por esa función o biblioteca.
233	`setErrorListaIngresos('')`	Invoca setErrorListaIngresos para ejecutar la operación definida por esa función o biblioteca.
234	`setCargandoIngresos(true)`	Invoca setCargandoIngresos para ejecutar la operación definida por esa función o biblioteca.
235	`try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
236	`await obtenerIngresos()`	Espera la resolución de una operación asíncrona.
237	`} catch (errorSolicitud) {`	Abre el bloque de código que contiene las instrucciones siguientes.
238	`setErrorListaIngresos(`	Invoca setErrorListaIngresos para ejecutar la operación definida por esa función o biblioteca.
239	`errorSolicitud.response?.data?.message ||`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
240	`errorSolicitud.message ||`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
241	`'No fue posible consultar los ingresos.',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
242	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
243	`} finally {`	Abre el bloque de código que contiene las instrucciones siguientes.
244	`setCargandoIngresos(false)`	Invoca setCargandoIngresos para ejecutar la operación definida por esa función o biblioteca.
245	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
246	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
247	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
248	`const guardarIngreso = async (evento) => {`	Define en guardarIngreso una función flecha para reutilizar o ejecutar una acción.
249	`evento.preventDefault()`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
250	`setErrorIngreso('')`	Invoca setErrorIngreso para ejecutar la operación definida por esa función o biblioteca.
251	`setErrorListaIngresos('')`	Invoca setErrorListaIngresos para ejecutar la operación definida por esa función o biblioteca.
252	`setGuardandoIngreso(true)`	Invoca setGuardandoIngreso para ejecutar la operación definida por esa función o biblioteca.
253	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.

N.º	Código original	Qué hace esta línea
254	`try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
255	`if (!API_URL) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
256	`throw new Error('Falta configurar VITE_API_URL en el archivo .env del frontend.')`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
257	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
258	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
259	`const url = `${API_URL.replace(/\/+$/, '')}/ingresos``	Declara url y le asigna el valor o expresión mostrado para reutilizarlo.
260	`if (ingresoEditando) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
261	`await apiClient.put(`${url}/${encodeURIComponent(ingresoEditando)}`, datosIngreso)`	Espera una petición HTTP al backend antes de continuar.
262	`setMensajeIngreso('Ingreso actualizado correctamente.')`	Invoca setMensajeIngreso para ejecutar la operación definida por esa función o biblioteca.
263	`} else {`	Abre el bloque de código que contiene las instrucciones siguientes.
264	`await apiClient.post(url, datosIngreso)`	Espera una petición HTTP al backend antes de continuar.
265	`setMensajeIngreso('Ingreso guardado correctamente.')`	Invoca setMensajeIngreso para ejecutar la operación definida por esa función o biblioteca.
266	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
267	`notifyIncomeDataChanged()`	Invoca notifyIncomeDataChanged para ejecutar la operación definida por esa función o biblioteca.
268	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
269	`setDatosIngreso(ingresoVacio)`	Invoca setDatosIngreso para ejecutar la operación definida por esa función o biblioteca.
270	`setIngresoEditando(null)`	Invoca setIngresoEditando para ejecutar la operación definida por esa función o biblioteca.
271	`if (listaIngresosAbierta) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
272	`try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
273	`await obtenerIngresos()`	Espera la resolución de una operación asíncrona.
274	`} catch (errorSolicitud) {`	Abre el bloque de código que contiene las instrucciones siguientes.
275	`setErrorListaIngresos(`	Invoca setErrorListaIngresos para ejecutar la operación definida por esa función o biblioteca.
276	`errorSolicitud.response?.data?.message ||`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
277	`errorSolicitud.message ||`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
278	`'El ingreso se guardó, pero no fue posible actualizar la consulta.',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
279	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
280	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
281	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
282	`cerrarIngresoAutomaticamente()`	Invoca cerrarIngresoAutomaticamente para ejecutar la operación definida por esa función o biblioteca.
283	`} catch (errorSolicitud) {`	Abre el bloque de código que contiene las instrucciones siguientes.
284	`setErrorIngreso(`	Invoca setErrorIngreso para ejecutar la operación definida por esa función o biblioteca.

N.º	Código original	Qué hace esta línea
285	`errorSolicitud.response?.data?.message ||`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
286	`errorSolicitud.message ||`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
287	`'No fue posible guardar el ingreso.',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
288	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
289	`} finally {`	Abre el bloque de código que contiene las instrucciones siguientes.
290	`setGuardandoIngreso(false)`	Invoca setGuardandoIngreso para ejecutar la operación definida por esa función o biblioteca.
291	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
292	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
293	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
294	`const editarIngreso = (ingreso) => {`	Define en editarIngreso una función flecha para reutilizar o ejecutar una acción.
295	`setIngresoEditando(String(ingreso.id))`	Invoca setIngresoEditando para ejecutar la operación definida por esa función o biblioteca.
296	`setDatosIngreso({`	Abre el bloque de código que contiene las instrucciones siguientes.
297	`usuarioId: String(ingreso.usuarioId),`	Define el valor de la propiedad «usuarioId» del objeto actual.
298	`cuentaBancaria:`	Define el valor de la propiedad «cuentaBancaria» del objeto actual.
299	`ingreso.metodoPago === 'transferencia' ? ingreso.cuentaBancaria : CUENTA_NO_APLICA,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
300	`metodoPago: ingreso.metodoPago,`	Define el valor de la propiedad «metodoPago» del objeto actual.
301	`fechaPago: String(ingreso.fechaPago).slice(0, 10),`	Define el valor de la propiedad «fechaPago» del objeto actual.
302	`valor: String(ingreso.valor),`	Define el valor de la propiedad «valor» del objeto actual.
303	`observacion: ingreso.observacion ?? '',`	Define el valor de la propiedad «observacion» del objeto actual.
304	`})`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
305	`setErrorIngreso('')`	Invoca setErrorIngreso para ejecutar la operación definida por esa función o biblioteca.
306	`setMensajeIngreso('')`	Invoca setMensajeIngreso para ejecutar la operación definida por esa función o biblioteca.
307	`document.getElementById('income-client')?.focus()`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
308	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
309	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
310	`const confirmarEliminarIngreso = async () => {`	Define en confirmarEliminarIngreso una función flecha para reutilizar o ejecutar una acción.
311	`if (!ingresoAEliminar) return`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
312	`if (!API_URL) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
313	`setErrorIngreso('Falta configurar VITE_API_URL en el archivo .env del frontend.')`	Invoca setErrorIngreso para ejecutar la operación definida por esa función o biblioteca.
314	`setIngresoAEliminar(null)`	Invoca setIngresoAEliminar para ejecutar la operación definida por esa función o biblioteca.
315	`return`	Devuelve un valor de la función o el elemento React que se renderizará.
316	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.

N.º	Código original	Qué hace esta línea
317	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
318	setEliminandoIngreso(true)	Invoca setEliminandoIngreso para ejecutar la operación definida por esa función o biblioteca.
319	setErrorIngreso('')	Invoca setErrorIngreso para ejecutar la operación definida por esa función o biblioteca.
320	setMensajeIngreso('')	Invoca setMensajeIngreso para ejecutar la operación definida por esa función o biblioteca.
321	try {	Inicia un bloque protegido; los errores pueden procesarse en catch.
322	await apiClient.delete(	Espera una petición HTTP al backend antes de continuar.
323	`${API_URL.replace(/\/+$/, '')}/ingresos/${encodeURIComponent(ingresoAEliminar.id)}`,	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
324	)	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
325	notifyIncomeDataChanged()	Invoca notifyIncomeDataChanged para ejecutar la operación definida por esa función o biblioteca.
326	setIngresos((ingresosActuales) =>	Invoca setIngresos para ejecutar la operación definida por esa función o biblioteca.
327	ingresosActuales.filter((ingreso) => ingreso.id !== ingresoAEliminar.id),	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
328	)	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
329	if (String(ingresoEditando) === String(ingresoAEliminar.id)) {	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
330	setIngresoEditando(null)	Invoca setIngresoEditando para ejecutar la operación definida por esa función o biblioteca.
331	setDatosIngreso(ingresoVacio)	Invoca setDatosIngreso para ejecutar la operación definida por esa función o biblioteca.
332	}	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
333	setMensajeIngreso('Ingreso eliminado correctamente.')	Invoca setMensajeIngreso para ejecutar la operación definida por esa función o biblioteca.
334	setIngresoAEliminar(null)	Invoca setIngresoAEliminar para ejecutar la operación definida por esa función o biblioteca.
335	cerrarIngresoAutomaticamente()	Invoca cerrarIngresoAutomaticamente para ejecutar la operación definida por esa función o biblioteca.
336	} catch (errorSolicitud) {	Abre el bloque de código que contiene las instrucciones siguientes.
337	setIngresoAEliminar(null)	Invoca setIngresoAEliminar para ejecutar la operación definida por esa función o biblioteca.
338	setErrorIngreso(	Invoca setErrorIngreso para ejecutar la operación definida por esa función o biblioteca.
339	errorSolicitud.response?.data?.message ||	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
340	'No fue posible eliminar el ingreso.',	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
341	)	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
342	} finally {	Abre el bloque de código que contiene las instrucciones siguientes.
343	setEliminandoIngreso(false)	Invoca setEliminandoIngreso para ejecutar la operación definida por esa función o biblioteca.
344	}	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
345	}	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
346	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
347	const obtenerUsuarios = async () => {	Define en obtenerUsuarios una función flecha para reutilizar o ejecutar una acción.

N.º	Código original	Qué hace esta línea
348	`    if (!API_URL) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
349	`      throw new Error('Falta configurar VITE_API_URL en el archivo .env del frontend.')`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
350	`    }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
351	*(línea vacía)*	Línea en blanco que separa instrucciones o bloques del código.
352	`    const respuesta = await apiClient.get(`${API_URL.replace(/\/+$/, '')}/users`)`	Espera una petición HTTP al backend antes de continuar.
353	`    setUsuarios(respuesta.data)`	Invoca setUsuarios para ejecutar la operación definida por esa función o biblioteca.
354	`    return respuesta.data`	Devuelve un valor de la función o el elemento React que se renderizará.
355	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
356	*(línea vacía)*	Línea en blanco que separa instrucciones o bloques del código.
357	`  const abrirAccion = async (accion) => {`	Define en abrirAccion una función flecha para reutilizar o ejecutar una acción.
358	`    clearTimeout(temporizadorCierreRef.current)`	Invoca clearTimeout para ejecutar la operación definida por esa función o biblioteca.
359	`    setAccionActiva(accion)`	Invoca setAccionActiva para ejecutar la operación definida por esa función o biblioteca.
360	`    setAbriendoPanel(false)`	Invoca setAbriendoPanel para ejecutar la operación definida por esa función o biblioteca.
361	`    setCerrandoPanel(false)`	Invoca setCerrandoPanel para ejecutar la operación definida por esa función o biblioteca.
362	`    setUsuarioSeleccionado('')`	Invoca setUsuarioSeleccionado para ejecutar la operación definida por esa función o biblioteca.
363	`    setDatosUsuario({ name: '', lastName: '', email: '' })`	Invoca setDatosUsuario para ejecutar la operación definida por esa función o biblioteca.
364	`    setError('')`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
365	`    setMensaje('')`	Invoca setMensaje para ejecutar la operación definida por esa función o biblioteca.
366	`    setCargando(true)`	Invoca setCargando para ejecutar la operación definida por esa función o biblioteca.
367	*(línea vacía)*	Línea en blanco que separa instrucciones o bloques del código.
368	`    try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
369	`      await obtenerUsuarios()`	Espera la resolución de una operación asíncrona.
370	`      if (accion === 'consultar') {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
371	`        cerrarPanelDespuesDe(DEMORA_CIERRE_TARJETA)`	Invoca cerrarPanelDespuesDe para ejecutar la operación definida por esa función o biblioteca.
372	`      } else if (accion === 'eliminar' || accion === 'modificar') {`	Abre el bloque de código que contiene las instrucciones siguientes.
373	`        cerrarPanelDespuesDe(DEMORA_CIERRE_TARJETA)`	Invoca cerrarPanelDespuesDe para ejecutar la operación definida por esa función o biblioteca.
374	`      }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
375	`    } catch (errorSolicitud) {`	Abre el bloque de código que contiene las instrucciones siguientes.
376	`      setError(`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
377	`        errorSolicitud.response?.data?.message ||`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
378	`          errorSolicitud.message ||`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.

N.º	Código original	Qué hace esta línea
379	`'No fue posible conectar con el servidor.',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
380	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
381	`} finally {`	Abre el bloque de código que contiene las instrucciones siguientes.
382	`setCargando(false)`	Invoca setCargando para ejecutar la operación definida por esa función o biblioteca.
383	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
384	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
385	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
386	`const cerrarPanelDespuesDe = (demora = DEMORA_CIERRE_TARJETA) => {`	Define en cerrarPanelDespuesDe una función flecha para reutilizar o ejecutar una acción.
387	`clearTimeout(temporizadorCierreRef.current)`	Invoca clearTimeout para ejecutar la operación definida por esa función o biblioteca.
388	`temporizadorCierreRef.current = setTimeout(() => {`	Abre el bloque de código que contiene las instrucciones siguientes.
389	`setAbriendoPanel(false)`	Invoca setAbriendoPanel para ejecutar la operación definida por esa función o biblioteca.
390	`setAlturaPanel(panelAccionRef.current?.getBoundingClientRect().height || 0)`	Invoca setAlturaPanel para ejecutar la operación definida por esa función o biblioteca.
391	`setCerrandoPanel(true)`	Invoca setCerrandoPanel para ejecutar la operación definida por esa función o biblioteca.
392	`}, demora)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
393	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
394	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
395	`const manejarFinCierrePanel = (evento) => {`	Define en manejarFinCierrePanel una función flecha para reutilizar o ejecutar una acción.
396	`if (evento.target !== evento.currentTarget) return`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
397	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
398	`if (evento.animationName === 'user-action-panel-open') {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
399	`setAbriendoPanel(false)`	Invoca setAbriendoPanel para ejecutar la operación definida por esa función o biblioteca.
400	`} else if (evento.animationName === 'user-action-panel-close' && cerrandoPanel) {`	Abre el bloque de código que contiene las instrucciones siguientes.
401	`setAccionActiva('')`	Invoca setAccionActiva para ejecutar la operación definida por esa función o biblioteca.
402	`setAbriendoPanel(false)`	Invoca setAbriendoPanel para ejecutar la operación definida por esa función o biblioteca.
403	`setCerrandoPanel(false)`	Invoca setCerrandoPanel para ejecutar la operación definida por esa función o biblioteca.
404	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
405	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
406	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
407	`const seleccionarUsuario = (id) => {`	Define en seleccionarUsuario una función flecha para reutilizar o ejecutar una acción.
408	`clearTimeout(temporizadorCierreRef.current)`	Invoca clearTimeout para ejecutar la operación definida por esa función o biblioteca.
409	`setUsuarioSeleccionado(id)`	Invoca setUsuarioSeleccionado para ejecutar la operación definida por esa función o biblioteca.

N.º	Código original	Qué hace esta línea
410	`    const usuario = usuarios.find((elemento) => String(elemento.id) === id)`	Define en usuario una función flecha para reutilizar o ejecutar una acción.
411	`    setDatosUsuario(`	Invoca setDatosUsuario para ejecutar la operación definida por esa función o biblioteca.
412	`      usuario`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
413	`        ? { name: usuario.name, lastName: usuario.lastName, email: usuario.email }`	Completa una condición, encadenamiento, separador o expresión anterior.
414	`        : { name: '', lastName: '', email: '' },`	Completa una condición, encadenamiento, separador o expresión anterior.
415	`    )`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
416	`    setError('')`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
417	`    setMensaje('')`	Invoca setMensaje para ejecutar la operación definida por esa función o biblioteca.
418	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
419	`    if (!id && (accionActiva === 'eliminar' || accionActiva === 'modificar')) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
420	`      cerrarPanelDespuesDe(DEMORA_CIERRE_TARJETA)`	Invoca cerrarPanelDespuesDe para ejecutar la operación definida por esa función o biblioteca.
421	`    }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
422	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
423	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
424	`  const manejarEliminacion = async (evento) => {`	Define en manejarEliminacion una función flecha para reutilizar o ejecutar una acción.
425	`    evento.preventDefault()`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
426	`    if (!usuarioSeleccionado) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
427	`      setError('Selecciona un usuario para eliminar.')`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
428	`      return`	Devuelve un valor de la función o el elemento React que se renderizará.
429	`    }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
430	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
431	`    setError('')`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
432	`    setMostrarConfirmacion(true)`	Invoca setMostrarConfirmacion para ejecutar la operación definida por esa función o biblioteca.
433	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
434	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
435	`  const confirmarEliminacion = async () => {`	Define en confirmarEliminacion una función flecha para reutilizar o ejecutar una acción.
436	`    setCargando(true)`	Invoca setCargando para ejecutar la operación definida por esa función o biblioteca.
437	`    setError('')`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
438	`    setMensaje('')`	Invoca setMensaje para ejecutar la operación definida por esa función o biblioteca.
439	`    try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
440	`      await apiClient.delete(`	Espera una petición HTTP al backend antes de continuar.

N.º	Código original	Qué hace esta línea
441	`${API_URL.replace(/\/+$/, '')}/users/${encodeURIComponent(usuarioSeleccionado)}`,	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
442	)	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
443	setUsuarios((usuariosActuales) =>	Invoca setUsuarios para ejecutar la operación definida por esa función o biblioteca.
444	usuariosActuales.filter((elemento) => String(elemento.id) !== usuarioSeleccionado),	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
445	)	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
446	setMensaje('Usuario eliminado correctamente.')	Invoca setMensaje para ejecutar la operación definida por esa función o biblioteca.
447	setUsuarioSeleccionado('')	Invoca setUsuarioSeleccionado para ejecutar la operación definida por esa función o biblioteca.
448	setMostrarConfirmacion(false)	Invoca setMostrarConfirmacion para ejecutar la operación definida por esa función o biblioteca.
449	cerrarPanelDespuesDe()	Invoca cerrarPanelDespuesDe para ejecutar la operación definida por esa función o biblioteca.
450	} catch (errorSolicitud) {	Abre el bloque de código que contiene las instrucciones siguientes.
451	setMostrarConfirmacion(false)	Invoca setMostrarConfirmacion para ejecutar la operación definida por esa función o biblioteca.
452	setError(	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
453	errorSolicitud.response?.data?.message || 'No fue posible eliminar el usuario.',	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
454	)	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
455	} finally {	Abre el bloque de código que contiene las instrucciones siguientes.
456	setCargando(false)	Invoca setCargando para ejecutar la operación definida por esa función o biblioteca.
457	}	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
458	}	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
459	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
460	const manejarModificacion = async (evento) => {	Define en manejarModificacion una función flecha para reutilizar o ejecutar una acción.
461	evento.preventDefault()	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
462	if (!usuarioSeleccionado) {	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
463	setError('Selecciona un usuario para modificar.')	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
464	return	Devuelve un valor de la función o el elemento React que se renderizará.
465	}	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
466	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
467	setCargando(true)	Invoca setCargando para ejecutar la operación definida por esa función o biblioteca.
468	setError('')	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
469	setMensaje('')	Invoca setMensaje para ejecutar la operación definida por esa función o biblioteca.
470	try {	Inicia un bloque protegido; los errores pueden procesarse en catch.
471	const respuesta = await apiClient.put(	Espera una petición HTTP al backend antes de continuar.

N.º	Código original	Qué hace esta línea
472	`${API_URL.replace(/\/+$/, '')}/users/${encodeURIComponent(usuarioSeleccionado)}`,	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
473	datosUsuario,	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
474	)	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
475	setUsuarios((usuariosActuales) =>	Invoca setUsuarios para ejecutar la operación definida por esa función o biblioteca.
476	usuariosActuales.map((usuario) =>	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
477	String(usuario.id) === usuarioSeleccionado ? respuesta.data.user : usuario,	Invoca String para ejecutar la operación definida por esa función o biblioteca.
478	),	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
479	)	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
480	setMensaje('Usuario actualizado correctamente.')	Invoca setMensaje para ejecutar la operación definida por esa función o biblioteca.
481	cerrarPanelDespuesDe()	Invoca cerrarPanelDespuesDe para ejecutar la operación definida por esa función o biblioteca.
482	} catch (errorSolicitud) {	Abre el bloque de código que contiene las instrucciones siguientes.
483	setError(	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
484	errorSolicitud.response?.data?.message || 'No fue posible actualizar el usuario.',	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
485	)	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
486	} finally {	Abre el bloque de código que contiene las instrucciones siguientes.
487	setCargando(false)	Invoca setCargando para ejecutar la operación definida por esa función o biblioteca.
488	}	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
489	}	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
490	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
491	const usuarioPorEliminar = usuarios.find(	Declara usuarioPorEliminar y le asigna el valor o expresión mostrado para reutilizarlo.
492	(usuario) => String(usuario.id) === usuarioSeleccionado,	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
493	)	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
494	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
495	return (	Devuelve un valor de la función o el elemento React que se renderizará.
496	<section className="funcionalidades" id="funcionalidades">	Abre el elemento <section>; sus atributos configuran contenido, accesibilidad o eventos.
497		Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
498	Funcionalidades	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
499	<h2 className="section-title section-title-light">	Abre el elemento <h2>; sus atributos configuran contenido, accesibilidad o eventos.
500	Todo el control de tu dinero, en un solo panel	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
501	</h2>	Cierra el elemento <h2> y delimita el contenido que pertenece a él.
502	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.

N.º	Código original	Qué hace esta línea
503	`<div className="func-grid">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
504	`{funcionalidades.map((func) => (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
505	`func.titulo === 'Ingreso de dinero' ? (`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
506	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
507	`className="func-card func-card-action"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
508	`key={func.titulo}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
509	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
510	`onClick={desplegarFormularioIngreso}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
511	`aria-expanded={ingresoAbierto && !cerrandoIngreso}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
512	`aria-controls="income-registration"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
513	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
514	`<span className="func-icon" aria-hidden="true">{func.icono}</span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
515	`<span className="func-card-title">{func.titulo}</span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
516	`<span className="func-card-description">{func.descripcion}</span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
517	`<span className="income-toggle-label">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
518	`{ingresoAbierto ? 'Cerrar formulario' : 'Abrir formulario'}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
519	`<span className={`income-toggle-arrow${ingresoAbierto ? ' open' : ''}`} aria-hidden="true">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
520	`▾`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
521	`</span>`	Cierra el elemento y delimita el contenido que pertenece a él.
522	`</span>`	Cierra el elemento y delimita el contenido que pertenece a él.
523	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
524	`) : (`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
525	`<article className="func-card" key={func.titulo}>`	Abre el elemento <article>; sus atributos configuran contenido, accesibilidad o eventos.
526	`<div className="func-icon">{func.icono}</div>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
527	`<h3>{func.titulo}</h3>`	Abre el elemento <h3>; sus atributos configuran contenido, accesibilidad o eventos.
528	`<p>{func.descripcion}</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
529	`</article>`	Cierra el elemento <article> y delimita el contenido que pertenece a él.
530	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
531	`))}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
532	`{accionesUsuarios.map((func) => (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.

N.º	Código original	Qué hace esta línea
533	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
534	`className="func-card func-card-action"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
535	`key={func.accion}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
536	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
537	`onClick={() => abrirAccion(func.accion)}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
538	`aria-expanded={accionActiva === func.accion}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
539	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
540	`<span className="func-icon" aria-hidden="true">{func.icono}</span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
541	`<span className="func-card-title">{func.titulo}</span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
542	`<span className="func-card-description">{func.descripcion}</span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
543	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
544	`))}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
545	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
546	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
547	`{ingresoAbierto && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
548	`<section`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
549	`ref={panelIngresoRef}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
550	`className={`income-registration${abriendoIngreso ? ' income-registration-abriendo' : ''}${cerrandoIngreso ? ' income-registration-cerrando' : ''}`}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
551	`style={abriendoIngreso || cerrandoIngreso ? { '--altura-income-panel': `${alturaIngreso}px` } : undefined}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
552	`onAnimationEnd={manejarFinCierreIngreso}`	Conecta el evento onAnimationEnd del elemento visual con la función que responde a la interacción.
553	`id="income-registration"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
554	`aria-labelledby="income-registration-title"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
555	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
556	`<h3 id="income-registration-title">`	Abre el elemento <h3>; sus atributos configuran contenido, accesibilidad o eventos.
557	`{ingresoEditando ? 'Editar ingreso:' : 'Nuevo registro:'}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
558	`</h3>`	Cierra el elemento <h3> y delimita el contenido que pertenece a él.
559	`<form className="income-registration-form" onSubmit={guardarIngreso}>`	Conecta el evento onSubmit del elemento visual con la función que responde a la interacción.
560	`<div className="income-registration-fields">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
561	`<label htmlFor="income-client">Cliente:</label>`	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.

N.º	Código original	Qué hace esta línea
562	`<select`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
563	`id="income-client"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
564	`name="usuarioId"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
565	`required`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
566	`disabled={cargandoClientes || usuarios.length === 0}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
567	`value={datosIngreso.usuarioId}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
568	`onChange={(evento) =>`	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
569	`setDatosIngreso({ ...datosIngreso, usuarioId: evento.target.value })`	Invoca setDatosIngreso para ejecutar la operación definida por esa función o biblioteca.
570	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
571	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
572	`<option value="" disabled>`	Abre el elemento <option>; sus atributos configuran contenido, accesibilidad o eventos.
573	`{cargandoClientes`	Abre un bloque de código, objeto o expresión JSX.
574	`? 'Cargando usuarios...'`	Completa una condición, encadenamiento, separador o expresión anterior.
575	`: usuarios.length`	Completa una condición, encadenamiento, separador o expresión anterior.
576	`? 'Selecciona un cliente'`	Completa una condición, encadenamiento, separador o expresión anterior.
577	`: 'No hay usuarios registrados'}`	Completa una condición, encadenamiento, separador o expresión anterior.
578	`</option>`	Cierra el elemento <option> y delimita el contenido que pertenece a él.
579	`{usuarios.map((usuario) => (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
580	`<option key={usuario.id} value={usuario.id}>`	Abre el elemento <option>; sus atributos configuran contenido, accesibilidad o eventos.
581	`{usuario.name} {usuario.lastName} — {usuario.email}`	Abre un bloque de código, objeto o expresión JSX.
582	`</option>`	Cierra el elemento <option> y delimita el contenido que pertenece a él.
583	`))}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
584	`</select>`	Cierra el elemento <select> y delimita el contenido que pertenece a él.
585	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
586	`<label htmlFor="income-account">Cuenta bancaria:</label>`	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.
587	`<input`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
588	`id="income-account"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
589	`name="cuentaBancaria"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
590	`type="text"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
591	`inputMode="numeric"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
592	`minLength={15}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.

N.º	Código original	Qué hace esta línea
593	`maxLength={15}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
594	`pattern="[0-9]{15}"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
595	`title="Ingresa exactamente 15 dígitos numéricos."`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
596	`placeholder="15 dígitos"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
597	`required`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
598	`disabled={['efectivo', 'tarjeta'].includes(datosIngreso.metodoPago)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
599	`value={datosIngreso.cuentaBancaria}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
600	`onChange={(evento) =>`	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
601	`setDatosIngreso({ ...datosIngreso, cuentaBancaria: evento.target.value })`	Invoca setDatosIngreso para ejecutar la operación definida por esa función o biblioteca.
602	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
603	`/>`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
604	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
605	`<label htmlFor="income-method">Método de pago:</label>`	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.
606	`<select`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
607	`id="income-method"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
608	`name="metodoPago"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
609	`required`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
610	`value={datosIngreso.metodoPago}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
611	`onChange={(evento) => {`	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
612	`const metodoPago = evento.target.value`	Declara metodoPago y le asigna el valor o expresión mostrado para reutilizarlo.
613	`setDatosIngreso((datosActuales) => ({`	Abre el bloque de código que contiene las instrucciones siguientes.
614	`...datosActuales,`	Completa una condición, encadenamiento, separador o expresión anterior.
615	`metodoPago,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
616	`cuentaBancaria:`	Define el valor de la propiedad «cuentaBancaria» del objeto actual.
617	`metodoPago === 'transferencia'`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
618	`? datosActuales.cuentaBancaria === CUENTA_NO_APLICA`	Completa una condición, encadenamiento, separador o expresión anterior.
619	`? ''`	Completa una condición, encadenamiento, separador o expresión anterior.
620	`: datosActuales.cuentaBancaria`	Completa una condición, encadenamiento, separador o expresión anterior.
621	`: CUENTA_NO_APLICA,`	Completa una condición, encadenamiento, separador o expresión anterior.
622	`}))`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
623	`}}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.

N.º	Código original	Qué hace esta línea
624	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
625	`<option value="" disabled>Selecciona un método</option>`	Abre el elemento <option>; sus atributos configuran contenido, accesibilidad o eventos.
626	`<option value="transferencia">Transferencia</option>`	Abre el elemento <option>; sus atributos configuran contenido, accesibilidad o eventos.
627	`<option value="efectivo">Efectivo</option>`	Abre el elemento <option>; sus atributos configuran contenido, accesibilidad o eventos.
628	`<option value="tarjeta">Tarjeta</option>`	Abre el elemento <option>; sus atributos configuran contenido, accesibilidad o eventos.
629	`</select>`	Cierra el elemento <select> y delimita el contenido que pertenece a él.
630	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
631	`<label htmlFor="income-date">Fecha de pago:</label>`	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.
632	`<input`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
633	`id="income-date"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
634	`name="fechaPago"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
635	`type="date"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
636	`required`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
637	`value={datosIngreso.fechaPago}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
638	`onChange={(evento) =>`	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
639	`setDatosIngreso({ ..datosIngreso, fechaPago: evento.target.value })`	Invoca setDatosIngreso para ejecutar la operación definida por esa función o biblioteca.
640	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
641	`/>`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
642	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
643	`<label htmlFor="income-amount">Valor del ingreso:</label>`	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.
644	`<input`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
645	`id="income-amount"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
646	`name="valor"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
647	`type="number"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
648	`min="0.01"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
649	`max="999999999999.99"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
650	`step="0.01"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
651	`required`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
652	`value={datosIngreso.valor}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
653	`onChange={(evento) =>`	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
654	`setDatosIngreso({ ..datosIngreso, valor: evento.target.value })`	Invoca setDatosIngreso para ejecutar la operación definida por esa función o biblioteca.

N.º	Código original	Qué hace esta línea
655	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
656	`/>`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
657	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
658	`<label htmlFor="income-observation">Observación:</label>`	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.
659	`<input`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
660	`id="income-observation"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
661	`name="observacion"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
662	`maxLength={10000}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
663	`value={datosIngreso.observacion}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
664	`onChange={(evento) =>`	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
665	`setDatosIngreso({ ...datosIngreso, observacion: evento.target.value })`	Invoca setDatosIngreso para ejecutar la operación definida por esa función o biblioteca.
666	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
667	`/>`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
668	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
669	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
670	`{errorClientes && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
671	`<p className="income-registration-error" role="alert">{errorClientes}</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
672	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
673	`{!cargandoClientes && !errorClientes && usuarios.length === 0 && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
674	`<p className="income-registration-error" role="status">`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
675	`Registra un usuario antes de crear un ingreso.`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
676	`</p>`	Cierra el elemento <p> y delimita el contenido que pertenece a él.
677	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
678	`{errorIngreso && <p className="income-registration-error" role="alert">{errorIngreso}</p>}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
679	`{mensajeIngreso && <p className="income-registration-message" role="status">{mensajeIngreso}</p>}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
680	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
681	`<div className="income-registration-actions">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
682	`{ingresoEditando && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
683	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
684	`className="income-cancel-button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.

N.º	Código original	Qué hace esta línea
685	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
686	`onClick={() => {`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
687	`setIngresoEditando(null)`	Invoca setIngresoEditando para ejecutar la operación definida por esa función o biblioteca.
688	`setDatosIngreso(ingresoVacio)`	Invoca setDatosIngreso para ejecutar la operación definida por esa función o biblioteca.
689	`setErrorIngreso('')`	Invoca setErrorIngreso para ejecutar la operación definida por esa función o biblioteca.
690	`setMensajeIngreso('')`	Invoca setMensajeIngreso para ejecutar la operación definida por esa función o biblioteca.
691	`}}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
692	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
693	`Cancelar edición`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
694	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
695	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
696	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
697	`className="income-save-button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
698	`type="submit"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
699	`disabled={guardandoIngreso}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
700	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
701	`{guardandoIngreso`	Abre un bloque de código, objeto o expresión JSX.
702	`? 'Guardando...'`	Completa una condición, encadenamiento, separador o expresión anterior.
703	`: ingresoEditando`	Completa una condición, encadenamiento, separador o expresión anterior.
704	`? 'Guardar cambios'`	Completa una condición, encadenamiento, separador o expresión anterior.
705	`: 'Guardar'}`	Completa una condición, encadenamiento, separador o expresión anterior.
706	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
707	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
708	`</form>`	Cierra el elemento <form> y delimita el contenido que pertenece a él.
709	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
710	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
711	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
712	`className="income-list-toggle"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
713	`onClick={desplegarListaIngresos}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
714	`aria-expanded={listaIngresosAbierta}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
715	`aria-controls="income-list"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.

N.º	Código original	Qué hace esta línea
716	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
717	`<span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
718	`{listaIngresosAbierta ? 'Ocultar ingresos registrados' : 'Ver ingresos registrados'}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
719	`</span>`	Cierra el elemento y delimita el contenido que pertenece a él.
720	`<span`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
721	`className={`income-toggle-arrow${listaIngresosAbierta ? ' open' : ''}`}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
722	`aria-hidden="true"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
723	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
724	`▾`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
725	`</span>`	Cierra el elemento y delimita el contenido que pertenece a él.
726	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
727	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
728	`{listaIngresosAbierta && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
729	`<section className="income-list" id="income-list" aria-labelledby="income-list-title">`	Abre el elemento <section>; sus atributos configuran contenido, accesibilidad o eventos.
730	`<h3 id="income-list-title">Ingresos registrados</h3>`	Abre el elemento <h3>; sus atributos configuran contenido, accesibilidad o eventos.
731	`{cargandoIngresos && <p role="status">Consultando ingresos...</p>}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
732	`{errorListaIngresos && <p className="income-registration-error" role="alert">{errorListaIngresos}</p>}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
733	`{!cargandoIngresos && !errorListaIngresos && ingresos.length === 0 && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
734	`<p>Aún no hay ingresos registrados.</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
735	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
736	`{!cargandoIngresos && ingresos.length > 0 && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
737	`<div className="income-table-wrapper" >`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
738	`<table className="income-table">`	Abre el elemento <table>; sus atributos configuran contenido, accesibilidad o eventos.
739	`<thead>`	Abre el elemento <thead>; sus atributos configuran contenido, accesibilidad o eventos.
740	`<tr>`	Abre el elemento <tr>; sus atributos configuran contenido, accesibilidad o eventos.
741	`<th>Fecha</th>`	Abre el elemento <th>; sus atributos configuran contenido, accesibilidad o eventos.
742	`<th>Cliente</th>`	Abre el elemento <th>; sus atributos configuran contenido, accesibilidad o eventos.
743	`<th>Cuenta bancaria</th>`	Abre el elemento <th>; sus atributos configuran contenido, accesibilidad o eventos.
744	`<th>Método de pago</th>`	Abre el elemento <th>; sus atributos configuran contenido, accesibilidad o eventos.

N.º	Código original	Qué hace esta línea
745	`<th>Valor</th>`	Abre el elemento <th>; sus atributos configuran contenido, accesibilidad o eventos.
746	`<th>Observación</th>`	Abre el elemento <th>; sus atributos configuran contenido, accesibilidad o eventos.
747	`<th>Acciones</th>`	Abre el elemento <th>; sus atributos configuran contenido, accesibilidad o eventos.
748	`</tr>`	Cierra el elemento <tr> y delimita el contenido que pertenece a él.
749	`</thead>`	Cierra el elemento <thead> y delimita el contenido que pertenece a él.
750	`<tbody>`	Abre el elemento <tbody>; sus atributos configuran contenido, accesibilidad o eventos.
751	`{ingresos.map((ingreso) => (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
752	`<tr key={ingreso.id}>`	Abre el elemento <tr>; sus atributos configuran contenido, accesibilidad o eventos.
753	`<td>{fechaComoDiaCalendario(ingreso.fechaPago)}</td>`	Abre el elemento <td>; sus atributos configuran contenido, accesibilidad o eventos.
754	`<td>{ingreso.cliente}</td>`	Abre el elemento <td>; sus atributos configuran contenido, accesibilidad o eventos.
755	`<td>{ingreso.cuentaBancaria ?? CUENTA_NO_APLICA}</td>`	Abre el elemento <td>; sus atributos configuran contenido, accesibilidad o eventos.
756	`<td>{ingreso.metodoPago}</td>`	Abre el elemento <td>; sus atributos configuran contenido, accesibilidad o eventos.
757	`<td>{formatoMoneda.format(Number(ingreso.valor))}</td>`	Abre el elemento <td>; sus atributos configuran contenido, accesibilidad o eventos.
758	`<td>{ingreso.observacion || '—'}</td>`	Abre el elemento <td>; sus atributos configuran contenido, accesibilidad o eventos.
759	`<td>`	Abre el elemento <td>; sus atributos configuran contenido, accesibilidad o eventos.
760	`<div className="income-row-actions">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
761	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
762	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
763	`onClick={() => editarIngreso(ingreso)}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
764	`aria-label={`Editar ingreso de ${ingreso.cliente}`}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
765	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
766	`Editar`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
767	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
768	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
769	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
770	`className="income-delete-button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
771	`onClick={() => setIngresoAEliminar(ingreso)}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
772	`aria-label={`Eliminar ingreso de ${ingreso.cliente}`}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
773	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
774	`Eliminar`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.

N.º	Código original	Qué hace esta línea
775	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
776	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
777	`</td>`	Cierra el elemento <td> y delimita el contenido que pertenece a él.
778	`</tr>`	Cierra el elemento <tr> y delimita el contenido que pertenece a él.
779	`))}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
780	`</tbody>`	Cierra el elemento <tbody> y delimita el contenido que pertenece a él.
781	`</table>`	Cierra el elemento <table> y delimita el contenido que pertenece a él.
782	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
783	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
784	`</section>`	Cierra el elemento <section> y delimita el contenido que pertenece a él.
785	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
786	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
787	`<dialog`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
788	`ref={dialogoIngresoRef}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
789	`className="delete-confirm-dialog"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
790	`onCancel={() => setIngresoAEliminar(null)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
791	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
792	`{ingresoAEliminar && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
793	`<>`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
794	`<p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
795	`¿Eliminar el ingreso de <strong>{ingresoAEliminar.cliente}</strong> por{' '}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
796	`<strong>{formatoMoneda.format(Number(ingresoAEliminar.valor))}</strong>?`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
797	`</p>`	Cierra el elemento <p> y delimita el contenido que pertenece a él.
798	`<div className="delete-confirm-actions">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
799	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
800	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
801	`onClick={() => setIngresoAEliminar(null)}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
802	`disabled={eliminandoIngreso}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
803	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
804	`Cancelar`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
805	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.

N.º	Código original	Qué hace esta línea
806	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
807	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
808	`onClick={confirmarEliminarIngreso}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
809	`disabled={eliminandoIngreso}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
810	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
811	`{eliminandoIngreso ? 'Eliminando...' : 'Eliminar'}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
812	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
813	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
814	`</>`	Cierra un elemento de la plantilla JSX.
815	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
816	`</dialog>`	Cierra el elemento <dialog> y delimita el contenido que pertenece a él.
817	`</section>`	Cierra el elemento <section> y delimita el contenido que pertenece a él.
818	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
819	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
820	`{accionActiva && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
821	`<section`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
822	`ref={panelAccionRef}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
823	`className={`user-action-panel${abriendoPanel ? ' user-action-panel-abriendo' : ''}${cerrandoPanel ? ' user-action-panel-cerrando' : ''}`}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
824	`style={abriendoPanel || cerrandoPanel ? { '--altura-panel': `${alturaPanel}px` } : undefined}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
825	`onAnimationEnd={manejarFinCierrePanel}`	Conecta el evento onAnimationEnd del elemento visual con la función que responde a la interacción.
826	`aria-labelledby="user-action-title"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
827	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
828	`<div className="user-action-heading">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
829	`<h3 id="user-action-title">`	Abre el elemento <h3>; sus atributos configuran contenido, accesibilidad o eventos.
830	`{accionesUsuarios.find((accion) => accion.accion === accionActiva).titulo}`	Abre un bloque de código, objeto o expresión JSX.
831	`</h3>`	Cierra el elemento <h3> y delimita el contenido que pertenece a él.
832	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
833	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
834	`className="user-action-close"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
835	`onClick={() => {`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.

N.º	Código original	Qué hace esta línea
836	clearTimeout(temporizadorCierreRef.current)	Invoca clearTimeout para ejecutar la operación definida por esa función o biblioteca.
837	setAccionActiva('')	Invoca setAccionActiva para ejecutar la operación definida por esa función o biblioteca.
838	setCerrandoPanel(false)	Invoca setCerrandoPanel para ejecutar la operación definida por esa función o biblioteca.
839	}}	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
840	aria-label="Cerrar panel"	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
841	>	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
842	×	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
843	</button>	Cierra el elemento <button> y delimita el contenido que pertenece a él.
844		Cierra el elemento y delimita el contenido que pertenece a él.
845	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
846	{cargando && Cargando... }	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
847	{error && {error} }	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
848	{mensaje && {mensaje} }	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
849	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
850	{!cargando && !error && accionActiva === 'consultar' && (	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
851	usuarios.length === 0 ? (	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
852	No hay usuarios registrados.	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
853	) : (	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
854		Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
855	<table className="user-table">	Abre el elemento <table>; sus atributos configuran contenido, accesibilidad o eventos.
856	<thead>	Abre el elemento <thead>; sus atributos configuran contenido, accesibilidad o eventos.
857	<tr>	Abre el elemento <tr>; sus atributos configuran contenido, accesibilidad o eventos.
858	<th>Nombre</th>	Abre el elemento <th>; sus atributos configuran contenido, accesibilidad o eventos.
859	<th>Apellido</th>	Abre el elemento <th>; sus atributos configuran contenido, accesibilidad o eventos.
860	<th>Correo electrónico</th>	Abre el elemento <th>; sus atributos configuran contenido, accesibilidad o eventos.
861	</tr>	Cierra el elemento <tr> y delimita el contenido que pertenece a él.
862	</thead>	Cierra el elemento <thead> y delimita el contenido que pertenece a él.
863	<tbody>	Abre el elemento <tbody>; sus atributos configuran contenido, accesibilidad o eventos.
864	{usuarios.map((usuario) => (	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
865	<tr key={usuario.id}>	Abre el elemento <tr>; sus atributos configuran contenido, accesibilidad o eventos.
866	<td>{usuario.name}</td>	Abre el elemento <td>; sus atributos configuran contenido, accesibilidad o eventos.

N.º	Código original	Qué hace esta línea
867	`<td>{usuario.lastName}</td>`	Abre el elemento <td>; sus atributos configuran contenido, accesibilidad o eventos.
868	`<td>{usuario.email}</td>`	Abre el elemento <td>; sus atributos configuran contenido, accesibilidad o eventos.
869	`</tr>`	Cierra el elemento <tr> y delimita el contenido que pertenece a él.
870	`))}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
871	`</tbody>`	Cierra el elemento <tbody> y delimita el contenido que pertenece a él.
872	`</table>`	Cierra el elemento <table> y delimita el contenido que pertenece a él.
873	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
874	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
875	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
876	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
877	`{!cargando && (accionActiva === 'eliminar' || accionActiva === 'modificar') && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
878	`usuarios.length === 0 ? (`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
879	`<p>No hay usuarios registrados.</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
880	`) : (`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
881	`<form`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
882	`className="user-action-form"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
883	`onSubmit={accionActiva === 'eliminar' ? manejarEliminacion : manejarModificacion}`	Conecta el evento onSubmit del elemento visual con la función que responde a la interacción.
884	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
885	`<label htmlFor="usuarioSeleccionado">Usuario:</label>`	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.
886	`<select`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
887	`id="usuarioSeleccionado"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
888	`value={usuarioSeleccionado}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
889	`onChange={(evento) => seleccionarUsuario(evento.target.value)}`	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
890	`required`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
891	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
892	`<option value="">Selecciona un usuario</option>`	Abre el elemento <option>; sus atributos configuran contenido, accesibilidad o eventos.
893	`{usuarios.map((usuario) => (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
894	`<option key={usuario.id} value={usuario.id}>`	Abre el elemento <option>; sus atributos configuran contenido, accesibilidad o eventos.
895	`{usuario.name} {usuario.lastName} — {usuario.email}`	Abre un bloque de código, objeto o expresión JSX.
896	`</option>`	Cierra el elemento <option> y delimita el contenido que pertenece a él.

N.º	Código original	Qué hace esta línea
897	))}	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
898	</select>	Cierra el elemento <select> y delimita el contenido que pertenece a él.
899	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
900	{accionActiva === 'modificar' && usuarioSeleccionado && (	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
901	<>	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
902	<label htmlFor="nombreModificado">Nombre:</label>	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.
903	<input	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
904	id="nombreModificado"	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
905	value={datosUsuario.name}	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
906	onChange={(evento) =>	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
907	setDatosUsuario({ ...datosUsuario, name: evento.target.value })	Invoca setDatosUsuario para ejecutar la operación definida por esa función o biblioteca.
908	}	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
909	maxLength={50}	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
910	required	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
911	/>	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
912	<label htmlFor="apellidoModificado">Apellido:</label>	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.
913	<input	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
914	id="apellidoModificado"	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
915	value={datosUsuario.lastName}	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
916	onChange={(evento) =>	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
917	setDatosUsuario({ ...datosUsuario, lastName: evento.target.value })	Invoca setDatosUsuario para ejecutar la operación definida por esa función o biblioteca.
918	}	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
919	maxLength={50}	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
920	required	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
921	/>	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
922	<label htmlFor="correoModificado">Correo electrónico:</label>	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.
923	<input	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
924	id="correoModificado"	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
925	type="email"	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
926	value={datosUsuario.email}	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.

N.º	Código original	Qué hace esta línea
927	`onChange={(evento) =>`	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
928	`setDatosUsuario({ ..datosUsuario, email: evento.target.value })`	Invoca setDatosUsuario para ejecutar la operación definida por esa función o biblioteca.
929	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
930	`maxLength={100}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
931	`required`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
932	`/>`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
933	`</>`	Cierra un elemento de la plantilla JSX.
934	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
935	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
936	`<button className="btn btn-primary" type="submit" disabled={cargando}>`	Abre el elemento <button>; sus atributos configuran contenido, accesibilidad o eventos.
937	`{accionActiva === 'eliminar' ? 'Eliminar usuario' : 'Guardar cambios'}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
938	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
939	`</form>`	Cierra el elemento <form> y delimita el contenido que pertenece a él.
940	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
941	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
942	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
943	`<dialog`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
944	`ref={dialogoConfirmacionRef}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
945	`className="delete-confirm-dialog"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
946	`onCancel={() => setMostrarConfirmacion(false)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
947	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
948	`{accionActiva === 'eliminar' && usuarioPorEliminar && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
949	`<>`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
950	`<p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
951	`¿Eliminar a <strong>{usuarioPorEliminar.name} {usuarioPorEliminar.lastName}</strong>?`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
952	`</p>`	Cierra el elemento <p> y delimita el contenido que pertenece a él.
953	`<div className="delete-confirm-actions">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
954	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
955	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
956	`onClick={() => setMostrarConfirmacion(false)}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
957	`disabled={cargando}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.

N.º	Código original	Qué hace esta línea
958	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
959	`Cancelar`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
960	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
961	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
962	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
963	`onClick={confirmarEliminacion}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
964	`disabled={cargando}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
965	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
966	`{cargando ? 'Eliminando...' : 'Eliminar'}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
967	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
968	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
969	`</>`	Cierra un elemento de la plantilla JSX.
970	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
971	`</dialog>`	Cierra el elemento <dialog> y delimita el contenido que pertenece a él.
972	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
973	`</section>`	Cierra el elemento <section> y delimita el contenido que pertenece a él.
974	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
975	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
976	`</section>`	Cierra el elemento <section> y delimita el contenido que pertenece a él.
977	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
978	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
979	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
980	`export default Funcionalidades`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.

Financiero/src/components/Header.css

Hoja de estilos de Header: define presentación, distribución, estados visuales y adaptación de pantalla.

158 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`.site-header {`	Abre las reglas visuales del selector .site-header.
2	`  background: var(--teal-800);`	La propiedad background define el fondo; valor: var(--teal-800).
3	`  position: sticky;`	La propiedad position define el tipo de posicionamiento; valor: sticky.
4	`  top: 0;`	La propiedad top establece la propiedad visual top; valor: 0.
5	`  z-index: 50;`	La propiedad z-index define el orden de superposición; valor: 50.
6	`  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.12);`	La propiedad box-shadow añade sombra; valor: 0 2px 10px rgba(0, 0, 0, 0.12).
7	`}`	Cierra el bloque CSS actual.
8	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
9	`.header-inner {`	Abre las reglas visuales del selector .header-inner.
10	`  max-width: var(--container);`	La propiedad max-width establece el ancho máximo; valor: var(--container).
11	`  margin: 0 auto;`	La propiedad margin define el espacio exterior; valor: 0 auto.
12	`  padding: 1rem 1.5rem;`	La propiedad padding define el espacio interior; valor: 1rem 1.5rem.
13	`  display: grid;`	La propiedad display define cómo se distribuyen los elementos; valor: grid.
14	`  grid-template-columns: 1fr auto auto;`	La propiedad grid-template-columns define las columnas de la cuadrícula; valor: 1fr auto auto.
15	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
16	`  gap: 1rem;`	La propiedad gap define el espacio entre elementos; valor: 1rem.
17	`  position: relative;`	La propiedad position define el tipo de posicionamiento; valor: relative.
18	`}`	Cierra el bloque CSS actual.
19	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
20	`.site-header-guest .header-inner {`	Abre las reglas visuales del selector .site-header-guest .header-inner.
21	`  grid-template-columns: 1fr auto;`	La propiedad grid-template-columns define las columnas de la cuadrícula; valor: 1fr auto.
22	`}`	Cierra el bloque CSS actual.
23	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
24	`.brand {`	Abre las reglas visuales del selector .brand.
25	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
26	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
27	`  gap: 0.6rem;`	La propiedad gap define el espacio entre elementos; valor: 0.6rem.
28	`  justify-self: start;`	La propiedad justify-self establece la propiedad visual justify-self; valor: start.
29	`}`	Cierra el bloque CSS actual.
30	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
31	`.brand-mark {`	Abre las reglas visuales del selector .brand-mark.
32	`  display: inline-flex;`	La propiedad display define cómo se distribuyen los elementos; valor: inline-flex.

N.º	Código original	Qué hace esta línea
33	`}`	Cierra el bloque CSS actual.
34	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
35	`.brand-name {`	Abre las reglas visuales del selector .brand-name.
36	`  font-family: var(--font-display);`	La propiedad font-family elige la tipografía; valor: var(--font-display).
37	`  color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
38	`  font-size: 1.05rem;`	La propiedad font-size define el tamaño de letra; valor: 1.05rem.
39	`  letter-spacing: 0.02em;`	La propiedad letter-spacing establece la propiedad visual letter-spacing; valor: 0.02em.
40	`  text-transform: uppercase;`	La propiedad text-transform establece la propiedad visual text-transform; valor: uppercase.
41	`}`	Cierra el bloque CSS actual.
42	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
43	`.main-nav {`	Abre las reglas visuales del selector .main-nav.
44	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
45	`  justify-content: center;`	La propiedad justify-content distribuye elementos en el eje principal; valor: center.
46	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
47	`  gap: 1.8rem;`	La propiedad gap define el espacio entre elementos; valor: 1.8rem.
48	`  justify-self: center;`	La propiedad justify-self establece la propiedad visual justify-self; valor: center.
49	`}`	Cierra el bloque CSS actual.
50	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
51	`.main-nav a {`	Abre las reglas visuales del selector .main-nav a.
52	`  color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
53	`  text-decoration: none;`	La propiedad text-decoration establece la propiedad visual text-decoration; valor: none.
54	`  font-weight: 600;`	La propiedad font-weight define el grosor de letra; valor: 600.
55	`  font-size: 0.95rem;`	La propiedad font-size define el tamaño de letra; valor: 0.95rem.
56	`  padding: 0.4rem 0.1rem;`	La propiedad padding define el espacio interior; valor: 0.4rem 0.1rem.
57	`  border-bottom: 2px solid transparent;`	La propiedad border-bottom establece la propiedad visual border-bottom; valor: 2px solid transparent.
58	`  transition: border-color 0.15s ease, color 0.15s ease;`	La propiedad transition anima cambios entre estados; valor: border-color 0.15s ease, color 0.15s ease.
59	`  white-space: nowrap;`	La propiedad white-space controla espacios y saltos de línea; valor: nowrap.
60	`}`	Cierra el bloque CSS actual.
61	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
62	`.main-nav a:hover {`	Abre las reglas visuales del selector .main-nav a:hover.
63	`  color: var(--cyan-300);`	La propiedad color define el color del texto; valor: var(--cyan-300).
64	`  border-bottom-color: var(--cyan-300);`	La propiedad border-bottom-color establece la propiedad visual border-bottom-color; valor: var(--cyan-300).
65	`}`	Cierra el bloque CSS actual.
66	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
67	`.main-nav a.active {`	Abre las reglas visuales del selector .main-nav a.active.

N.º	Código original	Qué hace esta línea
68	`  color: var(--cyan-300);`	La propiedad color define el color del texto; valor: var(--cyan-300).
69	`  border-bottom-color: var(--cyan-300);`	La propiedad border-bottom-color establece la propiedad visual border-bottom-color; valor: var(--cyan-300).
70	`}`	Cierra el bloque CSS actual.
71	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
72	`.main-nav a.main-nav-api {`	Abre las reglas visuales del selector .main-nav a.main-nav-api.
73	`  opacity: 0.55;`	La propiedad opacity define la transparencia; valor: 0.55.
74	`  font-size: 0.82rem;`	La propiedad font-size define el tamaño de letra; valor: 0.82rem.
75	`}`	Cierra el bloque CSS actual.
76	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
77	`.main-nav a.main-nav-api:hover,`	Completa una regla CSS que participa en la presentación visual.
78	`.main-nav a.main-nav-api:focus-visible,`	Completa una regla CSS que participa en la presentación visual.
79	`.main-nav a.main-nav-api.active {`	Abre las reglas visuales del selector .main-nav a.main-nav-api.active.
80	`  opacity: 1;`	La propiedad opacity define la transparencia; valor: 1.
81	`}`	Cierra el bloque CSS actual.
82	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
83	`.header-actions {`	Abre las reglas visuales del selector .header-actions.
84	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
85	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
86	`  justify-self: end;`	La propiedad justify-self establece la propiedad visual justify-self; valor: end.
87	`  gap: 0.75rem;`	La propiedad gap define el espacio entre elementos; valor: 0.75rem.
88	`}`	Cierra el bloque CSS actual.
89	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
90	`.login-toggle {`	Abre las reglas visuales del selector .login-toggle.
91	`  display: inline-flex;`	La propiedad display define cómo se distribuyen los elementos; valor: inline-flex.
92	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
93	`  justify-content: center;`	La propiedad justify-content distribuye elementos en el eje principal; valor: center.
94	`  padding: 0.55rem 0.85rem;`	La propiedad padding define el espacio interior; valor: 0.55rem 0.85rem.
95	`  color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
96	`  background: var(--teal-950);`	La propiedad background define el fondo; valor: var(--teal-950).
97	`  border: 1px solid var(--cyan-300);`	La propiedad border define el borde; valor: 1px solid var(--cyan-300).
98	`  border-radius: var(--radius-sm);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-sm).
99	`  cursor: pointer;`	La propiedad cursor define el cursor del puntero; valor: pointer.
100	`  font-size: 0.85rem;`	La propiedad font-size define el tamaño de letra; valor: 0.85rem.
101	`  white-space: nowrap;`	La propiedad white-space controla espacios y saltos de línea; valor: nowrap.
102	`}`	Cierra el bloque CSS actual.

N.º	Código original	Qué hace esta línea
103	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
104	`.login-toggle:hover {`	Abre las reglas visuales del selector .login-toggle:hover.
105	`  color: var(--teal-950);`	La propiedad color define el color del texto; valor: var(--teal-950).
106	`  background: var(--cyan-300);`	La propiedad background define el fondo; valor: var(--cyan-300).
107	`}`	Cierra el bloque CSS actual.
108	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
109	`.nav-toggle {`	Abre las reglas visuales del selector .nav-toggle.
110	`  display: none;`	La propiedad display define cómo se distribuyen los elementos; valor: none.
111	`  flex-direction: column;`	La propiedad flex-direction define la dirección del flexbox; valor: column.
112	`  gap: 5px;`	La propiedad gap define el espacio entre elementos; valor: 5px.
113	`  background: none;`	La propiedad background define el fondo; valor: none.
114	`  border: none;`	La propiedad border define el borde; valor: none.
115	`  cursor: pointer;`	La propiedad cursor define el cursor del puntero; valor: pointer.
116	`  padding: 0.4rem;`	La propiedad padding define el espacio interior; valor: 0.4rem.
117	`}`	Cierra el bloque CSS actual.
118	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
119	`.nav-toggle span {`	Abre las reglas visuales del selector .nav-toggle span.
120	`  width: 24px;`	La propiedad width establece el ancho; valor: 24px.
121	`  height: 3px;`	La propiedad height establece la altura; valor: 3px.
122	`  background: var(--white);`	La propiedad background define el fondo; valor: var(--white).
123	`  border-radius: 2px;`	La propiedad border-radius redondea las esquinas; valor: 2px.
124	`}`	Cierra el bloque CSS actual.
125	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
126	`@media (max-width: 860px) {`	Inicia una condición responsive; sus estilos se aplican cuando se cumple esta condición de pantalla.
127	`  .header-inner {`	Abre las reglas visuales del selector .header-inner.
128	`    grid-template-columns: 1fr auto;`	La propiedad grid-template-columns define las columnas de la cuadrícula; valor: 1fr auto.
129	`  }`	Cierra el bloque CSS actual.
130	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
131	`  .nav-toggle {`	Abre las reglas visuales del selector .nav-toggle.
132	`    display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
133	`    justify-self: end;`	La propiedad justify-self establece la propiedad visual justify-self; valor: end.
134	`  }`	Cierra el bloque CSS actual.
135	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
136	`  .main-nav {`	Abre las reglas visuales del selector .main-nav.
137	`    position: absolute;`	La propiedad position define el tipo de posicionamiento; valor: absolute.
138	`    top: 100%;`	La propiedad top establece la propiedad visual top; valor: 100%.
139	`    left: 0;`	La propiedad left establece la propiedad visual left; valor: 0.
140	`    right: 0;`	La propiedad right establece la propiedad visual right; valor: 0.

N.º	Código original	Qué hace esta línea
141	`background: var(--teal-800);`	La propiedad background define el fondo; valor: var(--teal-800).
142	`flex-direction: column;`	La propiedad flex-direction define la dirección del flexbox; valor: column.
143	`gap: 0;`	La propiedad gap define el espacio entre elementos; valor: 0.
144	`max-height: 0;`	La propiedad max-height establece la altura máxima; valor: 0.
145	`overflow: hidden;`	La propiedad overflow controla el contenido que desborda; valor: hidden.
146	`transition: max-height 0.25s ease;`	La propiedad transition anima cambios entre estados; valor: max-height 0.25s ease.
147	`justify-self: stretch;`	La propiedad justify-self establece la propiedad visual justify-self; valor: stretch.
148	`}`	Cierra el bloque CSS actual.
149	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
150	`.main-nav.open {`	Abre las reglas visuales del selector .main-nav.open.
151	`max-height: 300px;`	La propiedad max-height establece la altura máxima; valor: 300px.
152	`}`	Cierra el bloque CSS actual.
153	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
154	`.main-nav a {`	Abre las reglas visuales del selector .main-nav a.
155	`padding: 1rem 1.5rem;`	La propiedad padding define el espacio interior; valor: 1rem 1.5rem.
156	`border-bottom: 1px solid rgba(255, 255, 255, 0.08);`	La propiedad border-bottom establece la propiedad visual border-bottom; valor: 1px solid rgba(255, 255, 255, 0.08).
157	`}`	Cierra el bloque CSS actual.
158	`}`	Cierra el bloque CSS actual.

Financiero/src/components/Header.jsx

Componente o página React Header: construye una parte de la interfaz y usa las props y estados del módulo.

82 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { useState } from 'react'`	Importa { useState } desde react y conecta este archivo con ese recurso.
2	`import { NavLink } from 'react-router-dom'`	Importa { NavLink } desde react-router-dom y conecta este archivo con ese recurso.
3	`import './Header.css'`	Importa './Header.css' desde el módulo indicado y conecta este archivo con ese recurso.
4	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
5	`function Header({ usuario, onIniciarSesion, onCerrarSesion }) {`	Declara la función Header; el bloque siguiente contiene las instrucciones que ejecuta.
6	`  const [menuAbierto, setMenuAbierto] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
7	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
8	`  const enlaces = [`	Inicia en enlaces una estructura de datos que agrupa valores relacionados.
9	`    { to: '/inicio', texto: 'Inicio' },`	Abre un bloque de código, objeto o expresión JSX.
10	`    { to: '/', texto: 'Dashboard' },`	Abre un bloque de código, objeto o expresión JSX.
11	`    { to: '/funcionalidades', texto: 'Funcionalidades' },`	Abre un bloque de código, objeto o expresión JSX.
12	`    { to: '/proyecto', texto: 'Sobre nosotros' },`	Abre un bloque de código, objeto o expresión JSX.
13	`    { to: '/api-test', texto: 'API' },`	Abre un bloque de código, objeto o expresión JSX.
14	`  ]`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
15	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
16	`  return (`	Devuelve un valor de la función o el elemento React que se renderizará.
17	`    <header className={`site-header${usuario ? '' : ' site-header-guest'}`}>`	Abre el elemento <header>; sus atributos configuran contenido, accesibilidad o eventos.
18	`      <div className="header-inner">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
19	`        <div className="brand">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
20	`          <span className="brand-mark">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
21	`            <svg width="34" height="34" viewBox="0 0 48 48" fill="none" xmlns="http://www.w3.org/2000/svg">`	Abre el elemento <svg>; sus atributos configuran contenido, accesibilidad o eventos.
22	`              <path d="M6 24C6 14 14 6 24 6" stroke="#ffffff" strokeWidth="4" strokeLinecap="round" />`	Abre el elemento <path>; sus atributos configuran contenido, accesibilidad o eventos.
23	`              <path d="M42 24C42 34 34 42 24 42" stroke="#5fd6c4" strokeWidth="4" strokeLinecap="round" />`	Abre el elemento <path>; sus atributos configuran contenido, accesibilidad o eventos.
24	`              <path d="M10 30L20 20L27 27L40 12" stroke="#5fd6c4" strokeWidth="4" strokeLinecap="round" strokeLinejoin="round" />`	Abre el elemento <path>; sus atributos configuran contenido, accesibilidad o eventos.
25	`              <path d="M31 12H40V21" stroke="#5fd6c4" strokeWidth="4" strokeLinecap="round" strokeLinejoin="round" />`	Abre el elemento <path>; sus atributos configuran contenido, accesibilidad o eventos.
26	`            </svg>`	Cierra el elemento <svg> y delimita el contenido que pertenece a él.
27	`          </span>`	Cierra el elemento y delimita el contenido que pertenece a él.
28	`          <span className="brand-name">SOFTWARE FINANCIERO</span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.

N.º	Código original	Qué hace esta línea
29	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
30	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
31	`{usuario && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
32	`<nav className={`main-nav ${menuAbierto ? 'open' : ''}`}>`	Abre el elemento <nav>; sus atributos configuran contenido, accesibilidad o eventos.
33	`{enlaces.map((enlace) => (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
34	`<NavLink`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
35	`key={enlace.to}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
36	`to={enlace.to}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
37	`end={enlace.to === '/'}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
38	`className={({ isActive }) =>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
39	`[`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
40	`isActive ? 'active' : '',`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
41	`enlace.to === '/api-test' ? 'main-nav-api' : '',`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
42	`]`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
43	`.filter(Boolean)`	Completa una condición, encadenamiento, separador o expresión anterior.
44	`.join(' ')`	Completa una condición, encadenamiento, separador o expresión anterior.
45	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
46	`onClick={() => setMenuAbierto(false)}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
47	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
48	`{enlace.texto}`	Abre un bloque de código, objeto o expresión JSX.
49	`</NavLink>`	Cierra el elemento <NavLink> y delimita el contenido que pertenece a él.
50	`))}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
51	`</nav>`	Cierra el elemento <nav> y delimita el contenido que pertenece a él.
52	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
53	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
54	`<div className="header-actions">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
55	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
56	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
57	`className="login-toggle"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
58	`onClick={usuario ? onCerrarSesion : onIniciarSesion}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
59	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
60	`{usuario ? `Cerrar sesión (${usuario})` : 'Iniciar sesión'}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.

N.º	Código original	Qué hace esta línea
61	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
62	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
63	`{usuario && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
64	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
65	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
66	`className="nav-toggle"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
67	`aria-label="Abrir menú"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
68	`aria-expanded={menuAbierto}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
69	`onClick={() => setMenuAbierto(!menuAbierto)}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
70	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
71	`<span></span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
72	`<span></span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
73	`<span></span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
74	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
75	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
76	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
77	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
78	`</header>`	Cierra el elemento <header> y delimita el contenido que pertenece a él.
79	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
80	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
81	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
82	`export default Header`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.

Financiero/src/components/Hero.css

Hoja de estilos de Hero: define presentación, distribución, estados visuales y adaptación de pantalla.

36 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`.hero {`	Abre las reglas visuales del selector .hero.
2	`  flex: 1;`	La propiedad flex define el crecimiento y tamaño flexible; valor: 1.
3	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
4	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
5	`  justify-content: center;`	La propiedad justify-content distribuye elementos en el eje principal; valor: center.
6	`  background: linear-gradient(160deg, var(--teal-800) 0%, var(--teal-950) 100%);`	La propiedad background define el fondo; valor: linear-gradient(160deg, var(--teal-800) 0%, var(--teal-950) 100%).
7	`  color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
8	`  text-align: center;`	La propiedad text-align alinea el texto; valor: center.
9	`  padding: 6rem 1.5rem 5rem;`	La propiedad padding define el espacio interior; valor: 6rem 1.5rem 5rem.
10	`}`	Cierra el bloque CSS actual.
11	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
12	`.hero-inner {`	Abre las reglas visuales del selector .hero-inner.
13	`  max-width: 760px;`	La propiedad max-width establece el ancho máximo; valor: 760px.
14	`  margin: 0 auto;`	La propiedad margin define el espacio exterior; valor: 0 auto.
15	`}`	Cierra el bloque CSS actual.
16	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
17	`.hero h1 {`	Abre las reglas visuales del selector .hero h1.
18	`  color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
19	`  font-size: clamp(2.2rem, 6vw, 3.6rem);`	La propiedad font-size define el tamaño de letra; valor: clamp(2.2rem, 6vw, 3.6rem).
20	`  line-height: 1.05;`	La propiedad line-height define la altura de línea; valor: 1.05.
21	`  margin-bottom: 1.5rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 1.5rem.
22	`}`	Cierra el bloque CSS actual.
23	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
24	`.hero-desc {`	Abre las reglas visuales del selector .hero-desc.
25	`  color: #d7ecee;`	La propiedad color define el color del texto; valor: #d7ecee.
26	`  font-size: 1.1rem;`	La propiedad font-size define el tamaño de letra; valor: 1.1rem.
27	`  max-width: 560px;`	La propiedad max-width establece el ancho máximo; valor: 560px.
28	`  margin: 0 auto 2.2rem;`	La propiedad margin define el espacio exterior; valor: 0 auto 2.2rem.
29	`}`	Cierra el bloque CSS actual.
30	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
31	`.hero-actions {`	Abre las reglas visuales del selector .hero-actions.
32	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
33	`  gap: 1rem;`	La propiedad gap define el espacio entre elementos; valor: 1rem.

N.º	Código original	Qué hace esta línea
34	`justify-content: center;`	La propiedad justify-content distribuye elementos en el eje principal; valor: center.
35	`flex-wrap: wrap;`	La propiedad flex-wrap controla si los elementos flex pasan a otra fila; valor: wrap.
36	`}`	Cierra el bloque CSS actual.

Financiero/src/components/Hero.jsx

Componente o página React Hero: construye una parte de la interfaz y usa las props y estados del módulo.

38 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { useNavigate } from 'react-router-dom'`	Importa { useNavigate } desde react-router-dom y conecta este archivo con ese recurso.
2	`import './Hero.css'`	Importa './Hero.css' desde el módulo indicado y conecta este archivo con ese recurso.
3	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
4	`function Hero({ onUsuarioNuevo }) {`	Declara la función Hero; el bloque siguiente contiene las instrucciones que ejecuta.
5	`  const navigate = useNavigate()`	Declara navigate y le asigna el valor o expresión mostrado para reutilizarlo.
6	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
7	`  const irAProyecto = () => {`	Define en irAProyecto una función flecha para reutilizar o ejecutar una acción.
8	`    navigate('/proyecto')`	Invoca navigate para ejecutar la operación definida por esa función o biblioteca.
9	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
10	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
11	`  return (`	Devuelve un valor de la función o el elemento React que se renderizará.
12	`    <section className="hero" id="inicio">`	Abre el elemento <section>; sus atributos configuran contenido, accesibilidad o eventos.
13	`      <div className="hero-inner">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
14	`        <p className="eyebrow">Gestión financiera empresarial</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
15	`        <h1>`	Abre el elemento <h1>; sus atributos configuran contenido, accesibilidad o eventos.
16	`          SOFTWARE`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
17	`           `	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
18	`          FINANCIERO`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
19	`        </h1>`	Cierra el elemento <h1> y delimita el contenido que pertenece a él.
20	`        <p className="hero-desc">`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
21	`          Una plataforma para administrar el dinero de tu empresa en un solo lugar:`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
22	`          ingresos, gastos, riesgos, balances y cuentas por cobrar y pagar, con la`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
23	`          información clara y al día.`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
24	`        </p>`	Cierra el elemento <p> y delimita el contenido que pertenece a él.
25	`        <div className="hero-actions">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
26	`          <button className="btn btn-primary" onClick={onUsuarioNuevo}>`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
27	`            Usuario nuevo`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
28	`          </button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
29	`          <button className="btn btn-secondary" onClick={irAProyecto}>`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.

N.º	Código original	Qué hace esta línea
30	`Conocer más`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
31	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
32	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
33	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
34	`</section>`	Cierra el elemento <section> y delimita el contenido que pertenece a él.
35	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
36	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
37	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
38	`export default Hero`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.

Financiero/src/components/Login.css

Hoja de estilos de Login: define presentación, distribución, estados visuales y adaptación de pantalla.

80 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`.login-dialog {`	Abre las reglas visuales del selector .login-dialog.
2	`  width: min(440px, calc(100vw - 2rem));`	La propiedad width establece el ancho; valor: min(440px, calc(100vw - 2rem)).
3	`  padding: 2rem;`	La propiedad padding define el espacio interior; valor: 2rem.
4	`  border: 0;`	La propiedad border define el borde; valor: 0.
5	`  border-radius: var(--radius-md);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-md).
6	`  color: var(--ink);`	La propiedad color define el color del texto; valor: var(--ink).
7	`  background: var(--white);`	La propiedad background define el fondo; valor: var(--white).
8	`  box-shadow: 0 20px 60px rgba(13, 61, 67, 0.28);`	La propiedad box-shadow añade sombra; valor: 0 20px 60px rgba(13, 61, 67, 0.28).
9	`}`	Cierra el bloque CSS actual.
10	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
11	`.login-dialog::backdrop {`	Abre las reglas visuales del selector .login-dialog::backdrop.
12	`  background: rgba(13, 38, 42, 0.72);`	La propiedad background define el fondo; valor: rgba(13, 38, 42, 0.72).
13	`}`	Cierra el bloque CSS actual.
14	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
15	`.login-dialog-header {`	Abre las reglas visuales del selector .login-dialog-header.
16	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
17	`  align-items: flex-start;`	La propiedad align-items alinea elementos en el eje transversal; valor: flex-start.
18	`  justify-content: space-between;`	La propiedad justify-content distribuye elementos en el eje principal; valor: space-between.
19	`  gap: 1rem;`	La propiedad gap define el espacio entre elementos; valor: 1rem.
20	`  margin-bottom: 1.5rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 1.5rem.
21	`}`	Cierra el bloque CSS actual.
22	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
23	`.login-dialog-header h2 {`	Abre las reglas visuales del selector .login-dialog-header h2.
24	`  font-size: 1.7rem;`	La propiedad font-size define el tamaño de letra; valor: 1.7rem.
25	`}`	Cierra el bloque CSS actual.
26	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
27	`.login-close {`	Abre las reglas visuales del selector .login-close.
28	`  width: 2rem;`	La propiedad width establece el ancho; valor: 2rem.
29	`  height: 2rem;`	La propiedad height establece la altura; valor: 2rem.
30	`  padding: 0;`	La propiedad padding define el espacio interior; valor: 0.
31	`  border: 1px solid #cfe0e2;`	La propiedad border define el borde; valor: 1px solid #cfe0e2.
32	`  border-radius: 50%;`	La propiedad border-radius redondea las esquinas; valor: 50%.
33	`  color: var(--ink);`	La propiedad color define el color del texto; valor: var(--ink).
34	`  background: transparent;`	La propiedad background define el fondo; valor: transparent.
35	`  font-family: var(--font-body);`	La propiedad font-family elige la tipografía; valor: var(--font-body).

N.º	Código original	Qué hace esta línea
36	`  font-size: 1.4rem;`	La propiedad font-size define el tamaño de letra; valor: 1.4rem.
37	`  line-height: 1;`	La propiedad line-height define la altura de línea; valor: 1.
38	`  cursor: pointer;`	La propiedad cursor define el cursor del puntero; valor: pointer.
39	`}`	Cierra el bloque CSS actual.
40	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
41	`.login-form {`	Abre las reglas visuales del selector .login-form.
42	`  display: grid;`	La propiedad display define cómo se distribuyen los elementos; valor: grid.
43	`  gap: 0.65rem;`	La propiedad gap define el espacio entre elementos; valor: 0.65rem.
44	`}`	Cierra el bloque CSS actual.
45	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
46	`.login-form label {`	Abre las reglas visuales del selector .login-form label.
47	`  color: var(--ink-soft);`	La propiedad color define el color del texto; valor: var(--ink-soft).
48	`  font-weight: 600;`	La propiedad font-weight define el grosor de letra; valor: 600.
49	`}`	Cierra el bloque CSS actual.
50	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
51	`.login-form input {`	Abre las reglas visuales del selector .login-form input.
52	`  width: 100%;`	La propiedad width establece el ancho; valor: 100%.
53	`  padding: 0.75rem 0.9rem;`	La propiedad padding define el espacio interior; valor: 0.75rem 0.9rem.
54	`  border: 1px solid #cfe0e2;`	La propiedad border define el borde; valor: 1px solid #cfe0e2.
55	`  border-radius: var(--radius-sm);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-sm).
56	`  color: var(--ink);`	La propiedad color define el color del texto; valor: var(--ink).
57	`  background: var(--bg-light);`	La propiedad background define el fondo; valor: var(--bg-light).
58	`  font: inherit;`	La propiedad font establece la propiedad visual font; valor: inherit.
59	`}`	Cierra el bloque CSS actual.
60	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
61	`.login-form input:focus-visible,`	Completa una regla CSS que participa en la presentación visual.
62	`.login-close:focus-visible {`	Abre las reglas visuales del selector .login-close:focus-visible.
63	`  outline: 2px solid var(--cyan-500);`	La propiedad outline establece la propiedad visual outline; valor: 2px solid var(--cyan-500).
64	`  outline-offset: 2px;`	La propiedad outline-offset establece la propiedad visual outline-offset; valor: 2px.
65	`}`	Cierra el bloque CSS actual.
66	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
67	`.login-form .btn {`	Abre las reglas visuales del selector .login-form .btn.
68	`  justify-self: start;`	La propiedad justify-self establece la propiedad visual justify-self; valor: start.
69	`  margin-top: 0.65rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 0.65rem.
70	`}`	Cierra el bloque CSS actual.
71	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
72	`.login-form .btn:disabled {`	Abre las reglas visuales del selector .login-form .btn:disabled.

N.º	Código original	Qué hace esta línea
73	`cursor: wait;`	La propiedad cursor define el cursor del puntero; valor: wait.
74	`opacity: 0.7;`	La propiedad opacity define la transparencia; valor: 0.7.
75	`}`	Cierra el bloque CSS actual.
76	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
77	`.login-error {`	Abre las reglas visuales del selector .login-error.
78	`margin: 0.25rem 0 0;`	La propiedad margin define el espacio exterior; valor: 0.25rem 0 0.
79	`color: #b42318;`	La propiedad color define el color del texto; valor: #b42318.
80	`}`	Cierra el bloque CSS actual.

Financiero/src/components/Login.jsx

Envía credenciales a /auth/login y entrega el token de sesión al componente principal.

99 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { useEffect, useRef, useState } from 'react'`	Importa { useEffect, useRef, useState } desde react y conecta este archivo con ese recurso.
2	`import axios from 'axios'`	Importa axios desde axios y conecta este archivo con ese recurso.
3	`import './Login.css'`	Importa './Login.css' desde el módulo indicado y conecta este archivo con ese recurso.
4	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
5	`function Login({ abierto, onCerrar, onIniciarSesion }) {`	Declara la función Login; el bloque siguiente contiene las instrucciones que ejecuta.
6	`  const dialogoRef = useRef(null)`	Crea una referencia persistente para conservar un valor o acceder a un elemento DOM.
7	`  const [usuario, setUsuario] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
8	`  const [clave, setClave] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
9	`  const [error, setError] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
10	`  const [cargando, setCargando] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
11	`  const API_URL = import.meta.env.VITE_API_URL`	Lee en API_URL una variable de configuración expuesta por Vite al frontend.
12	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
13	`  useEffect(() => {`	Registra un efecto React para sincronizar la interfaz con eventos o recursos externos.
14	`    const dialogo = dialogoRef.current`	Declara dialogo y le asigna el valor o expresión mostrado para reutilizarlo.
15	`    if (!dialogo) return`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
16	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
17	`    if (abierto && !dialogo.open) dialogo.showModal()`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
18	`    if (!abierto && dialogo.open) dialogo.close()`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
19	`  }, [abierto])`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
20	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
21	`  async function enviarFormulario(evento) {`	Declara la función enviarFormulario; el bloque siguiente contiene las instrucciones que ejecuta.
22	`    evento.preventDefault()`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
23	`    setError('')`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
24	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
25	`    if (!API_URL) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
26	`      setError('Falta configurar VITE_API_URL en el archivo .env del frontend.')`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
27	`      return`	Devuelve un valor de la función o el elemento React que se renderizará.
28	`    }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
29	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
30	`    setCargando(true)`	Invoca setCargando para ejecutar la operación definida por esa función o biblioteca.

N.º	Código original	Qué hace esta línea
31	`try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
32	`const respuesta = await axios.post(`	Espera una petición HTTP al backend antes de continuar.
33	`` `${API_URL.replace(/\/+$/, '')}/auth/login`, ``	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
34	`{ username: usuario, password: clave },`	Abre un bloque de código, objeto o expresión JSX.
35	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
36	`onIniciarSesion(respuesta.data.user.username, respuesta.data.token)`	Invoca onIniciarSesion para ejecutar la operación definida por esa función o biblioteca.
37	`setUsuario('')`	Invoca setUsuario para ejecutar la operación definida por esa función o biblioteca.
38	`setClave('')`	Invoca setClave para ejecutar la operación definida por esa función o biblioteca.
39	`} catch (errorPeticion) {`	Abre el bloque de código que contiene las instrucciones siguientes.
40	`setError(`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
41	`errorPeticion.response?.data?.message ||`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
42	`'No fue posible iniciar sesión. Verifica la conexión con el servidor.',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
43	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
44	`} finally {`	Abre el bloque de código que contiene las instrucciones siguientes.
45	`setCargando(false)`	Invoca setCargando para ejecutar la operación definida por esa función o biblioteca.
46	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
47	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
48	*(línea vacía)*	Línea en blanco que separa instrucciones o bloques del código.
49	`return (`	Devuelve un valor de la función o el elemento React que se renderizará.
50	`<dialog`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
51	`ref={dialogoRef}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
52	`className="login-dialog"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
53	`aria-labelledby="login-title"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
54	`onClose={onCerrar}`	Conecta el evento onClose del elemento visual con la función que responde a la interacción.
55	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
56	`<div className="login-dialog-header">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
57	`<div>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
58	`<p className="eyebrow eyebrow-dark">Acceso administrativo</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
59	`<h2 id="login-title">Iniciar sesión</h2>`	Abre el elemento <h2>; sus atributos configuran contenido, accesibilidad o eventos.
60	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
61	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.

N.º	Código original	Qué hace esta línea
62	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
63	`className="login-close"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
64	`aria-label="Cerrar formulario de inicio de sesión"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
65	`onClick={onCerrar}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
66	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
67	`×`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
68	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
69	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
70	`<form className="login-form" onSubmit={enviarFormulario}>`	Conecta el evento onSubmit del elemento visual con la función que responde a la interacción.
71	`<label htmlFor="login-username">Usuario</label>`	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.
72	`<input`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
73	`id="login-username"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
74	`name="username"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
75	`autoComplete="username"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
76	`required`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
77	`value={usuario}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
78	`onChange={(evento) => setUsuario(evento.target.value)}`	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
79	`/>`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
80	`<label htmlFor="login-password">Clave</label>`	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.
81	`<input`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
82	`id="login-password"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
83	`name="password"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
84	`type="password"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
85	`autoComplete="current-password"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
86	`required`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
87	`value={clave}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
88	`onChange={(evento) => setClave(evento.target.value)}`	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
89	`/>`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
90	`{error && <p className="login-error" role="alert">{error}</p>}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
91	`<button className="btn btn-primary" type="submit" disabled={cargando}>`	Abre el elemento <button>; sus atributos configuran contenido, accesibilidad o eventos.
92	`{cargando ? 'Verificando…' : 'Entrar'}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.

N.º	Código original	Qué hace esta línea
93	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
94	`</form>`	Cierra el elemento <form> y delimita el contenido que pertenece a él.
95	`</dialog>`	Cierra el elemento <dialog> y delimita el contenido que pertenece a él.
96	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
97	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
98	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
99	`export default Login`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.

Financiero/src/components/Problema.css

Hoja de estilos de Problema: define presentación, distribución, estados visuales y adaptación de pantalla.

56 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`.problema {`	Abre las reglas visuales del selector .problema.
2	`  background: var(--bg-light);`	La propiedad background define el fondo; valor: var(--bg-light).
3	`}`	Cierra el bloque CSS actual.
4		Línea en blanco que separa visualmente reglas o bloques de estilos.
5	`.empleados-bloque {`	Abre las reglas visuales del selector .empleados-bloque.
6	`  margin-top: 1rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 1rem.
7	`}`	Cierra el bloque CSS actual.
8		Línea en blanco que separa visualmente reglas o bloques de estilos.
9	`.empleados-error {`	Abre las reglas visuales del selector .empleados-error.
10	`  color: #c23b3b;`	La propiedad color define el color del texto; valor: #c23b3b.
11	`  font-size: 0.85rem;`	La propiedad font-size define el tamaño de letra; valor: 0.85rem.
12	`  margin: 0.8rem 0 0;`	La propiedad margin define el espacio exterior; valor: 0.8rem 0 0.
13	`}`	Cierra el bloque CSS actual.
14		Línea en blanco que separa visualmente reglas o bloques de estilos.
15	`.empleados-lista {`	Abre las reglas visuales del selector .empleados-lista.
16	`  list-style: none;`	La propiedad list-style establece la propiedad visual list-style; valor: none.
17	`  margin: 1.5rem 0 0;`	La propiedad margin define el espacio exterior; valor: 1.5rem 0 0.
18	`  padding: 0;`	La propiedad padding define el espacio interior; valor: 0.
19	`  display: grid;`	La propiedad display define cómo se distribuyen los elementos; valor: grid.
20	`  grid-template-columns: repeat(3, 1fr);`	La propiedad grid-template-columns define las columnas de la cuadrícula; valor: repeat(3, 1fr).
21	`  gap: 1rem;`	La propiedad gap define el espacio entre elementos; valor: 1rem.
22	`}`	Cierra el bloque CSS actual.
23		Línea en blanco que separa visualmente reglas o bloques de estilos.
24	`.empleados-lista li {`	Abre las reglas visuales del selector .empleados-lista li.
25	`  background: var(--white);`	La propiedad background define el fondo; valor: var(--white).
26	`  border-left: 4px solid var(--cyan-500);`	La propiedad border-left establece la propiedad visual border-left; valor: 4px solid var(--cyan-500).
27	`  border-radius: var(--radius-sm);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-sm).
28	`  padding: 0.9rem 1.1rem;`	La propiedad padding define el espacio interior; valor: 0.9rem 1.1rem.
29	`  box-shadow: 0 4px 14px rgba(18, 61, 67, 0.06);`	La propiedad box-shadow añade sombra; valor: 0 4px 14px rgba(18, 61, 67, 0.06).
30	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
31	`  flex-direction: column;`	La propiedad flex-direction define la dirección del flexbox; valor: column.
32	`  gap: 0.2rem;`	La propiedad gap define el espacio entre elementos; valor: 0.2rem.
33	`}`	Cierra el bloque CSS actual.

N.º	Código original	Qué hace esta línea
34		Línea en blanco que separa visualmente reglas o bloques de estilos.
35	`.empleado-nombre {`	Abre las reglas visuales del selector .empleado-nombre.
36	`  font-weight: 600;`	La propiedad font-weight define el grosor de letra; valor: 600.
37	`  color: var(--ink);`	La propiedad color define el color del texto; valor: var(--ink).
38	`  font-size: 0.95rem;`	La propiedad font-size define el tamaño de letra; valor: 0.95rem.
39	`}`	Cierra el bloque CSS actual.
40		Línea en blanco que separa visualmente reglas o bloques de estilos.
41	`.empleado-cargo {`	Abre las reglas visuales del selector .empleado-cargo.
42	`  color: var(--teal-700);`	La propiedad color define el color del texto; valor: var(--teal-700).
43	`  font-size: 0.85rem;`	La propiedad font-size define el tamaño de letra; valor: 0.85rem.
44	`}`	Cierra el bloque CSS actual.
45		Línea en blanco que separa visualmente reglas o bloques de estilos.
46	`@media (max-width: 860px) {`	Inicia una condición responsive; sus estilos se aplican cuando se cumple esta condición de pantalla.
47	`  .empleados-lista {`	Abre las reglas visuales del selector .empleados-lista.
48	`    grid-template-columns: repeat(2, 1fr);`	La propiedad grid-template-columns define las columnas de la cuadrícula; valor: repeat(2, 1fr).
49	`  }`	Cierra el bloque CSS actual.
50	`}`	Cierra el bloque CSS actual.
51		Línea en blanco que separa visualmente reglas o bloques de estilos.
52	`@media (max-width: 500px) {`	Inicia una condición responsive; sus estilos se aplican cuando se cumple esta condición de pantalla.
53	`  .empleados-lista {`	Abre las reglas visuales del selector .empleados-lista.
54	`    grid-template-columns: 1fr;`	La propiedad grid-template-columns define las columnas de la cuadrícula; valor: 1fr.
55	`  }`	Cierra el bloque CSS actual.
56	`}`	Cierra el bloque CSS actual.

Financiero/src/components/Problema.jsx

Componente o página React Problema: construye una parte de la interfaz y usa las props y estados del módulo.

82 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { useEffect, useRef, useState } from 'react'`	Importa { useEffect, useRef, useState } desde react y conecta este archivo con ese recurso.
2	`import axios from 'axios'`	Importa axios desde axios y conecta este archivo con ese recurso.
3	`import './Problema.css'`	Importa './Problema.css' desde el módulo indicado y conecta este archivo con ese recurso.
4	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
5	`const cargosInventados = [`	Inicia en cargosInventados una estructura de datos que agrupa valores relacionados.
6	`  'Gerente Financiero',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
7	`  'Analista de Riesgos',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
8	`  'Contador Senior',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
9	`  'Coordinador de Tesorería',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
10	`  'Auditor Interno',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
11	`  'Especialista en Inversiones',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
12	`  'Analista de Cartera',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
13	`  'Jefe de Contabilidad',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
14	`  'Asesor Financiero',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
15	`  'Controller Financiero',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
16	`]`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
17	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
18	`function Problema() {`	Declara la función Problema; el bloque siguiente contiene las instrucciones que ejecuta.
19	`  const [empleados, setEmpleados] = useState([])`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
20	`  const [cargando, setCargando] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
21	`  const [error, setError] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
22	`  const temporizadorRef = useRef(null)`	Crea una referencia persistente para conservar un valor o acceder a un elemento DOM.
23	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
24	`  useEffect(() => () => clearTimeout(temporizadorRef.current), [])`	Registra un efecto React para sincronizar la interfaz con eventos o recursos externos.
25	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
26	`  const obtenerEmpleados = async () => {`	Define en obtenerEmpleados una función flecha para reutilizar o ejecutar una acción.
27	`    clearTimeout(temporizadorRef.current)`	Invoca clearTimeout para ejecutar la operación definida por esa función o biblioteca.
28	`    setCargando(true)`	Invoca setCargando para ejecutar la operación definida por esa función o biblioteca.
29	`    setError('')`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
30	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.

N.º	Código original	Qué hace esta línea
31	`try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
32	`const { data } = await axios.get('https://jsonplaceholder.typicode.com/users' )`	Espera una petición HTTP al backend antes de continuar.
33	`setEmpleados(data)`	Invoca setEmpleados para ejecutar la operación definida por esa función o biblioteca.
34	`} catch (errorSolicitud) {`	Abre el bloque de código que contiene las instrucciones siguientes.
35	`setError(errorSolicitud.response?.data?.message || 'Ocurrió un error al obtener los nombres de empleados.')`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
36	`setEmpleados([])`	Invoca setEmpleados para ejecutar la operación definida por esa función o biblioteca.
37	`} finally {`	Abre el bloque de código que contiene las instrucciones siguientes.
38	`setCargando(false)`	Invoca setCargando para ejecutar la operación definida por esa función o biblioteca.
39	`temporizadorRef.current = setTimeout(() => {`	Abre el bloque de código que contiene las instrucciones siguientes.
40	`setEmpleados([])`	Invoca setEmpleados para ejecutar la operación definida por esa función o biblioteca.
41	`setError('')`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
42	`}, 5000)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
43	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
44	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
45	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
46	`return (`	Devuelve un valor de la función o el elemento React que se renderizará.
47	`<section className="problema" id="proyecto">`	Abre el elemento <section>; sus atributos configuran contenido, accesibilidad o eventos.
48	`<div className="section-inner">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
49	`<p className="eyebrow eyebrow-dark">Sobre nosotros</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
50	`<h2 className="section-title">Conoce a nuestro equipo</h2>`	Abre el elemento <h2>; sus atributos configuran contenido, accesibilidad o eventos.
51	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
52	`<div className="empleados-bloque">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
53	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
54	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
55	`className="btn btn-primary"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
56	`onClick={obtenerEmpleados}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
57	`disabled={cargando}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
58	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
59	`{cargando ? 'Cargando...' : 'Obtener nombres de empleados'}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
60	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
61	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.

N.º	Código original	Qué hace esta línea
62	`{error && <p className="empleados-error">{error}</p>}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
63	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
64	`{empleados.length > 0 && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
65	`<ul className="empleados-lista">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
66	`{empleados.map((empleado, indice) => (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
67	`<li key={empleado.id}>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
68	`<span className="empleado-nombre">{empleado.name}</span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
69	`<span className="empleado-cargo">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
70	`{cargosInventados[indice % cargosInventados.length]}`	Abre un bloque de código, objeto o expresión JSX.
71	`</span>`	Cierra el elemento y delimita el contenido que pertenece a él.
72	`</li>`	Cierra el elemento y delimita el contenido que pertenece a él.
73	`))}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
74	`</ul>`	Cierra el elemento y delimita el contenido que pertenece a él.
75	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
76	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
77	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
78	`</section>`	Cierra el elemento <section> y delimita el contenido que pertenece a él.
79	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
80	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
81	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
82	`export default Problema`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.

Financiero/src/index.css

Hoja de estilos de index: define presentación, distribución, estados visuales y adaptación de pantalla.

132 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`/* ==========================================`	Comentario CSS; documenta el estilo, pero no cambia la apariencia renderizada.
2	`Variables globales — paleta tomada de los`	Completa una regla CSS que participa en la presentación visual.
3	`mockups originales de JSC: Software Financiero`	Completa una regla CSS que participa en la presentación visual.
4	`========================================== */`	Completa una regla CSS que participa en la presentación visual.
5	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
6	`:root {`	Abre las reglas visuales del selector :root.
7	`color-scheme: light;`	La propiedad color-scheme establece la propiedad visual color-scheme; valor: light.
8	`--teal-950: #0d3d43;`	La propiedad --teal-950 establece la propiedad visual --teal-950; valor: #0d3d43.
9	`--teal-900: #12626c;`	La propiedad --teal-900 establece la propiedad visual --teal-900; valor: #12626c.
10	`--teal-800: #17707c;`	La propiedad --teal-800 establece la propiedad visual --teal-800; valor: #17707c.
11	`--teal-700: #1c6e78;`	La propiedad --teal-700 establece la propiedad visual --teal-700; valor: #1c6e78.
12	`--cyan-500: #3ecbd9;`	La propiedad --cyan-500 establece la propiedad visual --cyan-500; valor: #3ecbd9.
13	`--cyan-300: #7fe0e8;`	La propiedad --cyan-300 establece la propiedad visual --cyan-300; valor: #7fe0e8.
14	`--bg-light: #eef3f4;`	La propiedad --bg-light establece la propiedad visual --bg-light; valor: #eef3f4.
15	`--white: #ffffff;`	La propiedad --white establece la propiedad visual --white; valor: #ffffff.
16	`--ink: #123138;`	La propiedad --ink establece la propiedad visual --ink; valor: #123138.
17	`--ink-soft: #3d5c62;`	La propiedad --ink-soft establece la propiedad visual --ink-soft; valor: #3d5c62.
18	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
19	`--font-display: "Anton", "Barlow", sans-serif;`	La propiedad --font-display establece la propiedad visual --font-display; valor: "Anton", "Barlow", sans-serif.
20	`--font-body: "Barlow", "Segoe UI", sans-serif;`	La propiedad --font-body establece la propiedad visual --font-body; valor: "Barlow", "Segoe UI", sans-serif.
21	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
22	`--radius-sm: 6px;`	La propiedad --radius-sm establece la propiedad visual --radius-sm; valor: 6px.
23	`--radius-md: 10px;`	La propiedad --radius-md establece la propiedad visual --radius-md; valor: 10px.
24	`--container: 1160px;`	La propiedad --container establece la propiedad visual --container; valor: 1160px.
25	`}`	Cierra el bloque CSS actual.
26	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
27	`* {`	Comentario CSS; documenta el estilo, pero no cambia la apariencia renderizada.
28	`box-sizing: border-box;`	La propiedad box-sizing define cómo se calculan ancho y alto; valor: border-box.
29	`}`	Cierra el bloque CSS actual.
30	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.

N.º	Código original	Qué hace esta línea
31	`html {`	Abre las reglas visuales del selector html.
32	`  scroll-behavior: smooth;`	La propiedad scroll-behavior establece la propiedad visual scroll-behavior; valor: smooth.
33	`}`	Cierra el bloque CSS actual.
34	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
35	`body {`	Abre las reglas visuales del selector body.
36	`  margin: 0;`	La propiedad margin define el espacio exterior; valor: 0.
37	`  font-family: var(--font-body);`	La propiedad font-family elige la tipografía; valor: var(--font-body).
38	`  color: var(--ink);`	La propiedad color define el color del texto; valor: var(--ink).
39	`  background: var(--bg-light);`	La propiedad background define el fondo; valor: var(--bg-light).
40	`  line-height: 1.55;`	La propiedad line-height define la altura de línea; valor: 1.55.
41	`}`	Cierra el bloque CSS actual.
42	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
43	`img {`	Abre las reglas visuales del selector img.
44	`  max-width: 100%;`	La propiedad max-width establece el ancho máximo; valor: 100%.
45	`  display: block;`	La propiedad display define cómo se distribuyen los elementos; valor: block.
46	`}`	Cierra el bloque CSS actual.
47	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
48	`h1, h2, h3, h4 {`	Abre las reglas visuales del selector h1, h2, h3, h4.
49	`  font-family: var(--font-display);`	La propiedad font-family elige la tipografía; valor: var(--font-display).
50	`  font-weight: 400;`	La propiedad font-weight define el grosor de letra; valor: 400.
51	`  text-transform: uppercase;`	La propiedad text-transform establece la propiedad visual text-transform; valor: uppercase.
52	`  letter-spacing: 0.01em;`	La propiedad letter-spacing establece la propiedad visual letter-spacing; valor: 0.01em.
53	`  margin: 0;`	La propiedad margin define el espacio exterior; valor: 0.
54	`  color: var(--teal-800);`	La propiedad color define el color del texto; valor: var(--teal-800).
55	`}`	Cierra el bloque CSS actual.
56	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
57	`p {`	Abre las reglas visuales del selector p.
58	`  margin: 0 0 1rem 0;`	La propiedad margin define el espacio exterior; valor: 0 0 1rem 0.
59	`}`	Cierra el bloque CSS actual.
60	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
61	`button {`	Abre las reglas visuales del selector button.
62	`  font-family: var(--font-display);`	La propiedad font-family elige la tipografía; valor: var(--font-display).
63	`}`	Cierra el bloque CSS actual.
64	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
65	`.section-inner {`	Abre las reglas visuales del selector .section-inner.
66	`  max-width: var(--container);`	La propiedad max-width establece el ancho máximo; valor: var(--container).
67	`  margin: 0 auto;`	La propiedad margin define el espacio exterior; valor: 0 auto.

N.º	Código original	Qué hace esta línea
68	`  padding: 5rem 1.5rem;`	La propiedad padding define el espacio interior; valor: 5rem 1.5rem.
69	`}`	Cierra el bloque CSS actual.
70	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
71	`.eyebrow {`	Abre las reglas visuales del selector .eyebrow.
72	`  font-family: var(--font-display);`	La propiedad font-family elige la tipografía; valor: var(--font-display).
73	`  text-transform: uppercase;`	La propiedad text-transform establece la propiedad visual text-transform; valor: uppercase.
74	`  letter-spacing: 0.12em;`	La propiedad letter-spacing establece la propiedad visual letter-spacing; valor: 0.12em.
75	`  color: var(--cyan-300);`	La propiedad color define el color del texto; valor: var(--cyan-300).
76	`  font-size: 0.85rem;`	La propiedad font-size define el tamaño de letra; valor: 0.85rem.
77	`  margin: 0 0 0.75rem 0;`	La propiedad margin define el espacio exterior; valor: 0 0 0.75rem 0.
78	`}`	Cierra el bloque CSS actual.
79	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
80	`.eyebrow-dark {`	Abre las reglas visuales del selector .eyebrow-dark.
81	`  color: var(--teal-700);`	La propiedad color define el color del texto; valor: var(--teal-700).
82	`}`	Cierra el bloque CSS actual.
83	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
84	`.section-title {`	Abre las reglas visuales del selector .section-title.
85	`  font-size: clamp(1.7rem, 3vw, 2.4rem);`	La propiedad font-size define el tamaño de letra; valor: clamp(1.7rem, 3vw, 2.4rem).
86	`  margin-bottom: 2.5rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 2.5rem.
87	`  max-width: 720px;`	La propiedad max-width establece el ancho máximo; valor: 720px.
88	`}`	Cierra el bloque CSS actual.
89	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
90	`.section-title-light {`	Abre las reglas visuales del selector .section-title-light.
91	`  color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
92	`}`	Cierra el bloque CSS actual.
93	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
94	`.btn {`	Abre las reglas visuales del selector .btn.
95	`  font-family: var(--font-display);`	La propiedad font-family elige la tipografía; valor: var(--font-display).
96	`  text-transform: uppercase;`	La propiedad text-transform establece la propiedad visual text-transform; valor: uppercase.
97	`  letter-spacing: 0.03em;`	La propiedad letter-spacing establece la propiedad visual letter-spacing; valor: 0.03em.
98	`  font-size: 0.95rem;`	La propiedad font-size define el tamaño de letra; valor: 0.95rem.
99	`  border: none;`	La propiedad border define el borde; valor: none.
100	`  border-radius: var(--radius-sm);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-sm).
101	`  padding: 0.85rem 1.8rem;`	La propiedad padding define el espacio interior; valor: 0.85rem 1.8rem.
102	`  cursor: pointer;`	La propiedad cursor define el cursor del puntero; valor: pointer.

N.º	Código original	Qué hace esta línea
103	`  transition: transform 0.15s ease, background-color 0.15s ease,` `box-shadow 0.15s ease;`	La propiedad transition anima cambios entre estados; valor: transform 0.15s ease, background-color 0.15s ease, box-shadow 0.15s ease.
104	`}`	Cierra el bloque CSS actual.
105	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
106	`.btn-primary {`	Abre las reglas visuales del selector .btn-primary.
107	`  background: var(--teal-700);`	La propiedad background define el fondo; valor: var(--teal-700).
108	`  color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
109	`}`	Cierra el bloque CSS actual.
110	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
111	`.btn-primary:hover {`	Abre las reglas visuales del selector .btn-primary:hover.
112	`  background: var(--teal-950);`	La propiedad background define el fondo; valor: var(--teal-950).
113	`  transform: translateY(-2px);`	La propiedad transform transforma visualmente el elemento; valor: translateY(-2px).
114	`  box-shadow: 0 8px 18px rgba(18, 61, 67, 0.25);`	La propiedad box-shadow añade sombra; valor: 0 8px 18px rgba(18, 61, 67, 0.25).
115	`}`	Cierra el bloque CSS actual.
116	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
117	`.btn-secondary {`	Abre las reglas visuales del selector .btn-secondary.
118	`  background: var(--cyan-300);`	La propiedad background define el fondo; valor: var(--cyan-300).
119	`  color: var(--teal-950);`	La propiedad color define el color del texto; valor: var(--teal-950).
120	`}`	Cierra el bloque CSS actual.
121	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
122	`.btn-secondary:hover {`	Abre las reglas visuales del selector .btn-secondary:hover.
123	`  background: var(--cyan-500);`	La propiedad background define el fondo; valor: var(--cyan-500).
124	`  transform: translateY(-2px);`	La propiedad transform transforma visualmente el elemento; valor: translateY(-2px).
125	`  box-shadow: 0 8px 18px rgba(62, 203, 217, 0.35);`	La propiedad box-shadow añade sombra; valor: 0 8px 18px rgba(62, 203, 217, 0.35).
126	`}`	Cierra el bloque CSS actual.
127	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
128	`@media (max-width: 860px) {`	Inicia una condición responsive; sus estilos se aplican cuando se cumple esta condición de pantalla.
129	`  .section-inner {`	Abre las reglas visuales del selector .section-inner.
130	`    padding: 3.5rem 1.5rem;`	La propiedad padding define el espacio interior; valor: 3.5rem 1.5rem.
131	`  }`	Cierra el bloque CSS actual.
132	`}`	Cierra el bloque CSS actual.

Financiero/src/main.jsx

Punto de montaje de React: crea la aplicación y la conecta al elemento raíz HTML.

13 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import React from 'react'`	Importa React desde react y conecta este archivo con ese recurso.
2	`import ReactDOM from 'react-dom/client'`	Importa ReactDOM desde react-dom/client y conecta este archivo con ese recurso.
3	`import { BrowserRouter } from 'react-router-dom'`	Importa { BrowserRouter } desde react-router-dom y conecta este archivo con ese recurso.
4	`import App from './App.jsx'`	Importa App desde ./App.jsx y conecta este archivo con ese recurso.
5	`import './index.css'`	Importa './index.css' desde el módulo indicado y conecta este archivo con ese recurso.
6	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
7	`ReactDOM.createRoot(document.getElementById('root')).render(`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
8	`  <React.StrictMode>`	Inserta el componente React React.StrictMode y le pasa sus props.
9	`    <BrowserRouter>`	Inserta el componente React BrowserRouter y le pasa sus props.
10	`      <App />`	Inserta el componente React App y le pasa sus props.
11	`    </BrowserRouter>`	Cierra el elemento <BrowserRouter> y delimita el contenido que pertenece a él.
12	`  </React.StrictMode>,`	Cierra el elemento <React> y delimita el contenido que pertenece a él.
13	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.

Financiero/src/pages/Backend.jsx

Comprueba la conexión al backend y a PostgreSQL mediante endpoints de salud.

79 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { useEffect, useRef, useState } from 'react';`	Importa { useEffect, useRef, useState } desde react y conecta este archivo con ese recurso.
2	`import axios from 'axios';`	Importa axios desde axios y conecta este archivo con ese recurso.
3	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
4	`function Backend() {`	Declara la función Backend; el bloque siguiente contiene las instrucciones que ejecuta.
5	`  const [status, setStatus] = useState(null);`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
6	`  const [loading, setLoading] = useState(false);`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
7	`  const temporizadorRef = useRef(null);`	Crea una referencia persistente para conservar un valor o acceder a un elemento DOM.
8	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
9	`  const API_URL = import.meta.env.VITE_API_URL;`	Lee en API_URL una variable de configuración expuesta por Vite al frontend.
10	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
11	`  useEffect(() => () => clearTimeout(temporizadorRef.current), []);`	Registra un efecto React para sincronizar la interfaz con eventos o recursos externos.
12	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
13	`  function mostrarStatus(nuevoStatus) {`	Declara la función mostrarStatus; el bloque siguiente contiene las instrucciones que ejecuta.
14	`    setStatus(nuevoStatus);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
15	`    clearTimeout(temporizadorRef.current);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
16	`    temporizadorRef.current = setTimeout(() => setStatus(null), 5000);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
17	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
18	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
19	`  async function checkDatabaseConnection() {`	Declara la función checkDatabaseConnection; el bloque siguiente contiene las instrucciones que ejecuta.
20	`    clearTimeout(temporizadorRef.current);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
21	`    setLoading(true);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
22	`    setStatus(null);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
23	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
24	`    try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
25	`      if (!API_URL) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
26	`        throw new Error('Falta configurar VITE_API_URL en el archivo .env del frontend.');`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
27	`      }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
28	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
29	`      const respuesta = await axios.get(`${API_URL.replace(/\/+$/, '')}/health/db`);`	Espera una petición HTTP al backend antes de continuar.
30	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
31	`      mostrarStatus({ type: 'success', ...respuesta.data });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.

N.º	Código original	Qué hace esta línea
32	`} catch (error) {`	Abre el bloque de código que contiene las instrucciones siguientes.
33	`mostrarStatus({`	Abre el bloque de código que contiene las instrucciones siguientes.
34	`type: 'error',`	Define el valor de la propiedad «type» del objeto actual.
35	`message:`	Define el valor de la propiedad «message» del objeto actual.
36	`error.response?.data?.message || 'No fue posible conectar con el backend.',`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
37	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
38	`} finally {`	Abre el bloque de código que contiene las instrucciones siguientes.
39	`setLoading(false);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
40	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
41	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
42	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
43	`return (`	Devuelve un valor de la función o el elemento React que se renderizará.
44	`<main`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
45	`style={{`	Abre el bloque de código que contiene las instrucciones siguientes.
46	`maxWidth: '720px',`	Define el valor de la propiedad «maxWidth» del objeto actual.
47	`margin: '0 auto',`	Define el valor de la propiedad «margin» del objeto actual.
48	`padding: '4rem 1.5rem',`	Define el valor de la propiedad «padding» del objeto actual.
49	`fontFamily: 'var(--font-body)',`	Define el valor de la propiedad «fontFamily» del objeto actual.
50	`textAlign: 'center',`	Define el valor de la propiedad «textAlign» del objeto actual.
51	`}}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
52	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
53	`<h1 className="section-title" style={{ margin: '0 auto 2rem' }}>`	Abre el elemento <h1>; sus atributos configuran contenido, accesibilidad o eventos.
54	`Conexión a la Base de Datos`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
55	`</h1>`	Cierra el elemento <h1> y delimita el contenido que pertenece a él.
56	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
57	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
58	`className="btn btn-primary"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
59	`onClick={checkDatabaseConnection}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
60	`disabled={loading}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
61	`style={{ fontSize: '1.1rem', padding: '1rem 2.4rem' }}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
62	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
63	`{loading ? 'Probando conexión...' : 'Probar conexión'}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
64	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.

N.º	Código original	Qué hace esta línea
65	`{status?.type === 'success' && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
66	`<p role="status" aria-live="polite" style={{ marginTop: '1rem' }}>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
67	`<strong>{status.message}</strong>.`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
68	`</p>`	Cierra el elemento <p> y delimita el contenido que pertenece a él.
69	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
70	`{status?.type === 'error' && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
71	`<p role="alert" style={{ marginTop: '1rem' }}>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
72	`Error: {status.message}`	Define el valor de la propiedad «Error» del objeto actual.
73	`</p>`	Cierra el elemento <p> y delimita el contenido que pertenece a él.
74	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
75	`</main>`	Cierra el elemento <main> y delimita el contenido que pertenece a él.
76	`);`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
77	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
78	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
79	`export default Backend;`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.

Financiero/src/pages/Dashboard.css

Hoja de estilos de Dashboard: define presentación, distribución, estados visuales y adaptación de pantalla.

426 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`.dashboard-page {`	Abre las reglas visuales del selector .dashboard-page.
2	`  width: min(1160px, calc(100% - 3rem));`	La propiedad width establece el ancho; valor: min(1160px, calc(100% - 3rem)).
3	`  margin: 0 auto;`	La propiedad margin define el espacio exterior; valor: 0 auto.
4	`  padding: 2.5rem 0 3.5rem;`	La propiedad padding define el espacio interior; valor: 2.5rem 0 3.5rem.
5	`}`	Cierra el bloque CSS actual.
6	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
7	`.dashboard-heading {`	Abre las reglas visuales del selector .dashboard-heading.
8	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
9	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
10	`  justify-content: space-between;`	La propiedad justify-content distribuye elementos en el eje principal; valor: space-between.
11	`  gap: 1.5rem;`	La propiedad gap define el espacio entre elementos; valor: 1.5rem.
12	`  margin-bottom: 1.75rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 1.75rem.
13	`}`	Cierra el bloque CSS actual.
14	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
15	`.dashboard-heading h1 {`	Abre las reglas visuales del selector .dashboard-heading h1.
16	`  font-size: clamp(1.8rem, 3vw, 2.5rem);`	La propiedad font-size define el tamaño de letra; valor: clamp(1.8rem, 3vw, 2.5rem).
17	`}`	Cierra el bloque CSS actual.
18	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
19	`.dashboard-heading .eyebrow {`	Abre las reglas visuales del selector .dashboard-heading .eyebrow.
20	`  margin-bottom: 0.25rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 0.25rem.
21	`}`	Cierra el bloque CSS actual.
22	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
23	`.dashboard-intro {`	Abre las reglas visuales del selector .dashboard-intro.
24	`  margin: 0.45rem 0 0;`	La propiedad margin define el espacio exterior; valor: 0.45rem 0 0.
25	`  color: var(--ink-soft);`	La propiedad color define el color del texto; valor: var(--ink-soft).
26	`}`	Cierra el bloque CSS actual.
27	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
28	`.dashboard-download {`	Abre las reglas visuales del selector .dashboard-download.
29	`  display: inline-flex;`	La propiedad display define cómo se distribuyen los elementos; valor: inline-flex.
30	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
31	`  gap: 0.8rem;`	La propiedad gap define el espacio entre elementos; valor: 0.8rem.
32	`  padding: 0;`	La propiedad padding define el espacio interior; valor: 0.

N.º	Código original	Qué hace esta línea
33	`  color: var(--ink);`	La propiedad color define el color del texto; valor: var(--ink).
34	`  background: none;`	La propiedad background define el fondo; valor: none.
35	`  border: 0;`	La propiedad border define el borde; valor: 0.
36	`  font-size: 0.95rem;`	La propiedad font-size define el tamaño de letra; valor: 0.95rem.
37	`  font-weight: 400;`	La propiedad font-weight define el grosor de letra; valor: 400.
38	`  text-transform: uppercase;`	La propiedad text-transform establece la propiedad visual text-transform; valor: uppercase.
39	`  cursor: pointer;`	La propiedad cursor define el cursor del puntero; valor: pointer.
40	`}`	Cierra el bloque CSS actual.
41	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
42	`.dashboard-download-action {`	Abre las reglas visuales del selector .dashboard-download-action.
43	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
44	`  align-items: flex-end;`	La propiedad align-items alinea elementos en el eje transversal; valor: flex-end.
45	`  flex-direction: column;`	La propiedad flex-direction define la dirección del flexbox; valor: column.
46	`  gap: 0.35rem;`	La propiedad gap define el espacio entre elementos; valor: 0.35rem.
47	`}`	Cierra el bloque CSS actual.
48	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
49	`.download-feedback {`	Abre las reglas visuales del selector .download-feedback.
50	`  margin: 0;`	La propiedad margin define el espacio exterior; valor: 0.
51	`  color: var(--teal-800);`	La propiedad color define el color del texto; valor: var(--teal-800).
52	`  font-size: 0.9rem;`	La propiedad font-size define el tamaño de letra; valor: 0.9rem.
53	`}`	Cierra el bloque CSS actual.
54	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
55	`.download-feedback-error {`	Abre las reglas visuales del selector .download-feedback-error.
56	`  color: #b42318;`	La propiedad color define el color del texto; valor: #b42318.
57	`}`	Cierra el bloque CSS actual.
58	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
59	`.dashboard-download:disabled {`	Abre las reglas visuales del selector . dashboard-download:disabled.
60	`  cursor: not-allowed;`	La propiedad cursor define el cursor del puntero; valor: not-allowed.
61	`  opacity: 0.48;`	La propiedad opacity define la transparencia; valor: 0.48.
62	`}`	Cierra el bloque CSS actual.
63	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
64	`.download-icon {`	Abre las reglas visuales del selector .download-icon.
65	`  display: grid;`	La propiedad display define cómo se distribuyen los elementos; valor: grid.
66	`  width: 44px;`	La propiedad width establece el ancho; valor: 44px.
67	`  height: 44px;`	La propiedad height establece la altura; valor: 44px.
68	`  place-items: center;`	La propiedad place-items establece la propiedad visual place-items; valor: center.
69	`  border-radius: 50%;`	La propiedad border-radius redondea las esquinas; valor: 50%.

N.º	Código original	Qué hace esta línea
70	`  background: var(--teal-800);`	La propiedad background define el fondo; valor: var(--teal-800).
71	`  color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
72	`  font-size: 1.8rem;`	La propiedad font-size define el tamaño de letra; valor: 1.8rem.
73	`  line-height: 1;`	La propiedad line-height define la altura de línea; valor: 1.
74	`}`	Cierra el bloque CSS actual.
75	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
76	`.dashboard-filters {`	Abre las reglas visuales del selector .dashboard-filters.
77	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
78	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
79	`  justify-content: space-between;`	La propiedad justify-content distribuye elementos en el eje principal; valor: space-between.
80	`  flex-wrap: wrap;`	La propiedad flex-wrap controla si los elementos flex pasan a otra fila; valor: wrap.
81	`  gap: 1rem;`	La propiedad gap define el espacio entre elementos; valor: 1rem.
82	`  margin-bottom: 1rem;`	La propiedad margin-bottom establece la propiedad visual margin-bottom; valor: 1rem.
83	`  padding: 0.85rem 1rem;`	La propiedad padding define el espacio interior; valor: 0.85rem 1rem.
84	`  border-bottom: 1px solid rgba(18, 49, 56, 0.16);`	La propiedad border-bottom establece la propiedad visual border-bottom; valor: 1px solid rgba(18, 49, 56, 0.16).
85	`}`	Cierra el bloque CSS actual.
86	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
87	`.filter-periods {`	Abre las reglas visuales del selector .filter-periods.
88	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
89	`  flex-wrap: wrap;`	La propiedad flex-wrap controla si los elementos flex pasan a otra fila; valor: wrap.
90	`  gap: 0.5rem;`	La propiedad gap define el espacio entre elementos; valor: 0.5rem.
91	`}`	Cierra el bloque CSS actual.
92	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
93	`.filter-period {`	Abre las reglas visuales del selector .filter-period.
94	`  padding: 0.55rem 0.9rem;`	La propiedad padding define el espacio interior; valor: 0.55rem 0.9rem.
95	`  border: 1px solid transparent;`	La propiedad border define el borde; valor: 1px solid transparent.
96	`  border-radius: 999px;`	La propiedad border-radius redondea las esquinas; valor: 999px.
97	`  background: transparent;`	La propiedad background define el fondo; valor: transparent.
98	`  color: var(--ink-soft);`	La propiedad color define el color del texto; valor: var(--ink-soft).
99	`  font-family: var(--font-body);`	La propiedad font-family elige la tipografía; valor: var(--font-body).
100	`  font-size: 0.92rem;`	La propiedad font-size define el tamaño de letra; valor: 0.92rem.
101	`  font-weight: 700;`	La propiedad font-weight define el grosor de letra; valor: 700.
102	`  cursor: pointer;`	La propiedad cursor define el cursor del puntero; valor: pointer.
103	`}`	Cierra el bloque CSS actual.
104	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
105	`.filter-period:hover,`	Completa una regla CSS que participa en la presentación visual.
106	`.filter-period.selected {`	Abre las reglas visuales del selector .filter-period.selected.

N.º	Código original	Qué hace esta línea
107	`  border-color: var(--teal-800);`	La propiedad border-color establece la propiedad visual border-color; valor: var(--teal-800).
108	`  background: var(--teal-800);`	La propiedad background define el fondo; valor: var(--teal-800).
109	`  color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
110	`}`	Cierra el bloque CSS actual.
111	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
112	`.filter-dates {`	Abre las reglas visuales del selector .filter-dates.
113	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
114	`  flex-wrap: wrap;`	La propiedad flex-wrap controla si los elementos flex pasan a otra fila; valor: wrap.
115	`  gap: 0.8rem;`	La propiedad gap define el espacio entre elementos; valor: 0.8rem.
116	`}`	Cierra el bloque CSS actual.
117	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
118	`.filter-dates label {`	Abre las reglas visuales del selector .filter-dates label.
119	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
120	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
121	`  gap: 0.45rem;`	La propiedad gap define el espacio entre elementos; valor: 0.45rem.
122	`  color: var(--teal-800);`	La propiedad color define el color del texto; valor: var(--teal-800).
123	`  font-size: 0.85rem;`	La propiedad font-size define el tamaño de letra; valor: 0.85rem.
124	`  font-weight: 700;`	La propiedad font-weight define el grosor de letra; valor: 700.
125	`}`	Cierra el bloque CSS actual.
126	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
127	`.filter-dates input {`	Abre las reglas visuales del selector .filter-dates input.
128	`  padding: 0.5rem 0.6rem;`	La propiedad padding define el espacio interior; valor: 0.5rem 0.6rem.
129	`  border: 1px solid #c9d6d8;`	La propiedad border define el borde; valor: 1px solid #c9d6d8.
130	`  border-radius: var(--radius-sm);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-sm).
131	`  color: var(--ink);`	La propiedad color define el color del texto; valor: var(--ink).
132	`  font: inherit;`	La propiedad font establece la propiedad visual font; valor: inherit.
133	`}`	Cierra el bloque CSS actual.
134	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
135	`.dashboard-total {`	Abre las reglas visuales del selector .dashboard-total.
136	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
137	`  min-height: 210px;`	La propiedad min-height establece la altura mínima; valor: 210px.
138	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
139	`  justify-content: space-between;`	La propiedad justify-content distribuye elementos en el eje principal; valor: space-between.
140	`  gap: 1rem;`	La propiedad gap define el espacio entre elementos; valor: 1rem.
141	`  overflow: hidden;`	La propiedad overflow controla el contenido que desborda; valor: hidden.

N.º	Código original	Qué hace esta línea
142	`  padding: 2rem clamp(1.5rem, 5vw, 3.5rem);`	La propiedad padding define el espacio interior; valor: 2rem clamp(1.5rem, 5vw, 3.5rem).
143	`  border-radius: var(--radius-md);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-md).
144	`  background: linear-gradient(120deg, var(--teal-800), #15919b);`	La propiedad background define el fondo; valor: linear-gradient(120deg, var(--teal-800), #15919b).
145	`  color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
146	`  box-shadow: 0 12px 30px rgba(13, 61, 67, 0.13);`	La propiedad box-shadow añade sombra; valor: 0 12px 30px rgba(13, 61, 67, 0.13).
147	`}`	Cierra el bloque CSS actual.
148	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
149	`.dashboard-total-label {`	Abre las reglas visuales del selector .dashboard-total-label.
150	`  margin: 0 0 0.4rem;`	La propiedad margin define el espacio exterior; valor: 0 0 0.4rem.
151	`  color: #bcecf0;`	La propiedad color define el color del texto; valor: #bcecf0.
152	`  font-size: 1rem;`	La propiedad font-size define el tamaño de letra; valor: 1rem.
153	`  font-weight: 700;`	La propiedad font-weight define el grosor de letra; valor: 700.
154	`  text-transform: uppercase;`	La propiedad text-transform establece la propiedad visual text-transform; valor: uppercase.
155	`  letter-spacing: 0.06em;`	La propiedad letter-spacing establece la propiedad visual letter-spacing; valor: 0.06em.
156	`}`	Cierra el bloque CSS actual.
157	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
158	`.dashboard-total-value {`	Abre las reglas visuales del selector .dashboard-total-value.
159	`  margin: 0;`	La propiedad margin define el espacio exterior; valor: 0.
160	`  color: var(--white);`	La propiedad color define el color del texto; valor: var(--white).
161	`  font-family: var(--font-display);`	La propiedad font-family elige la tipografía; valor: var(--font-display).
162	`  font-size: clamp(2.5rem, 7vw, 4.7rem);`	La propiedad font-size define el tamaño de letra; valor: clamp(2.5rem, 7vw, 4.7rem).
163	`  line-height: 1.08;`	La propiedad line-height define la altura de línea; valor: 1.08.
164	`}`	Cierra el bloque CSS actual.
165	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
166	`.dashboard-total-caption {`	Abre las reglas visuales del selector .dashboard-total-caption.
167	`  margin: 0.65rem 0 0;`	La propiedad margin define el espacio exterior; valor: 0.65rem 0 0.
168	`  color: #e2f6f7;`	La propiedad color define el color del texto; valor: #e2f6f7.
169	`  font-size: 0.95rem;`	La propiedad font-size define el tamaño de letra; valor: 0.95rem.
170	`}`	Cierra el bloque CSS actual.
171	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
172	`.dashboard-error {`	Abre las reglas visuales del selector .dashboard-error.
173	`  margin: 0.8rem 0 0;`	La propiedad margin define el espacio exterior; valor: 0.8rem 0 0.
174	`  color: #ffe4df;`	La propiedad color define el color del texto; valor: #ffe4df.
175	`  font-weight: 600;`	La propiedad font-weight define el grosor de letra; valor: 600.
176	`}`	Cierra el bloque CSS actual.
177	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
178	`.total-decoration {`	Abre las reglas visuales del selector .total-decoration.

N.º	Código original	Qué hace esta línea
179	`  display: grid;`	La propiedad display define cómo se distribuyen los elementos; valor: grid.
180	`  width: 128px;`	La propiedad width establece el ancho; valor: 128px.
181	`  height: 128px;`	La propiedad height establece la altura; valor: 128px.
182	`  flex: 0 0 auto;`	La propiedad flex define el crecimiento y tamaño flexible; valor: 0 0 auto.
183	`  place-items: center;`	La propiedad place-items establece la propiedad visual place-items; valor: center.
184	`  border: 1px solid rgba(255, 255, 255, 0.27);`	La propiedad border define el borde; valor: 1px solid rgba(255, 255, 255, 0.27).
185	`  border-radius: 50%;`	La propiedad border-radius redondea las esquinas; valor: 50%.
186	`  color: rgba(255, 255, 255, 0.23);`	La propiedad color define el color del texto; valor: rgba(255, 255, 255, 0.23).
187	`  font-family: var(--font-display);`	La propiedad font-family elige la tipografía; valor: var(--font-display).
188	`  font-size: 5rem;`	La propiedad font-size define el tamaño de letra; valor: 5rem.
189	`}`	Cierra el bloque CSS actual.
190	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
191	`.dashboard-charts {`	Abre las reglas visuales del selector .dashboard-charts.
192	`  display: grid;`	La propiedad display define cómo se distribuyen los elementos; valor: grid.
193	`  grid-template-columns: minmax(0, 0.9fr) minmax(0, 1.1fr);`	La propiedad grid-template-columns define las columnas de la cuadrícula; valor: minmax(0, 0.9fr) minmax(0, 1.1fr).
194	`  gap: 1.25rem;`	La propiedad gap define el espacio entre elementos; valor: 1.25rem.
195	`  margin-top: 1.25rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 1.25rem.
196	`}`	Cierra el bloque CSS actual.
197	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
198	`.dashboard-chart-card {`	Abre las reglas visuales del selector .dashboard-chart-card.
199	`  min-width: 0;`	La propiedad min-width establece el ancho mínimo; valor: 0.
200	`  padding: 1.5rem;`	La propiedad padding define el espacio interior; valor: 1.5rem.
201	`  border: 1px solid rgba(18, 49, 56, 0.08);`	La propiedad border define el borde; valor: 1px solid rgba(18, 49, 56, 0.08).
202	`  border-radius: var(--radius-md);`	La propiedad border-radius redondea las esquinas; valor: var(--radius-md).
203	`  background: rgba(255, 255, 255, 0.72);`	La propiedad background define el fondo; valor: rgba(255, 255, 255, 0.72).
204	`  box-shadow: 0 6px 20px rgba(18, 49, 56, 0.045);`	La propiedad box-shadow añade sombra; valor: 0 6px 20px rgba(18, 49, 56, 0.045).
205	`}`	Cierra el bloque CSS actual.
206	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
207	`.chart-card-heading h2 {`	Abre las reglas visuales del selector .chart-card-heading h2.
208	`  font-family: var(--font-body);`	La propiedad font-family elige la tipografía; valor: var(--font-body).
209	`  font-size: 1.1rem;`	La propiedad font-size define el tamaño de letra; valor: 1.1rem.
210	`  font-weight: 800;`	La propiedad font-weight define el grosor de letra; valor: 800.
211	`  text-transform: none;`	La propiedad text-transform establece la propiedad visual text-transform; valor: none.
212	`  color: var(--ink);`	La propiedad color define el color del texto; valor: var(--ink).
213	`}`	Cierra el bloque CSS actual.

N.º	Código original	Qué hace esta línea
214	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
215	.chart-card-heading p {	Abre las reglas visuales del selector .chart-card-heading p.
216	margin: 0.25rem 0 0;	La propiedad margin define el espacio exterior; valor: 0.25rem 0 0.
217	color: var(--ink-soft);	La propiedad color define el color del texto; valor: var(--ink-soft).
218	font-size: 0.88rem;	La propiedad font-size define el tamaño de letra; valor: 0.88rem.
219	}	Cierra el bloque CSS actual.
220	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
221	.payment-chart {	Abre las reglas visuales del selector .payment-chart.
222	display: flex;	La propiedad display define cómo se distribuyen los elementos; valor: flex.
223	align-items: center;	La propiedad align-items alinea elementos en el eje transversal; valor: center.
224	justify-content: center;	La propiedad justify-content distribuye elementos en el eje principal; valor: center.
225	gap: clamp(1.2rem, 4vw, 2.5rem);	La propiedad gap define el espacio entre elementos; valor: clamp(1.2rem, 4vw, 2.5rem).
226	min-height: 270px;	La propiedad min-height establece la altura mínima; valor: 270px.
227	padding-top: 1rem;	La propiedad padding-top establece la propiedad visual padding-top; valor: 1rem.
228	}	Cierra el bloque CSS actual.
229	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
230	.payment-donut {	Abre las reglas visuales del selector .payment-donut.
231	display: grid;	La propiedad display define cómo se distribuyen los elementos; valor: grid.
232	width: clamp(150px, 17vw, 210px);	La propiedad width establece el ancho; valor: clamp(150px, 17vw, 210px).
233	aspect-ratio: 1;	La propiedad aspect-ratio establece la propiedad visual aspect-ratio; valor: 1.
234	flex: 0 0 auto;	La propiedad flex define el crecimiento y tamaño flexible; valor: 0 0 auto.
235	place-items: center;	La propiedad place-items establece la propiedad visual place-items; valor: center.
236	border-radius: 50%;	La propiedad border-radius redondea las esquinas; valor: 50%.
237	}	Cierra el bloque CSS actual.
238	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
239	.payment-donut-center {	Abre las reglas visuales del selector .payment-donut-center.
240	display: flex;	La propiedad display define cómo se distribuyen los elementos; valor: flex.
241	width: 58%;	La propiedad width establece el ancho; valor: 58%.
242	aspect-ratio: 1;	La propiedad aspect-ratio establece la propiedad visual aspect-ratio; valor: 1.
243	align-items: center;	La propiedad align-items alinea elementos en el eje transversal; valor: center.
244	justify-content: center;	La propiedad justify-content distribuye elementos en el eje principal; valor: center.
245	flex-direction: column;	La propiedad flex-direction define la dirección del flexbox; valor: column.
246	border-radius: 50%;	La propiedad border-radius redondea las esquinas; valor: 50%.
247	background: #fff;	La propiedad background define el fondo; valor: #fff.
248	}	Cierra el bloque CSS actual.

N.º	Código original	Qué hace esta línea
249	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
250	`.payment-donut-center span {`	Abre las reglas visuales del selector .payment-donut-center span.
251	`  color: var(--teal-800);`	La propiedad color define el color del texto; valor: var(--teal-800).
252	`  font-family: var(--font-display);`	La propiedad font-family elige la tipografía; valor: var(--font-display).
253	`  font-size: 2rem;`	La propiedad font-size define el tamaño de letra; valor: 2rem.
254	`  line-height: 1;`	La propiedad line-height define la altura de línea; valor: 1.
255	`}`	Cierra el bloque CSS actual.
256	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
257	`.payment-donut-center small {`	Abre las reglas visuales del selector .payment-donut-center small.
258	`  color: var(--ink-soft);`	La propiedad color define el color del texto; valor: var(--ink-soft).
259	`  font-size: 0.75rem;`	La propiedad font-size define el tamaño de letra; valor: 0.75rem.
260	`}`	Cierra el bloque CSS actual.
261	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
262	`.payment-legend {`	Abre las reglas visuales del selector .payment-legend.
263	`  display: grid;`	La propiedad display define cómo se distribuyen los elementos; valor: grid.
264	`  gap: 0.75rem;`	La propiedad gap define el espacio entre elementos; valor: 0.75rem.
265	`  margin: 0;`	La propiedad margin define el espacio exterior; valor: 0.
266	`  padding: 0;`	La propiedad padding define el espacio interior; valor: 0.
267	`  list-style: none;`	La propiedad list-style establece la propiedad visual list-style; valor: none.
268	`}`	Cierra el bloque CSS actual.
269	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
270	`.payment-legend li {`	Abre las reglas visuales del selector .payment-legend li.
271	`  display: grid;`	La propiedad display define cómo se distribuyen los elementos; valor: grid.
272	`  grid-template-columns: 10px minmax(70px, 1fr);`	La propiedad grid-template-columns define las columnas de la cuadrícula; valor: 10px minmax(70px, 1fr).
273	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
274	`  gap: 0.5rem;`	La propiedad gap define el espacio entre elementos; valor: 0.5rem.
275	`  font-size: 0.84rem;`	La propiedad font-size define el tamaño de letra; valor: 0.84rem.
276	`}`	Cierra el bloque CSS actual.
277	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
278	`.legend-dot {`	Abre las reglas visuales del selector .legend-dot.
279	`  width: 9px;`	La propiedad width establece el ancho; valor: 9px.
280	`  height: 9px;`	La propiedad height establece la altura; valor: 9px.
281	`  border-radius: 50%;`	La propiedad border-radius redondea las esquinas; valor: 50%.
282	`}`	Cierra el bloque CSS actual.
283	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
284	`.legend-name {`	Abre las reglas visuales del selector .legend-name.
285	`  color: var(--ink-soft);`	La propiedad color define el color del texto; valor: var(--ink-soft).

N.º	Código original	Qué hace esta línea
286	`}`	Cierra el bloque CSS actual.
287	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
288	`.payment-legend strong {`	Abre las reglas visuales del selector .payment-legend strong.
289	`  grid-column: 2;`	La propiedad grid-column establece la propiedad visual grid-column; valor: 2.
290	`  color: var(--ink);`	La propiedad color define el color del texto; valor: var(--ink).
291	`  font-size: 0.88rem;`	La propiedad font-size define el tamaño de letra; valor: 0.88rem.
292	`}`	Cierra el bloque CSS actual.
293	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
294	`.dashboard-line-chart {`	Abre las reglas visuales del selector .dashboard-line-chart.
295	`  display: block;`	La propiedad display define cómo se distribuyen los elementos; valor: block.
296	`  width: 100%;`	La propiedad width establece el ancho; valor: 100%.
297	`  min-height: 245px;`	La propiedad min-height establece la altura mínima; valor: 245px.
298	`  margin-top: 1.1rem;`	La propiedad margin-top establece la propiedad visual margin-top; valor: 1.1rem.
299	`  overflow: visible;`	La propiedad overflow controla el contenido que desborda; valor: visible.
300	`}`	Cierra el bloque CSS actual.
301	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
302	`.chart-grid {`	Abre las reglas visuales del selector .chart-grid.
303	`  stroke: #d7e0e1;`	La propiedad stroke establece la propiedad visual stroke; valor: #d7e0e1.
304	`  stroke-width: 1;`	La propiedad stroke-width establece la propiedad visual stroke-width; valor: 1.
305	`}`	Cierra el bloque CSS actual.
306	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
307	`.chart-axis {`	Abre las reglas visuales del selector .chart-axis.
308	`  fill: #526a6f;`	La propiedad fill establece la propiedad visual fill; valor: #526a6f.
309	`  font-family: var(--font-body);`	La propiedad font-family elige la tipografía; valor: var(--font-body).
310	`  font-size: 11px;`	La propiedad font-size define el tamaño de letra; valor: 11px.
311	`}`	Cierra el bloque CSS actual.
312	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
313	`.chart-line {`	Abre las reglas visuales del selector .chart-line.
314	`  fill: none;`	La propiedad fill establece la propiedad visual fill; valor: none.
315	`  stroke: var(--cyan-500);`	La propiedad stroke establece la propiedad visual stroke; valor: var(--cyan-500).
316	`  stroke-width: 3;`	La propiedad stroke-width establece la propiedad visual stroke-width; valor: 3.
317	`  stroke-linecap: round;`	La propiedad stroke-linecap establece la propiedad visual stroke-linecap; valor: round.
318	`  stroke-linejoin: round;`	La propiedad stroke-linejoin establece la propiedad visual stroke-linejoin; valor: round.
319	`}`	Cierra el bloque CSS actual.
320	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
321	`.chart-point {`	Abre las reglas visuales del selector .chart-point.

N.º	Código original	Qué hace esta línea
322	`  fill: var(--teal-800);`	La propiedad fill establece la propiedad visual fill; valor: var(--teal-800).
323	`  stroke: #fff;`	La propiedad stroke establece la propiedad visual stroke; valor: #fff.
324	`  stroke-width: 2;`	La propiedad stroke-width establece la propiedad visual stroke-width; valor: 2.
325	`}`	Cierra el bloque CSS actual.
326	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
327	`.trend-legend {`	Abre las reglas visuales del selector .trend-legend.
328	`  display: flex;`	La propiedad display define cómo se distribuyen los elementos; valor: flex.
329	`  flex-wrap: wrap;`	La propiedad flex-wrap controla si los elementos flex pasan a otra fila; valor: wrap.
330	`  gap: 0.8rem 1.25rem;`	La propiedad gap define el espacio entre elementos; valor: 0.8rem 1.25rem.
331	`  margin: 0;`	La propiedad margin define el espacio exterior; valor: 0.
332	`  padding: 0 0.5rem 0.25rem;`	La propiedad padding define el espacio interior; valor: 0 0.5rem 0.25rem.
333	`  color: var(--ink-soft);`	La propiedad color define el color del texto; valor: var(--ink-soft).
334	`  font-size: 0.84rem;`	La propiedad font-size define el tamaño de letra; valor: 0.84rem.
335	`  list-style: none;`	La propiedad list-style establece la propiedad visual list-style; valor: none.
336	`}`	Cierra el bloque CSS actual.
337	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
338	`.trend-legend li {`	Abre las reglas visuales del selector .trend-legend li.
339	`  display: inline-flex;`	La propiedad display define cómo se distribuyen los elementos; valor: inline-flex.
340	`  align-items: center;`	La propiedad align-items alinea elementos en el eje transversal; valor: center.
341	`  gap: 0.45rem;`	La propiedad gap define el espacio entre elementos; valor: 0.45rem.
342	`}`	Cierra el bloque CSS actual.
343	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
344	`.chart-empty {`	Abre las reglas visuales del selector .chart-empty.
345	`  display: grid;`	La propiedad display define cómo se distribuyen los elementos; valor: grid.
346	`  min-height: 235px;`	La propiedad min-height establece la altura mínima; valor: 235px.
347	`  place-items: center;`	La propiedad place-items establece la propiedad visual place-items; valor: center.
348	`  margin: 0;`	La propiedad margin define el espacio exterior; valor: 0.
349	`  color: var(--ink-soft);`	La propiedad color define el color del texto; valor: var(--ink-soft).
350	`  text-align: center;`	La propiedad text-align alinea el texto; valor: center.
351	`}`	Cierra el bloque CSS actual.
352	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
353	`.dashboard-footnote {`	Abre las reglas visuales del selector .dashboard-footnote.
354	`  margin: 1rem 0 0;`	La propiedad margin define el espacio exterior; valor: 1rem 0 0.
355	`  color: var(--ink-soft);`	La propiedad color define el color del texto; valor: var(--ink-soft).
356	`  font-size: 0.83rem;`	La propiedad font-size define el tamaño de letra; valor: 0.83rem.
357	`}`	Cierra el bloque CSS actual.

N.º	Código original	Qué hace esta línea
358	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
359	`@media (max-width: 760px) {`	Inicia una condición responsive; sus estilos se aplican cuando se cumple esta condición de pantalla.
360	`.dashboard-page {`	Abre las reglas visuales del selector .dashboard-page.
361	`width: min(100% - 2rem, 620px);`	La propiedad width establece el ancho; valor: min(100% - 2rem, 620px).
362	`padding-top: 1.75rem;`	La propiedad padding-top establece la propiedad visual padding-top; valor: 1.75rem.
363	`}`	Cierra el bloque CSS actual.
364	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
365	`.dashboard-heading {`	Abre las reglas visuales del selector .dashboard-heading.
366	`align-items: flex-start;`	La propiedad align-items alinea elementos en el eje transversal; valor: flex-start.
367	`}`	Cierra el bloque CSS actual.
368	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
369	`.dashboard-download {`	Abre las reglas visuales del selector .dashboard-download.
370	`gap: 0.45rem;`	La propiedad gap define el espacio entre elementos; valor: 0.45rem.
371	`font-size: 0.75rem;`	La propiedad font-size define el tamaño de letra; valor: 0.75rem.
372	`}`	Cierra el bloque CSS actual.
373	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
374	`.download-icon {`	Abre las reglas visuales del selector .download-icon.
375	`width: 38px;`	La propiedad width establece el ancho; valor: 38px.
376	`height: 38px;`	La propiedad height establece la altura; valor: 38px.
377	`}`	Cierra el bloque CSS actual.
378	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
379	`.dashboard-charts {`	Abre las reglas visuales del selector .dashboard-charts.
380	`grid-template-columns: 1fr;`	La propiedad grid-template-columns define las columnas de la cuadrícula; valor: 1fr.
381	`}`	Cierra el bloque CSS actual.
382	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
383	`.payment-chart {`	Abre las reglas visuales del selector .payment-chart.
384	`justify-content: space-around;`	La propiedad justify-content distribuye elementos en el eje principal; valor: space-around.
385	`}`	Cierra el bloque CSS actual.
386	`}`	Cierra el bloque CSS actual.
387	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
388	`@media (max-width: 480px) {`	Inicia una condición responsive; sus estilos se aplican cuando se cumple esta condición de pantalla.
389	`.dashboard-heading {`	Abre las reglas visuales del selector .dashboard-heading.
390	`flex-direction: column;`	La propiedad flex-direction define la dirección del flexbox; valor: column.
391	`}`	Cierra el bloque CSS actual.
392	(línea vacía)	Línea en blanco que separa visualmente reglas o bloques de estilos.
393	`.dashboard-filters {`	Abre las reglas visuales del selector .dashboard-filters.

N.º	Código original	Qué hace esta línea
394	`    padding-inline: 0;`	La propiedad padding-inline establece la propiedad visual padding-inline; valor: 0.
395	`  }`	Cierra el bloque CSS actual.
396	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
397	`  .filter-period {`	Abre las reglas visuales del selector .filter-period.
398	`    padding-inline: 0.7rem;`	La propiedad padding-inline establece la propiedad visual padding-inline; valor: 0.7rem.
399	`    font-size: 0.84rem;`	La propiedad font-size define el tamaño de letra; valor: 0.84rem.
400	`  }`	Cierra el bloque CSS actual.
401	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
402	`  .dashboard-total {`	Abre las reglas visuales del selector .dashboard-total.
403	`    min-height: 175px;`	La propiedad min-height establece la altura mínima; valor: 175px.
404	`    padding: 1.5rem;`	La propiedad padding define el espacio interior; valor: 1.5rem.
405	`  }`	Cierra el bloque CSS actual.
406	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
407	`  .total-decoration {`	Abre las reglas visuales del selector .total-decoration.
408	`    width: 68px;`	La propiedad width establece el ancho; valor: 68px.
409	`    height: 68px;`	La propiedad height establece la altura; valor: 68px.
410	`    font-size: 2.8rem;`	La propiedad font-size define el tamaño de letra; valor: 2.8rem.
411	`  }`	Cierra el bloque CSS actual.
412	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
413	`  .dashboard-total-value {`	Abre las reglas visuales del selector .dashboard-total-value.
414	`    font-size: 2.35rem;`	La propiedad font-size define el tamaño de letra; valor: 2.35rem.
415	`  }`	Cierra el bloque CSS actual.
416	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
417	`  .payment-chart {`	Abre las reglas visuales del selector .payment-chart.
418	`    flex-direction: column;`	La propiedad flex-direction define la dirección del flexbox; valor: column.
419	`    padding-bottom: 0.5rem;`	La propiedad padding-bottom establece la propiedad visual padding-bottom; valor: 0.5rem.
420	`  }`	Cierra el bloque CSS actual.
421	`(línea vacía)`	Línea en blanco que separa visualmente reglas o bloques de estilos.
422	`  .payment-legend {`	Abre las reglas visuales del selector .payment-legend.
423	`    grid-template-columns: repeat(2, minmax(0, 1fr));`	La propiedad grid-template-columns define las columnas de la cuadrícula; valor: repeat(2, minmax(0, 1fr)).
424	`    width: 100%;`	La propiedad width establece el ancho; valor: 100%.
425	`  }`	Cierra el bloque CSS actual.
426	`}`	Cierra el bloque CSS actual.

Financiero/src/pages/Dashboard.jsx

Consulta ingresos, filtra fechas, calcula indicadores y presenta gráficos del panel.

554 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { useEffect, useMemo, useState } from 'react'`	Importa { useEffect, useMemo, useState } desde react y conecta este archivo con ese recurso.
2	`import apiClient from '../services/api.js'`	Importa apiClient desde ../services/api.js y conecta este archivo con ese recurso.
3	`import { INCOME_DATA_CHANGED } from '../services/incomeEvents.js'`	Importa { INCOME_DATA_CHANGED } desde ./services/incomeEvents.js y conecta este archivo con ese recurso.
4	`import './Dashboard.css'`	Importa './Dashboard.css' desde el módulo indicado y conecta este archivo con ese recurso.
5	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
6	`const API_URL = import.meta.env.VITE_API_URL`	Lee en API_URL una variable de configuración expuesta por Vite al frontend.
7	`const CUENTA_NO_APLICA = 'No aplica'`	Declara CUENTA_NO_APLICA y le asigna el valor o expresión mostrado para reutilizarlo.
8	`const formatoMoneda = new Intl.NumberFormat('es-CO', {`	Declara formatoMoneda y le asigna el valor o expresión mostrado para reutilizarlo.
9	`  style: 'currency',`	Define el valor de la propiedad «style» del objeto actual.
10	`  currency: 'COP',`	Define el valor de la propiedad «currency» del objeto actual.
11	`  maximumFractionDigits: 0,`	Define el valor de la propiedad «maximumFractionDigits» del objeto actual.
12	`})`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
13	`const formatoFecha = new Intl.DateTimeFormat('es-CO', {`	Declara formatoFecha y le asigna el valor o expresión mostrado para reutilizarlo.
14	`  day: '2-digit',`	Define el valor de la propiedad «day» del objeto actual.
15	`  month: 'short',`	Define el valor de la propiedad «month» del objeto actual.
16	`  timeZone: 'UTC',`	Define el valor de la propiedad «timeZone» del objeto actual.
17	`})`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
18	`const coloresMetodo = {`	Inicia en coloresMetodo una estructura de datos que agrupa valores relacionados.
19	`  transferencia: '#168b96',`	Define el valor de la propiedad «transferencia» del objeto actual.
20	`  efectivo: '#42bdc8',`	Define el valor de la propiedad «efectivo» del objeto actual.
21	`  tarjeta: '#376b9c',`	Define el valor de la propiedad «tarjeta» del objeto actual.
22	`  otro: '#343c75',`	Define el valor de la propiedad «otro» del objeto actual.
23	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
24	`const metodosPago = {`	Inicia en metodosPago una estructura de datos que agrupa valores relacionados.
25	`  transferencia: 'Transferencia',`	Define el valor de la propiedad «transferencia» del objeto actual.
26	`  efectivo: 'Efectivo',`	Define el valor de la propiedad «efectivo» del objeto actual.
27	`  tarjeta: 'Tarjeta',`	Define el valor de la propiedad «tarjeta» del objeto actual.
28	`  otro: 'Otro',`	Define el valor de la propiedad «otro» del objeto actual.
29	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
30	`const colorMetodo = (metodo) => coloresMetodo[metodo] || coloresMetodo.otro`	Define en colorMetodo una función flecha para reutilizar o ejecutar una acción.
31	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
32	`function fechaBogotaIso(fecha) {`	Declara la función fechaBogotaIso; el bloque siguiente contiene las instrucciones que ejecuta.

N.º	Código original	Qué hace esta línea
33	`  const partes = new Intl.DateTimeFormat('en-CA', {`	Declara partes y le asigna el valor o expresión mostrado para reutilizarlo.
34	`    timeZone: 'America/Bogota',`	Define el valor de la propiedad «timeZone» del objeto actual.
35	`    year: 'numeric',`	Define el valor de la propiedad «year» del objeto actual.
36	`    month: '2-digit',`	Define el valor de la propiedad «month» del objeto actual.
37	`    day: '2-digit',`	Define el valor de la propiedad «day» del objeto actual.
38	`  }).formatToParts(fecha)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
39	`  const valores = Object.fromEntries(partes.map(({ type, value }) => [type, value]))`	Define en valores una función flecha para reutilizar o ejecutar una acción.
40	`  const { year: anio, month: mes, day: dia } = valores`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
41	`  return `${anio}-${mes}-${dia}``	Devuelve un valor de la función o el elemento React que se renderizará.
42	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
43	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
44	`function fechaUtc(iso) {`	Declara la función fechaUtc; el bloque siguiente contiene las instrucciones que ejecuta.
45	`  return new Date(`${iso}T00:00:00Z`)`	Devuelve un valor de la función o el elemento React que se renderizará.
46	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
47	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
48	`function fechaIsoUtc(fecha) {`	Declara la función fechaIsoUtc; el bloque siguiente contiene las instrucciones que ejecuta.
49	`  return fecha.toISOString().slice(0, 10)`	Devuelve un valor de la función o el elemento React que se renderizará.
50	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
51	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
52	`function obtenerRango(periodo, hoy) {`	Declara la función obtenerRango; el bloque siguiente contiene las instrucciones que ejecuta.
53	`  const fin = fechaUtc(hoy)`	Declara fin y le asigna el valor o expresión mostrado para reutilizarlo.
54	`  let inicio = fechaUtc(hoy)`	Declara inicio y le asigna el valor o expresión mostrado para reutilizarlo.
55	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
56	`  if (periodo === 'semana') {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
57	`    inicio.setUTCDate(inicio.getUTCDate() - ((inicio.getUTCDay() + 6) % 7))`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
58	`  } else if (periodo === 'mes') {`	Abre el bloque de código que contiene las instrucciones siguientes.
59	`    inicio.setUTCDate(1)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
60	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
61	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
62	`  return { desde: fechaIsoUtc(inicio), hasta: fechaIsoUtc(fin) }`	Devuelve un valor de la función o el elemento React que se renderizará.
63	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
64	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
65	`function resumirIngresos(ingresos, desde, hasta) {`	Declara la función resumirIngresos; el bloque siguiente contiene las instrucciones que ejecuta.

N.º	Código original	Qué hace esta línea
66	`  const diariosPorMetodo = new Map()`	Declara diariosPorMetodo y le asigna el valor o expresión mostrado para reutilizarlo.
67	`  const metodos = new Map()`	Declara metodos y le asigna el valor o expresión mostrado para reutilizarlo.
68	`  let total = 0`	Declara total y le asigna el valor o expresión mostrado para reutilizarlo.
69	`  let cantidad = 0`	Declara cantidad y le asigna el valor o expresión mostrado para reutilizarlo.
70	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
71	`  ingresos.forEach((ingreso) => {`	Abre el bloque de código que contiene las instrucciones siguientes.
72	`    const fecha = String(ingreso.fechaPago).slice(0, 10)`	Declara fecha y le asigna el valor o expresión mostrado para reutilizarlo.
73	`    if (fecha < desde || fecha > hasta) return`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
74	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
75	`    const valor = Number(ingreso.valor)`	Declara valor y le asigna el valor o expresión mostrado para reutilizarlo.
76	`    if (!Number.isFinite(valor)) return`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
77	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
78	`    total += valor`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
79	`    cantidad += 1`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
80	`    const metodo = ingreso.metodoPago || 'otro'`	Declara metodo y le asigna el valor o expresión mostrado para reutilizarlo.
81	`    metodos.set(metodo, (metodos.get(metodo) || 0) + valor)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
82	`    if (!diariosPorMetodo.has(metodo)) diariosPorMetodo.set(metodo, new Map())`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
83	`    const diarios = diariosPorMetodo.get(metodo)`	Declara diarios y le asigna el valor o expresión mostrado para reutilizarlo.
84	`    diarios.set(fecha, (diarios.get(fecha) || 0) + valor)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
85	`  })`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
86	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
87	`  return { total, cantidad, diariosPorMetodo, metodos }`	Devuelve un valor de la función o el elemento React que se renderizará.
88	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
89	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
90	`function obtenerTendencia(diariosPorMetodo, desde, hasta) {`	Declara la función obtenerTendencia; el bloque siguiente contiene las instrucciones que ejecuta.
91	`  const inicio = fechaUtc(desde)`	Declara inicio y le asigna el valor o expresión mostrado para reutilizarlo.
92	`  const fin = fechaUtc(hasta)`	Declara fin y le asigna el valor o expresión mostrado para reutilizarlo.
93	`  const dias = Math.floor((fin - inicio) / 86400000) + 1`	Declara dias y le asigna el valor o expresión mostrado para reutilizarlo.
94	`  const periodo = dias <= 14 ? 'dia' : dias <= 90 ? 'semana' : 'mes'`	Declara periodo y le asigna el valor o expresión mostrado para reutilizarlo.
95	`  const acumuladosPorMetodo = new Map()`	Declara acumuladosPorMetodo y le asigna el valor o expresión mostrado para reutilizarlo.
96	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
97	`  diariosPorMetodo.forEach((diarios, metodo) => {`	Abre el bloque de código que contiene las instrucciones siguientes.

N.º	Código original	Qué hace esta línea
98	`    const acumulados = new Map()`	Declara acumulados y le asigna el valor o expresión mostrado para reutilizarlo.
99	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
100	`    diarios.forEach((valor, fecha) => {`	Abre el bloque de código que contiene las instrucciones siguientes.
101	`      const dia = fechaUtc(fecha)`	Declara dia y le asigna el valor o expresión mostrado para reutilizarlo.
102	`      let clave = fecha`	Declara clave y le asigna el valor o expresión mostrado para reutilizarlo.
103	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
104	`      if (periodo === 'semana') {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
105	`        dia.setUTCDate(dia.getUTCDate() - ((dia.getUTCDay() + 6) % 7))`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
106	`        clave = fechaIsoUtc(dia)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
107	`      } else if (periodo === 'mes') {`	Abre el bloque de código que contiene las instrucciones siguientes.
108	`        clave = fecha.slice(0, 7)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
109	`      }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
110	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
111	`      acumulados.set(clave, (acumulados.get(clave) || 0) + valor)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
112	`    })`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
113	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
114	`    acumuladosPorMetodo.set(metodo, acumulados)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
115	`  })`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
116	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
117	`  const claves = []`	Inicia en claves una estructura de datos que agrupa valores relacionados.
118	`  const cursor = new Date(inicio)`	Declara cursor y le asigna el valor o expresión mostrado para reutilizarlo.
119	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
120	`  if (periodo === 'semana') {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
121	`    cursor.setUTCDate(cursor.getUTCDate() - ((cursor.getUTCDay() + 6) % 7))`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
122	`  } else if (periodo === 'mes') {`	Abre el bloque de código que contiene las instrucciones siguientes.
123	`    cursor.setUTCDate(1)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
124	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
125	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
126	`  while (cursor <= fin) {`	Repite el bloque siguiente mientras la condición sea verdadera.
127	`    claves.push(periodo === 'mes' ? cursor.toISOString().slice(0, 7) : cursor.toISOString().slice(0, 10))`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
128	`    if (periodo === 'dia') cursor.setUTCDate(cursor.getUTCDate() + 1)`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
129	`    else if (periodo === 'semana') cursor.setUTCDate(cursor.getUTCDate() + 7)`	Define la alternativa que se ejecuta si no se cumple la condición anterior.

N.º	Código original	Qué hace esta línea
130	`else cursor.setUTCMonth(cursor.getUTCMonth() + 1)`	Define la alternativa que se ejecuta si no se cumple la condición anterior.
131	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
132	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
133	`return claves.map((fecha) => ({`	Devuelve un valor de la función o el elemento React que se renderizará.
134	`fecha,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
135	`valores: Object.fromEntries(`	Define el valor de la propiedad «valores» del objeto actual.
136	`[...acumuladosPorMetodo.entries()].map(([metodo, acumulados]) => [`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
137	`metodo,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
138	`acumulados.get(fecha) || 0,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
139	`]),`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
140	`),`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
141	`etiqueta:`	Define el valor de la propiedad «etiqueta» del objeto actual.
142	`periodo === 'mes'`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
143	`? new Intl.DateTimeFormat('es-CO', { month: 'short', year: '2-digit', timeZone: 'UTC' })`	Completa una condición, encadenamiento, separador o expresión anterior.
144	`.format(fechaUtc(`${fecha}-01`))`	Completa una condición, encadenamiento, separador o expresión anterior.
145	`: formatoFecha.format(fechaUtc(fecha)),`	Completa una condición, encadenamiento, separador o expresión anterior.
146	`}))`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
147	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
148	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
149	`function graficoLinea(puntos, metodos) {`	Declara la función graficoLinea; el bloque siguiente contiene las instrucciones que ejecuta.
150	`if (puntos.length === 0) return null`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
151	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
152	`const ancho = 640`	Declara ancho y le asigna el valor o expresión mostrado para reutilizarlo.
153	`const alto = 240`	Declara alto y le asigna el valor o expresión mostrado para reutilizarlo.
154	`const margen = { arriba: 18, derecha: 14, abajo: 42, izquierda: 70 }`	Inicia en margen una estructura de datos que agrupa valores relacionados.
155	`const valores = puntos.flatMap((punto) => Object.values(punto.valores))`	Define en valores una función flecha para reutilizar o ejecutar una acción.
156	`const maximo = Math.max(...valores, 1)`	Declara maximo y le asigna el valor o expresión mostrado para reutilizarlo.
157	`const minimo = Math.min(...valores, 0)`	Declara minimo y le asigna el valor o expresión mostrado para reutilizarlo.
158	`const rango = maximo - minimo || 1`	Declara rango y le asigna el valor o expresión mostrado para reutilizarlo.
159	`const coordenadas = metodos.map((metodo) => ({`	Define en coordenadas una función flecha para reutilizar o ejecutar una acción.
160	`metodo,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
161	`puntos: puntos.map((punto, indice) => ({`	Define el valor de la propiedad «puntos» del objeto actual.

N.º	Código original	Qué hace esta línea
162	`...punto,`	Completa una condición, encadenamiento, separador o expresión anterior.
163	`x:`	Define el valor de la propiedad «x» del objeto actual.
164	`margen.izquierda +`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
165	`(indice * (ancho - margen.izquierda - margen.derecha)) / Math.max(puntos.length - 1, 1),`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
166	`y:`	Define el valor de la propiedad «y» del objeto actual.
167	`margen.arriba +`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
168	`((maximo - punto.valores[metodo]) * (alto - margen.arriba - margen.abajo)) / rango,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
169	`})),`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
170	`}))`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
171	`const divisiones = [0, 1, 2, 3].map((indice) => ({`	Define en divisiones una función flecha para reutilizar o ejecutar una acción.
172	`valor: maximo - (indice * (maximo - minimo)) / 3,`	Define el valor de la propiedad «valor» del objeto actual.
173	`y: margen.arriba + (indice * (alto - margen.arriba - margen.abajo)) / 3,`	Define el valor de la propiedad «y» del objeto actual.
174	`}))`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
175	`const etiquetas = puntos.length <= 6`	Declara etiquetas y le asigna el valor o expresión mostrado para reutilizarlo.
176	`? puntos.map((punto, indice) => ({ punto, indice }))`	Completa una condición, encadenamiento, separador o expresión anterior.
177	`: [0, Math.floor((puntos.length - 1) / 2), puntos.length - 1].map((indice) => ({`	Completa una condición, encadenamiento, separador o expresión anterior.
178	`punto: puntos[indice],`	Define el valor de la propiedad «punto» del objeto actual.
179	`indice,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
180	`}))`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
181	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
182	`return (`	Devuelve un valor de la función o el elemento React que se renderizará.
183	`<>`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
184	`<svg`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
185	`className="dashboard-line-chart"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
186	`viewBox={`0 0 ${ancho} ${alto}`}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
187	`role="img"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
188	`aria-label="Evolución de ingresos por método de pago"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
189	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
190	`{divisiones.map((division) => (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
191	`<g key={division.y}>`	Abre el elemento <g>; sus atributos configuran contenido, accesibilidad o eventos.
192	`<line`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.

N.º	Código original	Qué hace esta línea
193	`x1={margen.izquierda}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
194	`x2={ancho - margen.derecha}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
195	`y1={division.y}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
196	`y2={division.y}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
197	`className="chart-grid"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
198	`/>`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
199	`<text x={margen.izquierda - 10} y={division.y + 4} textAnchor="end" className="chart-axis">`	Abre el elemento <text>; sus atributos configuran contenido, accesibilidad o eventos.
200	`{new Intl.NumberFormat('es-CO', { notation: 'compact', maximumFractionDigits: 1 }).format(`	Abre un bloque de código, objeto o expresión JSX.
201	`division.valor,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
202	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
203	`</text>`	Cierra el elemento <text> y delimita el contenido que pertenece a él.
204	`</g>`	Cierra el elemento <g> y delimita el contenido que pertenece a él.
205	`))}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
206	`{coordenadas.map(({ metodo, puntos: serie }) => (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
207	`<g key={metodo}>`	Abre el elemento <g>; sus atributos configuran contenido, accesibilidad o eventos.
208	`<polyline`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
209	`points={serie.map(({ x, y }) => `${x},${y}`).join(' ')}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
210	`className="chart-line"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
211	`style={{ stroke: colorMetodo(metodo) }}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
212	`/>`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
213	`{serie.map((punto) => (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
214	`<circle`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
215	`key={`${metodo}-${punto.fecha}`}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
216	`cx={punto.x}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
217	`cy={punto.y}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
218	`r="4"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
219	`className="chart-point"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
220	`style={{ fill: colorMetodo(metodo) }}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
221	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
222	`<title>{`${metodosPago[metodo] || metodo} · ${punto.etiqueta}: ${formatoMoneda.format(`	Abre el elemento <title>; sus atributos configuran contenido, accesibilidad o eventos.

N.º	Código original	Qué hace esta línea
223	`punto.valores[metodo],`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
224	`)}`}</title>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
225	`</circle>`	Cierra el elemento <circle> y delimita el contenido que pertenece a él.
226	`))}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
227	`</g>`	Cierra el elemento <g> y delimita el contenido que pertenece a él.
228	`))}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
229	`{etiquetas.map(({ punto, indice }) => (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
230	`<text`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
231	`key={punto.fecha}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
232	`x={coordenadas[0].puntos[indice].x}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
233	`y={alto - 12}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
234	`textAnchor="middle"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
235	`className="chart-axis"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
236	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
237	`{punto.etiqueta}`	Abre un bloque de código, objeto o expresión JSX.
238	`</text>`	Cierra el elemento <text> y delimita el contenido que pertenece a él.
239	`))}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
240	`</svg>`	Cierra el elemento <svg> y delimita el contenido que pertenece a él.
241	`<ul className="trend-legend" aria-label="Leyenda de métodos de pago">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
242	`{metodos.map((metodo) => (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
243	`<li key={metodo}>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
244	`<span className="legend-dot" style={{ backgroundColor: colorMetodo(metodo) }} />`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
245	`<span>{metodosPago[metodo] || metodo}</span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
246	`</li>`	Cierra el elemento y delimita el contenido que pertenece a él.
247	`))}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
248	`</ul>`	Cierra el elemento y delimita el contenido que pertenece a él.
249	`</>`	Cierra un elemento de la plantilla JSX.
250	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
251	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
252	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
253	`function Dashboard() {`	Declara la función Dashboard; el bloque siguiente contiene las instrucciones que ejecuta.

N.º	Código original	Qué hace esta línea
254	`  const hoy = fechaBogotaIso(new Date())`	Declara hoy y le asigna el valor o expresión mostrado para reutilizarlo.
255	`  const rangoInicial = obtenerRango('hoy', hoy)`	Declara rangoInicial y le asigna el valor o expresión mostrado para reutilizarlo.
256	`  const [periodo, setPeriodo] = useState('hoy')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
257	`  const [desde, setDesde] = useState(rangoInicial.desde)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
258	`  const [hasta, setHasta] = useState(rangoInicial.hasta)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
259	`  const [ingresos, setIngresos] = useState([])`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
260	`  const [cargando, setCargando] = useState(true)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
261	`  const [error, setError] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
262	`  const [errorDescarga, setErrorDescarga] = useState('')`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
263	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
264	`  useEffect(() => {`	Registra un efecto React para sincronizar la interfaz con eventos o recursos externos.
265	`    let activo = true`	Declara activo y le asigna el valor o expresión mostrado para reutilizarlo.
266	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
267	`    async function cargarIngresos() {`	Declara la función cargarIngresos; el bloque siguiente contiene las instrucciones que ejecuta.
268	`      try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
269	`        if (!API_URL) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
270	`          throw new Error('Falta configurar VITE_API_URL en el archivo .env del frontend.')`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
271	`        }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
272	`        const respuesta = await apiClient.get(`${API_URL.replace(/\/+$/, '')}/ingresos`)`	Espera una petición HTTP al backend antes de continuar.
273	`        if (activo) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
274	`          setIngresos(respuesta.data)`	Invoca setIngresos para ejecutar la operación definida por esa función o biblioteca.
275	`          setError('')`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
276	`        }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
277	`      } catch (errorSolicitud) {`	Abre el bloque de código que contiene las instrucciones siguientes.
278	`        if (activo) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
279	`          setError(`	Invoca setError para ejecutar la operación definida por esa función o biblioteca.
280	`            errorSolicitud.response?.data?.message ||`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
281	`              errorSolicitud.message ||`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
282	`              'No fue posible consultar los ingresos.',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
283	`          )`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
284	`        }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.

N.º	Código original	Qué hace esta línea
285	`} finally {`	Abre el bloque de código que contiene las instrucciones siguientes.
286	`if (activo) setCargando(false)`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
287	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
288	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
289	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
290	`function actualizarAlVolver() {`	Declara la función actualizarAlVolver; el bloque siguiente contiene las instrucciones que ejecuta.
291	`if (document.visibilityState === 'visible') cargarIngresos()`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
292	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
293	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
294	`cargarIngresos()`	Invoca cargarIngresos para ejecutar la operación definida por esa función o biblioteca.
295	`window.addEventListener(INCOME_DATA_CHANGED, cargarIngresos)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
296	`window.addEventListener('focus', actualizarAlVolver)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
297	`document.addEventListener('visibilitychange', actualizarAlVolver)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
298	`return () => {`	Devuelve un valor de la función o el elemento React que se renderizará.
299	`activo = false`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
300	`window.removeEventListener(INCOME_DATA_CHANGED, cargarIngresos)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
301	`window.removeEventListener('focus', actualizarAlVolver)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
302	`document.removeEventListener('visibilitychange', actualizarAlVolver)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
303	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
304	`}, [])`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
305	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
306	`const rangoValido = desde && hasta && desde <= hasta`	Declara rangoValido y le asigna el valor o expresión mostrado para reutilizarlo.
307	`const resumen = useMemo(`	Declara resumen y le asigna el valor o expresión mostrado para reutilizarlo.
308	`() => (rangoValido ? resumirIngresos(ingresos, desde, hasta) : null),`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
309	`[ingresos, desde, hasta, rangoValido],`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
310	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
311	`const tendencia = useMemo(`	Declara tendencia y le asigna el valor o expresión mostrado para reutilizarlo.
312	`() => (resumen ? obtenerTendencia(resumen.diariosPorMetodo, desde, hasta) : []),`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
313	`[resumen, desde, hasta],`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
314	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
315	`const metodosOrdenados = useMemo(`	Declara metodosOrdenados y le asigna el valor o expresión mostrado para reutilizarlo.

N.º	Código original	Qué hace esta línea
316	`() =>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
317	`resumen`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
318	`? [...resumen.metodos.entries()].sort(([, valorA], [, valorB]) => valorB - valorA)`	Completa una condición, encadenamiento, separador o expresión anterior.
319	`: [],`	Completa una condición, encadenamiento, separador o expresión anterior.
320	`[resumen],`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
321	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
322	`const totalMetodos = resumen`	Declara totalMetodos y le asigna el valor o expresión mostrado para reutilizarlo.
323	`? metodosOrdenados.reduce((suma, [, valor]) => suma + valor, 0)`	Completa una condición, encadenamiento, separador o expresión anterior.
324	`: 0`	Completa una condición, encadenamiento, separador o expresión anterior.
325	`const etiquetaPeriodo =`	Declara etiquetaPeriodo y le asigna el valor o expresión mostrado para reutilizarlo.
326	`periodo === 'hoy'`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
327	`? 'Hoy'`	Completa una condición, encadenamiento, separador o expresión anterior.
328	`: periodo === 'semana'`	Completa una condición, encadenamiento, separador o expresión anterior.
329	`? 'Esta semana'`	Completa una condición, encadenamiento, separador o expresión anterior.
330	`: periodo === 'mes'`	Completa una condición, encadenamiento, separador o expresión anterior.
331	`? 'Este mes'`	Completa una condición, encadenamiento, separador o expresión anterior.
332	`: 'Periodo personalizado'`	Completa una condición, encadenamiento, separador o expresión anterior.
333	*(línea vacía)*	Línea en blanco que separa instrucciones o bloques del código.
334	`function cambiarPeriodo(nuevoPeriodo) {`	Declara la función cambiarPeriodo; el bloque siguiente contiene las instrucciones que ejecuta.
335	`setPeriodo(nuevoPeriodo)`	Invoca setPeriodo para ejecutar la operación definida por esa función o biblioteca.
336	`if (nuevoPeriodo !== 'personalizado') {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
337	`const rango = obtenerRango(nuevoPeriodo, fechaBogotaIso(new Date()))`	Declara rango y le asigna el valor o expresión mostrado para reutilizarlo.
338	`setDesde(rango.desde)`	Invoca setDesde para ejecutar la operación definida por esa función o biblioteca.
339	`setHasta(rango.hasta)`	Invoca setHasta para ejecutar la operación definida por esa función o biblioteca.
340	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
341	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
342	*(línea vacía)*	Línea en blanco que separa instrucciones o bloques del código.
343	`function descargarReporte() {`	Declara la función descargarReporte; el bloque siguiente contiene las instrucciones que ejecuta.
344	`if (!resumen) return`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
345	`setErrorDescarga('')`	Invoca setErrorDescarga para ejecutar la operación definida por esa función o biblioteca.
346	*(línea vacía)*	Línea en blanco que separa instrucciones o bloques del código.

N.º	Código original	Qué hace esta línea
347	`try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
348	`const columnas = [`	Inicia en columnas una estructura de datos que agrupa valores relacionados.
349	`'ID ingreso',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
350	`'ID usuario',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
351	`'Cliente',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
352	`'Correo del cliente',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
353	`'Cuenta bancaria',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
354	`'Método de pago',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
355	`'Fecha de pago',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
356	`'Valor (COP)',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
357	`'Observación',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
358	`'Fecha de registro',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
359	`]`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
360	`const filas = ingresos`	Declara filas y le asigna el valor o expresión mostrado para reutilizarlo.
361	`.filter((ingreso) => {`	Completa una condición, encadenamiento, separador o expresión anterior.
362	`const fecha = String(ingreso.fechaPago).slice(0, 10)`	Declara fecha y le asigna el valor o expresión mostrado para reutilizarlo.
363	`return fecha >= desde && fecha <= hasta`	Devuelve un valor de la función o el elemento React que se renderizará.
364	`})`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
365	`.map((ingreso) => [`	Completa una condición, encadenamiento, separador o expresión anterior.
366	`ingreso.id,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
367	`ingreso.usuarioId,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
368	`ingreso.cliente,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
369	`ingreso.clienteEmail,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
370	`ingreso.cuentaBancaria == null ? CUENTA_NO_APLICA : String(ingreso.cuentaBancaria),`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
371	`metodosPago[ingreso.metodoPago] || ingreso.metodoPago,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
372	`String(ingreso.fechaPago).slice(0, 10),`	Invoca String para ejecutar la operación definida por esa función o biblioteca.
373	`ingreso.valor,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
374	`ingreso.observacion || '',`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
375	`ingreso.creadoEn ? new Date(ingreso.creadoEn).toLocaleString('es-CO') : '',`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
376	`])`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.

N.º	Código original	Qué hace esta línea
377	`const escaparCampo = (valor) => `"${String(valor ?? '').replace(/"/g, '""')}"`	Define en escaparCampo una función flecha para reutilizar o ejecutar una acción.
378	`const contenido = `\uFEFF${[columnas, ...filas]`	Declara contenido y le asigna el valor o expresión mostrado para reutilizarlo.
379	`.map((fila) => fila.map(escaparCampo).join(',' ))`	Completa una condición, encadenamiento, separador o expresión anterior.
380	`.join('\r\n')}`	Completa una condición, encadenamiento, separador o expresión anterior.
381	`const archivo = new Blob([contenido], { type: 'text/csv;charset=utf-8' })`	Declara archivo y le asigna el valor o expresión mostrado para reutilizarlo.
382	`const url = URL.createObjectURL(archivo)`	Declara url y le asigna el valor o expresión mostrado para reutilizarlo.
383	`const enlace = document.createElement('a')`	Declara enlace y le asigna el valor o expresión mostrado para reutilizarlo.
384	`enlace.href = url`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
385	`enlace.download = `ReporteIngresos_${fechaBogotaIso(new Date())}.csv`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
386	`document.body.appendChild(enlace)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
387	`enlace.click()`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
388	`enlace.remove()`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
389	`setTimeout(() => URL.revokeObjectURL(url), 1000)`	Invoca setTimeout para ejecutar la operación definida por esa función o biblioteca.
390	`} catch (error) {`	Abre el bloque de código que contiene las instrucciones siguientes.
391	`console.error('No fue posible generar el reporte de ingresos en CSV:', error)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
392	`setErrorDescarga('No fue posible descargar el reporte CSV. Intenta nuevamente.')`	Invoca setErrorDescarga para ejecutar la operación definida por esa función o biblioteca.
393	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
394	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
395	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
396	`return (`	Devuelve un valor de la función o el elemento React que se renderizará.
397	`<section className="dashboard-page">`	Abre el elemento <section>; sus atributos configuran contenido, accesibilidad o eventos.
398	`<div className="dashboard-heading">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
399	`<div>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
400	`<p className="eyebrow eyebrow-dark">Vista general</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
401	`<h1>Panel de ingresos</h1>`	Abre el elemento <h1>; sus atributos configuran contenido, accesibilidad o eventos.
402	`<p className="dashboard-intro">Consulta la totalidad de ingresos del periodo que elijas.</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
403	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
404	`<div className="dashboard-download-action">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
405	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
406	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.

N.º	Código original	Qué hace esta línea
407	`className="dashboard-download"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
408	`onClick={descargarReporte}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
409	`disabled={!rangoValido || cargando || Boolean(error)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
410	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
411	`<span>Descargar</span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
412	`<span className="download-icon" aria-hidden="true">↓</span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
413	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
414	`{errorDescarga && <p className="download-feedback download-feedback-error" role="alert">{errorDescarga}</p>}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
415	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
416	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
417	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
418	`<section className="dashboard-filters" aria-label="Filtros del panel">`	Abre el elemento <section>; sus atributos configuran contenido, accesibilidad o eventos.
419	`<div className="filter-periods" role="group" aria-label="Periodo de consulta">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
420	`{[`	Abre un bloque de código, objeto o expresión JSX.
421	`['hoy', 'Hoy'],`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
422	`['personalizado', 'Personalizado'],`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
423	`].map(([valor, texto]) => (`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
424	`<button`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
425	`type="button"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
426	`key={valor}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
427	`className={`filter-period${periodo === valor ? ' selected' : ''}`}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
428	`aria-pressed={periodo === valor}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
429	`onClick={() => cambiarPeriodo(valor)}`	Conecta el evento onClick del elemento visual con la función que responde a la interacción.
430	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
431	`{texto}`	Abre un bloque de código, objeto o expresión JSX.
432	`</button>`	Cierra el elemento <button> y delimita el contenido que pertenece a él.
433	`))}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
434	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
435	`{periodo === 'personalizado' && (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
436	`<div className="filter-dates">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.

N.º	Código original	Qué hace esta línea
437	`<label>`	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.
438	`Desde`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
439	`<input`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
440	`type="date"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
441	`value={desde}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
442	`max={hasta || undefined}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
443	`onChange={(evento) => setDesde(evento.target.value)}`	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
444	`/>`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
445	`</label>`	Cierra el elemento <label> y delimita el contenido que pertenece a él.
446	`<label>`	Abre el elemento <label>; sus atributos configuran contenido, accesibilidad o eventos.
447	`Hasta`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
448	`<input`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
449	`type="date"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
450	`value={hasta}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
451	`min={desde || undefined}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
452	`onChange={(evento) => setHasta(evento.target.value)}`	Conecta el evento onChange del elemento visual con la función que responde a la interacción.
453	`/>`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
454	`</label>`	Cierra el elemento <label> y delimita el contenido que pertenece a él.
455	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
456	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
457	`</section>`	Cierra el elemento <section> y delimita el contenido que pertenece a él.
458	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
459	`<section className="dashboard-total" aria-live="polite">`	Abre el elemento <section>; sus atributos configuran contenido, accesibilidad o eventos.
460	`<div>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
461	`<p className="dashboard-total-label">Total de ingresos · {etiquetaPeriodo}</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
462	`{cargando ? (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
463	`<p className="dashboard-total-value">Cargando...</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
464	`) : error ? (`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
465	`<p className="dashboard-error" role="alert">{error}</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
466	`) : !rangoValido ? (`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
467	`<p className="dashboard-error" role="alert">La fecha inicial debe ser anterior a la fecha final.</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.

N.º	Código original	Qué hace esta línea
468	`) : (`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
469	`<>`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
470	`<p className="dashboard-total-value">{formatoMoneda.format(resumen.total)}</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
471	`<p className="dashboard-total-caption">`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
472	`{resumen.cantidad} {resumen.cantidad === 1 ? 'ingreso registrado' : 'ingresos registrados'}`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
473	`{' · '}{formatoFecha.format(fechaUtc(desde))} – {formatoFecha.format(fechaUtc(hasta))}`	Abre un bloque de código, objeto o expresión JSX.
474	`</p>`	Cierra el elemento <p> y delimita el contenido que pertenece a él.
475	`</>`	Cierra un elemento de la plantilla JSX.
476	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
477	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
478	`<div className="total-decoration" aria-hidden="true">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
479	`<span>Σ</span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
480	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
481	`</section>`	Cierra el elemento <section> y delimita el contenido que pertenece a él.
482	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
483	`<div className="dashboard-charts">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
484	`<section className="dashboard-chart-card">`	Abre el elemento <section>; sus atributos configuran contenido, accesibilidad o eventos.
485	`<div className="chart-card-heading">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
486	`<div>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
487	`<h2>Distribución por método</h2>`	Abre el elemento <h2>; sus atributos configuran contenido, accesibilidad o eventos.
488	`<p>Composición de los ingresos del periodo</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
489	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
490	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
491	`{cargando || error || !rangoValido ? null : totalMetodos === 0 ? (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
492	`<p className="chart-empty">No hay ingresos registrados en este periodo.</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
493	`) : (`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
494	`<div className="payment-chart">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
495	`<div`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
496	`className="payment-donut"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
497	`role="img"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.

N.º	Código original	Qué hace esta línea
498	`aria-label={`Distribución de ${formatoMoneda.format(totalMetodos)} por método de pago`}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
499	`style={{`	Abre el bloque de código que contiene las instrucciones siguientes.
500	`background: `conic-gradient(${metodosOrdenados`	Define el valor de la propiedad «background» del objeto actual.
501	`.map(([metodo, valor], indice) => {`	Completa una condición, encadenamiento, separador o expresión anterior.
502	`const anterior = metodosOrdenados`	Declara anterior y le asigna el valor o expresión mostrado para reutilizarlo.
503	`.slice(0, indice)`	Completa una condición, encadenamiento, separador o expresión anterior.
504	`.reduce((suma, [, monto]) => suma + monto, 0)`	Completa una condición, encadenamiento, separador o expresión anterior.
505	`const inicio = (anterior / totalMetodos) * 100`	Declara inicio y le asigna el valor o expresión mostrado para reutilizarlo.
506	`const fin = ((anterior + valor) / totalMetodos) * 100`	Declara fin y le asigna el valor o expresión mostrado para reutilizarlo.
507	`return `${colorMetodo(metodo)} ${inicio}% ${fin}%``	Devuelve un valor de la función o el elemento React que se renderizará.
508	`})`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
509	`.join(', ')})`,`	Completa una condición, encadenamiento, separador o expresión anterior.
510	`}}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
511	`>`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
512	`<div className="payment-donut-center">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
513	`<span>{metodosOrdenados.length}</span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
514	`<small>métodos</small>`	Abre el elemento <small>; sus atributos configuran contenido, accesibilidad o eventos.
515	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
516	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
517	`<ul className="payment-legend">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
518	`{metodosOrdenados.map(([metodo, valor]) => (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
519	`<li key={metodo}>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
520	`<span`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
521	`className="legend-dot"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
522	`style={{ backgroundColor: colorMetodo(metodo) }}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
523	`/>`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
524	`<span className="legend-name">{metodosPago[metodo] || metodo}</span>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
525	`<strong>{formatoMoneda.format(valor)}</strong>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
526	`</li>`	Cierra el elemento y delimita el contenido que pertenece a él.
527	`))}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.

N.º	Código original	Qué hace esta línea
528	`</ul>`	Cierra el elemento y delimita el contenido que pertenece a él.
529	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
530	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
531	`</section>`	Cierra el elemento <section> y delimita el contenido que pertenece a él.
532	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
533	`<section className="dashboard-chart-card">`	Abre el elemento <section>; sus atributos configuran contenido, accesibilidad o eventos.
534	`<div className="chart-card-heading">`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
535	`<div>`	Abre el elemento ; sus atributos configuran contenido, accesibilidad o eventos.
536	`<h2>Evolución de ingresos</h2>`	Abre el elemento <h2>; sus atributos configuran contenido, accesibilidad o eventos.
537	`<p>Ingresos registrados a lo largo del periodo</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
538	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
539	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
540	`{cargando || error || !rangoValido ? null : tendencia.length === 0 || metodosOrdenados.length === 0 ? (`	Inserta una expresión JavaScript en JSX para mostrar contenido condicional o generar elementos.
541	`<p className="chart-empty">No hay ingresos registrados en este periodo.</p>`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
542	`) : (`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
543	`graficoLinea(tendencia, metodosOrdenados.map(([metodo]) => metodo))`	Invoca graficoLinea para ejecutar la operación definida por esa función o biblioteca.
544	`)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
545	`</section>`	Cierra el elemento <section> y delimita el contenido que pertenece a él.
546	`</div>`	Cierra el elemento y delimita el contenido que pertenece a él.
547	`<p className="dashboard-footnote">`	Abre el elemento <p>; sus atributos configuran contenido, accesibilidad o eventos.
548	`El total se calcula con los ingresos registrados según su fecha de pago.`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
549	`</p>`	Cierra el elemento <p> y delimita el contenido que pertenece a él.
550	`</section>`	Cierra el elemento <section> y delimita el contenido que pertenece a él.
551	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
552	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
553	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
554	`export default Dashboard`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.

Financiero/src/pages/FuncionalidadesPage.jsx

Página que presenta la interfaz de funcionalidades financieras.

7 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import Funcionalidades from '../components/Funcionalidades.jsx'`	Importa Funcionalidades desde ../components/Funcionalidades.jsx y conecta este archivo con ese recurso.
2	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
3	`function FuncionalidadesPage() {`	Declara la función FuncionalidadesPage; el bloque siguiente contiene las instrucciones que ejecuta.
4	`  return <Funcionalidades />`	Devuelve un valor de la función o el elemento React que se renderizará.
5	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
6	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
7	`export default FuncionalidadesPage`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.

Financiero/src/pages/Inicio.jsx

Compone la página de inicio a partir de secciones visuales reutilizables.

19 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { useState } from 'react'`	Importa { useState } desde react y conecta este archivo con ese recurso.
2	`import Hero from '../components/Hero.jsx'`	Importa Hero desde ../components/Hero.jsx y conecta este archivo con ese recurso.
3	`import Bienvenida from '../components/Bienvenida.jsx'`	Importa Bienvenida desde ../components/Bienvenida.jsx y conecta este archivo con ese recurso.
4	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
5	`function Inicio() {`	Declara la función Inicio; el bloque siguiente contiene las instrucciones que ejecuta.
6	`  const [mostrarFormularioUsuario, setMostrarFormularioUsuario] = useState(false)`	Crea estado React y su función actualizadora; los cambios vuelven a renderizar el componente.
7	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
8	`  return (`	Devuelve un valor de la función o el elemento React que se renderizará.
9	`    <>`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
10	`      <Hero onUsuarioNuevo={() => setMostrarFormularioUsuario(true)} />`	Inserta el componente React Hero y le pasa sus props.
11	`      <Bienvenida`	Añade texto o una parte de la plantilla JSX que React convierte en interfaz.
12	`        visible={mostrarFormularioUsuario}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
13	`        onCerrar={() => setMostrarFormularioUsuario(false)}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
14	`      />`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
15	`    </>`	Cierra un elemento de la plantilla JSX.
16	`  )`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
17	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
18	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
19	`export default Inicio`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.

Financiero/src/pages/Proyecto.jsx

Compone la página informativa del proyecto con sus componentes.

7 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import Problema from '../components/Problema.jsx'`	Importa Problema desde ../components/Problema.jsx y conecta este archivo con ese recurso.
2	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
3	`function Proyecto() {`	Declara la función Proyecto; el bloque siguiente contiene las instrucciones que ejecuta.
4	`  return <Problema />`	Devuelve un valor de la función o el elemento React que se renderizará.
5	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
6	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
7	`export default Proyecto`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.

Financiero/src/services/api.js

Crea el cliente Axios compartido y adjunta el token almacenado a las solicitudes protegidas.

13 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import axios from 'axios'`	Importa axios desde axios y conecta este archivo con ese recurso.
2	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
3	`const apiClient = axios.create()`	Prepara o ejecuta una llamada CREATE con Axios hacia el backend.
4	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
5	`apiClient.interceptors.request.use((config) => {`	Prepara o ejecuta una llamada INTERCEPTORS con Axios hacia el backend.
6	`  const token = sessionStorage.getItem('authToken')`	Lee, guarda o elimina un dato de sesión en el almacenamiento temporal del navegador.
7	`  if (token) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
8	`    config.headers.Authorization = `Bearer ${token}``	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
9	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
10	`  return config`	Devuelve un valor de la función o el elemento React que se renderizará.
11	`})`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
12	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
13	`export default apiClient`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.

Financiero/src/services/incomeEvents.js

Notifica a otras vistas del navegador que cambiaron los datos de ingresos.

5 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`export const INCOME_DATA_CHANGED = 'income-data-changed'`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
2	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
3	`export function notifyIncomeDataChanged() {`	Exporta la función notifyIncomeDataChanged para que rutas u otros módulos puedan invocarla.
4	`  window.dispatchEvent(new Event(INCOME_DATA_CHANGED))`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
5	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.

Financiero/vite.config.js

Configura Vite y su complemento de React para desarrollo y compilación.

6 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { defineConfig } from 'vite'`	Importa { defineConfig } desde vite y conecta este archivo con ese recurso.
2	`import react from '@vitejs/plugin-react'`	Importa react desde @vitejs/plugin-react y conecta este archivo con ese recurso.
3	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
4	`export default defineConfig({`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.
5	`  plugins: [react()],`	Define el valor de la propiedad «plugins» del objeto actual.
6	`})`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.

package.json

Manifiesto npm: declara identidad, comandos disponibles y dependencias del módulo.

5 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`{`	Abre un objeto JSON que agrupa valores de configuración.
2	`  "dependencies": {`	Configura «dependencies»: declara bibliotecas necesarias en ejecución. Valor: {.
3	`    "react-router-dom": "^6.30.6"`	Configura «react-router-dom»: define esta opción del manifiesto. Valor: "^6.30.6".
4	`  }`	Cierra el objeto o sección de configuración JSON.
5	`}`	Cierra el objeto o sección de configuración JSON.

server/migrations/20261029_align_ingresos_with_app.sql

Alinea el esquema PostgreSQL de ingresos con usuario_id y las restricciones que usa la aplicación.

33 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`BEGIN;`	Inicia una transacción para agrupar cambios que podrán confirmarse o revertirse.
2	`(línea vacía)`	Línea en blanco que separa sentencias o cláusulas SQL.
3	`DO $$`	Inicia un bloque procedural anónimo que PostgreSQL ejecuta.
4	`BEGIN`	Inicia una transacción para agrupar cambios que podrán confirmarse o revertirse.
5	`  IF EXISTS (SELECT 1 FROM public.ingresos) THEN`	Inicia una condición SQL.
6	`    RAISE EXCEPTION`	Emite un aviso o error desde el bloque procedural.
7	`      'La tabla ingresos ya tiene registros. Migra esos datos manualmente antes de aplicar esta migración.';`	Completa una sentencia o cláusula SQL de la migración.
8	`  END IF;`	Cierra el bloque condicional o procedural.
9	`END $$;`	Cierra el bloque condicional o procedural.
10	`(línea vacía)`	Línea en blanco que separa sentencias o cláusulas SQL.
11	`DO $$`	Inicia un bloque procedural anónimo que PostgreSQL ejecuta.
12	`BEGIN`	Inicia una transacción para agrupar cambios que podrán confirmarse o revertirse.
13	`  IF NOT EXISTS (`	Inicia una condición SQL.
14	`    SELECT 1`	Consulta datos o calcula un valor en PostgreSQL.
15	`    FROM information_schema.columns`	Indica la tabla o subconsulta de donde se obtienen los datos.
16	`    WHERE table_schema = 'public'`	Limita la operación a las filas que cumplan esta condición.
17	`      AND table_name = 'ingresos'`	Completa una sentencia o cláusula SQL de la migración.
18	`      AND column_name = 'id'`	Completa una sentencia o cláusula SQL de la migración.
19	`      AND is_identity = 'YES'`	Completa una sentencia o cláusula SQL de la migración.
20	`  ) THEN`	Completa una sentencia o cláusula SQL de la migración.
21	`    ALTER TABLE public.ingresos`	Modifica la definición de una tabla PostgreSQL existente.
22	`      ALTER COLUMN id ADD GENERATED BY DEFAULT AS IDENTITY;`	Completa una sentencia o cláusula SQL de la migración.
23	`  END IF;`	Cierra el bloque condicional o procedural.
24	`END $$;`	Cierra el bloque condicional o procedural.
25	`(línea vacía)`	Línea en blanco que separa sentencias o cláusulas SQL.
26	`ALTER TABLE public.ingresos`	Modifica la definición de una tabla PostgreSQL existente.
27	`  ADD COLUMN usuario_id INTEGER NOT NULL`	Agrega una columna a la tabla actual.
28	`  REFERENCES public.users(id);`	Vincula una columna con otra tabla mediante una relación foránea.
29	`(línea vacía)`	Línea en blanco que separa sentencias o cláusulas SQL.
30	`ALTER TABLE public.ingresos`	Modifica la definición de una tabla PostgreSQL existente.
31	`  DROP COLUMN cliente;`	Elimina una columna de la tabla actual.
32	`(línea vacía)`	Línea en blanco que separa sentencias o cláusulas SQL.
33	`COMMIT;`	Confirma la transacción y hace permanentes los cambios.

server/package.json

Manifiesto npm: declara identidad, comandos disponibles y dependencias del módulo.

19 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`{`	Abre un objeto JSON que agrupa valores de configuración.
2	`  "name": "server",`	Configura «name»: identifica el paquete. Valor: "server".
3	`  "version": "1.0.0",`	Configura «version»: indica su versión. Valor: "1.0.0".
4	`  "type": "module",`	Configura «type»: indica el sistema de módulos de Node.js. Valor: "module".
5	`  "main": "src/index.js",`	Configura «main»: señala el archivo principal. Valor: " src/index.js".
6	`  "scripts": {`	Configura «scripts»: agrupa comandos ejecutables. Valor: {.
7	`    "dev": "nodemon src/index.js",`	Configura «dev»: define el comando de desarrollo. Valor: " nodemon src/index.js".
8	`    "start": "node src/index.js"`	Configura «start»: define el comando de inicio. Valor: "node src/index.js".
9	`  },`	Cierra el objeto o sección de configuración JSON.
10	`  "dependencies": {`	Configura «dependencies»: declara bibliotecas necesarias en ejecución. Valor: {.
11	`    "cors": "^2.8.5",`	Configura «cors»: define esta opción del manifiesto. Valor: " ^2.8.5".
12	`    "dotenv": "^16.4.5",`	Configura «dotenv»: define esta opción del manifiesto. Valor: " ^16.4.5".
13	`    "express": "^4.19.2",`	Configura «express»: define esta opción del manifiesto. Valor: " ^4.19.2".
14	`    "pg": "^8.23.1"`	Configura «pg»: define esta opción del manifiesto. Valor: " ^8.23.1".
15	`  },`	Cierra el objeto o sección de configuración JSON.
16	`  "devDependencies": {`	Configura «devDependencies»: declara herramientas de desarrollo o compilación. Valor: {.
17	`    "nodemon": "^3.1.0"`	Configura «nodemon»: define esta opción del manifiesto. Valor: " ^3.1.0".
18	`  }`	Cierra el objeto o sección de configuración JSON.
19	`}`	Cierra el objeto o sección de configuración JSON.

server/src/authSessions.js

Administra tokens de sesión en memoria, vencimientos e invalidación al cerrar sesión.

29 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { randomBytes } from 'node:crypto';`	Importa { randomBytes } desde node:crypto y conecta este archivo con ese recurso.
2	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
3	`const sesiones = new Map();`	Declara sesiones y le asigna el valor o expresión mostrado para reutilizarlo.
4	`const duracionSesion = 8 * 60 * 60 * 1000;`	Declara duracionSesion y le asigna el valor o expresión mostrado para reutilizarlo.
5	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
6	`export function crearSesion(username) {`	Exporta la función crearSesion para que rutas u otros módulos puedan invocarla.
7	`  const token = randomBytes(32).toString('hex');`	Declara token y le asigna el valor o expresión mostrado para reutilizarlo.
8	`  sesiones.set(token, {`	Abre el bloque de código que contiene las instrucciones siguientes.
9	`    username,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
10	`    expiraEn: Date.now() + duracionSesion,`	Define el valor de la propiedad «expiraEn» del objeto actual.
11	`  });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
12	`  return token;`	Devuelve un valor de la función o el elemento React que se renderizará.
13	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
14	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
15	`export function obtenerSesion(token) {`	Exporta la función obtenerSesion para que rutas u otros módulos puedan invocarla.
16	`  const sesion = sesiones.get(token);`	Declara sesion y le asigna el valor o expresión mostrado para reutilizarlo.
17	`  if (!sesion) return null;`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
18	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
19	`  if (sesion.expiraEn <= Date.now()) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
20	`    sesiones.delete(token);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
21	`    return null;`	Devuelve un valor de la función o el elemento React que se renderizará.
22	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
23	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
24	`  return sesion;`	Devuelve un valor de la función o el elemento React que se renderizará.
25	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
26	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
27	`export function eliminarSesion(token) {`	Exporta la función eliminarSesion para que rutas u otros módulos puedan invocarla.
28	`  sesiones.delete(token);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
29	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.

server/src/config.js

Carga variables de entorno antes de acceder a la configuración del servidor.

4 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import dotenv from 'dotenv';`	Importa dotenv desde dotenv y conecta este archivo con ese recurso.
2	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
3	`dotenv.config({ path: '.env.local' });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
4	`dotenv.config();`	Completa una instrucción JavaScript/JSX dentro del flujo actual.

server/src/controllers/incomesController.js

Implementa operaciones CRUD de ingresos y guarda lotes en una transacción SQL.

184 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { pool } from '../db.js';`	Importa { pool } desde ../db.js y conecta este archivo con ese recurso.
2	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
3	`const columnasIngreso = `	Declara columnasIngreso y le asigna el valor o expresión mostrado para reutilizarlo.
4	`  i.id,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
5	`  i.usuario_id AS "usuarioId",`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
6	`  CONCAT_WS(' ', u.name, u."lastName") AS cliente,`	Invoca CONCAT_WS para ejecutar la operación definida por esa función o biblioteca.
7	`  u.email AS "clienteEmail",`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
8	`  i.cuenta_bancaria AS "cuentaBancaria",`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
9	`  i.metodo_pago AS "metodoPago",`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
10	`  TO_CHAR(i.fecha_pago, 'YYYY-MM-DD') AS "fechaPago",`	Invoca TO_CHAR para ejecutar la operación definida por esa función o biblioteca.
11	`  i.valor,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
12	`  i.observacion,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
13	`  i.creado_en AS "creadoEn"`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
14	`` `; ``	Completa una instrucción JavaScript/JSX dentro del flujo actual.
15	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
16	`export async function obtenerIngresos(req, res) {`	Exporta la función obtenerIngresos para que rutas u otros módulos puedan invocarla.
17	`  try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
18	`    const { rows } = await pool.query(`	Espera a que PostgreSQL termine la consulta antes de continuar.
19	`      `SELECT ${columnasIngreso}`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
20	`       FROM ingresos i`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
21	`       JOIN users u ON u.id = i.usuario_id`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
22	`       ORDER BY i.fecha_pago DESC, i.id DESC`,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
23	`    );`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
24	`    return res.status(200).json(rows);`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
25	`  } catch (error) {`	Abre el bloque de código que contiene las instrucciones siguientes.
26	`    console.error('Error al obtener los ingresos de PostgreSQL:', error);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
27	`    return res.status(500).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
28	`      message: 'No fue posible consultar los ingresos en la base de datos.',`	Define el valor de la propiedad «message» del objeto actual.
29	`    });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.

N.º	Código original	Qué hace esta línea
30	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
31	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
32	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
33	`export async function crearIngreso(req, res) {`	Exporta la función crearIngreso para que rutas u otros módulos puedan invocarla.
34	`  if (Array.isArray(req.body)) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
35	`    const client = await pool.connect();`	Espera la resolución de una operación asíncrona.
36	`    let transaccionIniciada = false;`	Declara transaccionIniciada y le asigna el valor o expresión mostrado para reutilizarlo.
37	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
38	`    try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
39	`      await client.query('BEGIN');`	Espera a que PostgreSQL termine la consulta antes de continuar.
40	`      transaccionIniciada = true;`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
41	`      const ingresos = [];`	Inicia en ingresos una estructura de datos que agrupa valores relacionados.
42	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
43	`      for (const datos of req.body) {`	Inicia un ciclo para procesar elementos o repetir instrucciones.
44	`        const { rows } = await client.query(`	Espera a que PostgreSQL termine la consulta antes de continuar.
45	`          `WITH inserted AS (`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
46	`            INSERT INTO ingresos`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
47	`              (usuario_id, cuenta_bancaria, metodo_pago, fecha_pago, valor, observacion)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
48	`            VALUES ($1, $2, $3, $4, $5, $6)`	Invoca VALUES para ejecutar la operación definida por esa función o biblioteca.
49	`            RETURNING *`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
50	`          )`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
51	`          SELECT ${columnasIngreso}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
52	`          FROM inserted i`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
53	`          JOIN users u ON u.id = i.usuario_id`,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
54	`        [`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
55	`          datos.usuarioId,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
56	`          datos.cuentaBancaria,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
57	`          datos.metodoPago,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
58	`          datos.fechaPago,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
59	`          datos.valor,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
60	`          datos.observacion || null,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
61	`        ],`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
62	`      );`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.

N.º	Código original	Qué hace esta línea
63	`ingresos.push(rows[0]);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
64	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
65	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
66	`await client.query('COMMIT');`	Espera a que PostgreSQL termine la consulta antes de continuar.
67	`return res.status(201).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
68	`message: \`${ingresos.length} ingresos guardados correctamente.\`,`	Define el valor de la propiedad «message» del objeto actual.
69	`ingresos,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
70	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
71	`} catch (error) {`	Abre el bloque de código que contiene las instrucciones siguientes.
72	`console.error('Error al guardar el lote de ingresos en PostgreSQL:', error);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
73	`if (transaccionIniciada) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
74	`try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
75	`await client.query('ROLLBACK');`	Espera a que PostgreSQL termine la consulta antes de continuar.
76	`} catch (rollbackError) {`	Abre el bloque de código que contiene las instrucciones siguientes.
77	`console.error('Error al revertir el lote de ingresos en PostgreSQL:', rollbackError);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
78	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
79	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
80	`if (error.code === '23503') {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
81	`return res.status(400).json({ message: 'Uno de los usuarios seleccionados ya no existe.' });`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
82	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
83	`return res.status(500).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
84	`message: 'No fue posible guardar los ingresos en la base de datos.',`	Define el valor de la propiedad «message» del objeto actual.
85	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
86	`} finally {`	Abre el bloque de código que contiene las instrucciones siguientes.
87	`client.release();`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
88	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
89	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
90	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
91	`const { usuarioId, cuentaBancaria, metodoPago, fechaPago, valor, observacion } = req.body;`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
92	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
93	`try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
94	`const { rows } = await pool.query(`	Espera a que PostgreSQL termine la consulta antes de continuar.

N.º	Código original	Qué hace esta línea
95	`WITH inserted AS (`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
96	INSERT INTO ingresos	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
97	(usuario_id, cuenta_bancaria, metodo_pago, fecha_pago, valor, observacion)	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
98	VALUES ($1, $2, $3, $4, $5, $6)	Invoca VALUES para ejecutar la operación definida por esa función o biblioteca.
99	RETURNING *	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
100	)	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
101	SELECT ${columnasIngreso}	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
102	FROM inserted i	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
103	JOIN users u ON u.id = i.usuario_id`,	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
104	[usuarioId, cuentaBancaria, metodoPago, fechaPago, valor, observacion || null],	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
105	);	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
106	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
107	return res.status(201).json({	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
108	message: 'Ingreso guardado correctamente.',	Define el valor de la propiedad «message» del objeto actual.
109	ingreso: rows[0],	Define el valor de la propiedad «ingreso» del objeto actual.
110	});	Completa una instrucción JavaScript/JSX dentro del flujo actual.
111	} catch (error) {	Abre el bloque de código que contiene las instrucciones siguientes.
112	console.error('Error al guardar el ingreso en PostgreSQL:', error);	Completa una instrucción JavaScript/JSX dentro del flujo actual.
113	if (error.code === '23503') {	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
114	return res.status(400).json({ message: 'El usuario seleccionado ya no existe.' });	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
115	}	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
116	return res.status(500).json({	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
117	message: 'No fue posible guardar el ingreso en la base de datos.',	Define el valor de la propiedad «message» del objeto actual.
118	});	Completa una instrucción JavaScript/JSX dentro del flujo actual.
119	}	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
120	}	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
121	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
122	export async function actualizarIngreso(req, res) {	Exporta la función actualizarIngreso para que rutas u otros módulos puedan invocarla.
123	const { id } = req.params;	Completa una instrucción JavaScript/JSX dentro del flujo actual.
124	const { usuarioId, cuentaBancaria, metodoPago, fechaPago, valor, observacion } = req.body;	Completa una instrucción JavaScript/JSX dentro del flujo actual.
125	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
126	try {	Inicia un bloque protegido; los errores pueden procesarse en catch.

N.º	Código original	Qué hace esta línea
127	`const { rows } = await pool.query(`	Espera a que PostgreSQL termine la consulta antes de continuar.
128	`` `WITH updated AS ( ``	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
129	`UPDATE ingresos AS i`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
130	`SET usuario_id = $1,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
131	`cuenta_bancaria = $2,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
132	`metodo_pago = $3,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
133	`fecha_pago = $4,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
134	`valor = $5,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
135	`observacion = $6`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
136	`WHERE i.id = $7`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
137	`RETURNING i.*`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
138	`)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
139	`SELECT ${columnasIngreso}`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
140	`FROM updated i`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
141	`JOIN users u ON u.id = i.usuario_id`,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
142	`[usuarioId, cuentaBancaria, metodoPago, fechaPago, valor, observacion || null, id],`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
143	`);`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
144	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
145	`if (rows.length === 0) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
146	`return res.status(404).json({ message: 'Ingreso no encontrado.' });`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
147	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
148	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
149	`return res.status(200).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
150	`message: 'Ingreso actualizado correctamente.',`	Define el valor de la propiedad «message» del objeto actual.
151	`ingreso: rows[0],`	Define el valor de la propiedad «ingreso» del objeto actual.
152	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
153	`} catch (error) {`	Abre el bloque de código que contiene las instrucciones siguientes.
154	`console.error('Error al actualizar el ingreso en PostgreSQL:', error);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
155	`if (error.code === '23503') {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
156	`return res.status(400).json({ message: 'El usuario seleccionado ya no existe.' });`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
157	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
158	`return res.status(500).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.

N.º	Código original	Qué hace esta línea
159	`      message: 'No fue posible actualizar el ingreso en la base de datos.',`	Define el valor de la propiedad «message» del objeto actual.
160	`    });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
161	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
162	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
163	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
164	`export async function eliminarIngreso(req, res) {`	Exporta la función eliminarIngreso para que rutas u otros módulos puedan invocarla.
165	`  const { id } = req.params;`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
166	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
167	`  try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
168	`    const { rows } = await pool.query(`	Espera a que PostgreSQL termine la consulta antes de continuar.
169	`      'DELETE FROM ingresos WHERE id = $1 RETURNING id',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
170	`      [id],`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
171	`    );`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
172	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
173	`    if (rows.length === 0) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
174	`      return res.status(404).json({ message: 'Ingreso no encontrado.' });`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
175	`    }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
176	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
177	`    return res.status(200).json({ message: 'Ingreso eliminado correctamente.' });`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
178	`  } catch (error) {`	Abre el bloque de código que contiene las instrucciones siguientes.
179	`    console.error('Error al eliminar el ingreso de PostgreSQL:', error);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
180	`    return res.status(500).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
181	`      message: 'No fue posible eliminar el ingreso de la base de datos.',`	Define el valor de la propiedad «message» del objeto actual.
182	`    });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
183	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
184	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.

server/src/controllers/usersController.js

Implementa operaciones CRUD de usuarios con consultas parametrizadas a PostgreSQL.

105 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { pool } from '../db.js';`	Importa { pool } desde ../db.js y conecta este archivo con ese recurso.
2	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
3	`export async function obtenerUsuarios(req, res) {`	Exporta la función obtenerUsuarios para que rutas u otros módulos puedan invocarla.
4	`  try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
5	`    const { rows } = await pool.query('SELECT id, name, "lastName", email FROM users');`	Espera a que PostgreSQL termine la consulta antes de continuar.
6	`    return res.status(200).json(rows);`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
7	`  } catch (error) {`	Abre el bloque de código que contiene las instrucciones siguientes.
8	`    console.error('Error al obtener los usuarios de PostgreSQL:', error);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
9	`    return res.status(500).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
10	`      message: 'No fue posible obtener los usuarios de la base de datos.',`	Define el valor de la propiedad «message» del objeto actual.
11	`    });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
12	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
13	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
14	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
15	`export async function crearUsuario(req, res) {`	Exporta la función crearUsuario para que rutas u otros módulos puedan invocarla.
16	`  const { name, lastName, email } = req.body;`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
17	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
18	`  try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
19	`    const { rows } = await pool.query(`	Espera a que PostgreSQL termine la consulta antes de continuar.
20	`      'INSERT INTO users (name, "lastName", email) VALUES ($1, $2, $3) RETURNING id, name, "lastName", email',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
21	`      [name, lastName, email]`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
22	`    );`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
23	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
24	`    return res.status(201).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
25	`      message: 'Usuario guardado correctamente',`	Define el valor de la propiedad «message» del objeto actual.
26	`      user: rows[0],`	Define el valor de la propiedad «user» del objeto actual.
27	`    });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
28	`  } catch (error) {`	Abre el bloque de código que contiene las instrucciones siguientes.
29	`    console.error('Error al guardar el usuario en PostgreSQL:', error);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
30	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
31	`    if (error.code === '22001' || error.code === '23502' || error.code === '23514') {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.

N.º	Código original	Qué hace esta línea
32	`return res.status(400).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
33	`message: 'Los datos no cumplen con las restricciones definidas en la base de datos.',`	Define el valor de la propiedad «message» del objeto actual.
34	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
35	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
36	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
37	`return res.status(500).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
38	`message: 'No fue posible guardar los datos en la base de datos.',`	Define el valor de la propiedad «message» del objeto actual.
39	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
40	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
41	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
42	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
43	`export async function actualizarUsuario(req, res) {`	Exporta la función actualizarUsuario para que rutas u otros módulos puedan invocarla.
44	`const { id } = req.params;`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
45	`const { name, lastName, email } = req.body;`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
46	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
47	`try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
48	`const { rows } = await pool.query(`	Espera a que PostgreSQL termine la consulta antes de continuar.
49	`'UPDATE users SET name = $1, "lastName" = $2, email = $3 WHERE id = $4 RETURNING id, name, "lastName", email',`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
50	`[name, lastName, email, id]`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
51	`);`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
52	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
53	`if (rows.length === 0) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
54	`return res.status(404).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
55	`message: 'Usuario no encontrado.',`	Define el valor de la propiedad «message» del objeto actual.
56	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
57	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
58	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
59	`return res.status(200).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
60	`message: 'Usuario actualizado correctamente.',`	Define el valor de la propiedad «message» del objeto actual.
61	`user: rows[0],`	Define el valor de la propiedad «user» del objeto actual.
62	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
63	`} catch (error) {`	Abre el bloque de código que contiene las instrucciones siguientes.
64	`console.error('Error al actualizar el usuario en PostgreSQL:', error);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
65	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
66	`if (error.code === '22001' || error.code === '23502' || error.code === '23514') {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.

N.º	Código original	Qué hace esta línea
67	`return res.status(400).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
68	`message: 'Los datos no cumplen con las restricciones definidas en la base de datos.',`	Define el valor de la propiedad «message» del objeto actual.
69	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
70	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
71	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
72	`return res.status(500).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
73	`message: 'No fue posible actualizar los datos del usuario en la base de datos.',`	Define el valor de la propiedad «message» del objeto actual.
74	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
75	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
76	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
77	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
78	`export async function eliminarUsuario(req, res) {`	Exporta la función eliminarUsuario para que rutas u otros módulos puedan invocarla.
79	`const { id } = req.params;`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
80	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
81	`try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
82	`const { rows } = await pool.query('DELETE FROM users WHERE id = $1 RETURNING id', [id]);`	Espera a que PostgreSQL termine la consulta antes de continuar.
83	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
84	`if (rows.length === 0) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
85	`return res.status(404).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
86	`message: 'Usuario no encontrado.',`	Define el valor de la propiedad «message» del objeto actual.
87	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
88	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
89	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
90	`return res.status(200).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
91	`message: 'Usuario eliminado correctamente.',`	Define el valor de la propiedad «message» del objeto actual.
92	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
93	`} catch (error) {`	Abre el bloque de código que contiene las instrucciones siguientes.
94	`console.error('Error al eliminar el usuario en PostgreSQL:', error);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
95	`if (error.code === '23503') {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
96	`return res.status(409).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
97	`message: 'No se puede eliminar el usuario porque tiene ingresos asociados.',`	Define el valor de la propiedad «message» del objeto actual.
98	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
99	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
100	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.

N.º	Código original	Qué hace esta línea
101	`            return res.status(500).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
102	`              message: 'No fue posible eliminar el usuario de la base de datos.',`	Define el valor de la propiedad «message» del objeto actual.
103	`            });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
104	`          }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
105	`        }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.

server/src/db.js

Crea el pool compartido de conexiones PostgreSQL a partir de DATABASE_URL.

13 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { Pool } from 'pg';`	Importa { Pool } desde pg y conecta este archivo con ese recurso.
2	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
3	`const connectionString = process.env.DATABASE_URL;`	Declara connectionString y le asigna el valor o expresión mostrado para reutilizarlo.
4	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
5	`if (!connectionString) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
6	`  throw new Error('Debes configurar DATABASE_URL en server/.env');`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
7	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
8	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
9	`export const pool = new Pool({ connectionString });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
10	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
11	`pool.on('error', (error) => {`	Abre el bloque de código que contiene las instrucciones siguientes.
12	`  console.error('Error inesperado en el pool de PostgreSQL:', error);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
13	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.

server/src/index.js

Inicia Express, configura middleware, conecta routers y publica verificaciones del backend y PostgreSQL.

43 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import './config.js';`	Importa './config.js'; desde el módulo indicado y conecta este archivo con ese recurso.
2	`import express from 'express';`	Importa express desde express y conecta este archivo con ese recurso.
3	`import cors from 'cors';`	Importa cors desde cors y conecta este archivo con ese recurso.
4	`import { pool } from './db.js';`	Importa { pool } desde ./db.js y conecta este archivo con ese recurso.
5	`import usersRouter from './routes/users.js';`	Importa usersRouter desde ./routes/users.js y conecta este archivo con ese recurso.
6	`import authRouter from './routes/auth.js';`	Importa authRouter desde ./routes/auth.js y conecta este archivo con ese recurso.
7	`import incomesRouter from './routes/incomes.js';`	Importa incomesRouter desde ./routes/incomes.js y conecta este archivo con ese recurso.
8	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
9	`const app = express();`	Declara app y le asigna el valor o expresión mostrado para reutilizarlo.
10	`const PORT = process.env.PORT || 3000;`	Declara PORT y le asigna el valor o expresión mostrado para reutilizarlo.
11	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
12	`// Middlewares`	Comentario de documentación; no se ejecuta como instrucción.
13	`app.use(cors());`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
14	`app.use(express.json());`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
15	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
16	`app.use('/api/auth', authRouter);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
17	`app.use('/api/users', usersRouter);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
18	`app.use('/api/ingresos', incomesRouter);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
19	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
20	`// Ruta de prueba (Healthcheck)`	Comentario de documentación; no se ejecuta como instrucción.
21	`app.get('/api/health', (req, res) => {`	Abre el bloque de código que contiene las instrucciones siguientes.
22	`  res.json({ message: 'Conexión exitosa con el backend en Express' });`	Construye o envía la respuesta HTTP del endpoint.
23	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
24	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
25	`app.get('/api/health/db', async (req, res) => {`	Abre el bloque de código que contiene las instrucciones siguientes.
26	`  try {`	Inicia un bloque protegido; los errores pueden procesarse en catch.
27	`    const { rows } = await pool.query(`	Espera a que PostgreSQL termine la consulta antes de continuar.
28	`      'SELECT current_database() AS database, inet_server_port() AS port'`	Proporciona texto literal utilizado como etiqueta, mensaje o valor de la expresión actual.
29	`    );`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
30	`    res.json({`	Construye o envía la respuesta HTTP del endpoint.
31	`      message: 'Conexión exitosa con PostgreSQL',`	Define el valor de la propiedad «message» del objeto actual.
32	`      database: rows[0].database,`	Define el valor de la propiedad «database» del objeto actual.
33	`      port: rows[0].port,`	Define el valor de la propiedad «port» del objeto actual.
34	`    });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.

N.º	Código original	Qué hace esta línea
35	`  } catch (error) {`	Abre el bloque de código que contiene las instrucciones siguientes.
36	`    console.error('Error al conectar con PostgreSQL:', error);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
37	`    res.status(503).json({ message: 'No fue posible conectar con PostgreSQL' });`	Construye o envía la respuesta HTTP del endpoint.
38	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
39	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
40	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
41	`app.listen(PORT, () => {`	Abre el bloque de código que contiene las instrucciones siguientes.
42	`  console.log(`Servidor Express corriendo en http://localhost:${PORT}`);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
43	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.

server/src/middleware/requireAuth.js

Verifica el token Bearer de cada solicitud protegida y bloquea sesiones inválidas.

16 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { obtenerSesion } from '../authSessions.js';`	Importa { obtenerSesion } desde ../authSessions.js y conecta este archivo con ese recurso.
2	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
3	`export function requireAuth(req, res, next) {`	Exporta la función requireAuth para que rutas u otros módulos puedan invocarla.
4	`  const authorization = req.get('authorization') ?? '';`	Declara authorization y le asigna el valor o expresión mostrado para reutilizarlo.
5	`  const coincidencia = authorization.match(/^Bearer ([\da-f]+)$/i);`	Declara coincidencia y le asigna el valor o expresión mostrado para reutilizarlo.
6	`  const token = coincidencia?.[1];`	Declara token y le asigna el valor o expresión mostrado para reutilizarlo.
7	`  const sesion = token ? obtenerSesion(token) : null;`	Declara sesion y le asigna el valor o expresión mostrado para reutilizarlo.
8	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
9	`  if (!sesion) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
10	`    return res.status(401).json({ message: 'Debes iniciar sesión para continuar.' });`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
11	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
12	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
13	`  req.authToken = token;`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
14	`  req.user = { username: sesion.username };`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
15	`  return next();`	Devuelve un valor de la función o el elemento React que se renderizará.
16	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.

server/src/middleware/validateIncome.js

Valida campos e identificadores de ingresos, también cuando se envía un lote.

99 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`const metodosPago = new Set(['transferencia', ' efectivo', 'tarjeta']);`	Declara metodosPago y le asigna el valor o expresión mostrado para reutilizarlo.
2	`const fechaIso = /^\d{4}-\d{2}-\d{2}$/;`	Declara fechaIso y le asigna el valor o expresión mostrado para reutilizarlo.
3	`const valorMonetario = /^\d{1,12}(?:\.\d{1,2})?$/;`	Declara valorMonetario y le asigna el valor o expresión mostrado para reutilizarlo.
4	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
5	`export function validarIdIngreso(req, res, next) {`	Exporta la función validarIdIngreso para que rutas u otros módulos puedan invocarla.
6	`  if (!/^[1-9]\d*$/.test(req.params.id)) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
7	`    return res.status(400).json({ message: 'El identificador del ingreso no es válido.' });`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
8	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
9	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
10	`  return next();`	Devuelve un valor de la función o el elemento React que se renderizará.
11	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
12	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
13	`function validarDatosIngreso(datos) {`	Declara la función validarDatosIngreso; el bloque siguiente contiene las instrucciones que ejecuta.
14	`  const usuarioId =`	Declara usuarioId y le asigna el valor o expresión mostrado para reutilizarlo.
15	`    typeof datos.usuarioId === 'string' && /^[1-9]\d*$/.test(datos.usuarioId)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
16	`      ? datos.usuarioId`	Completa una condición, encadenamiento, separador o expresión anterior.
17	`      : '';`	Completa una condición, encadenamiento, separador o expresión anterior.
18	`  let cuentaBancaria =`	Declara cuentaBancaria y le asigna el valor o expresión mostrado para reutilizarlo.
19	`    typeof datos.cuentaBancaria === 'string' ? datos.cuentaBancaria.trim()  : '';`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
20	`  const metodoPago = typeof datos.metodoPago === 'string' ? datos.metodoPago : '';`	Declara metodoPago y le asigna el valor o expresión mostrado para reutilizarlo.
21	`  const fechaPago = typeof datos.fechaPago === 'string' ? datos.fechaPago : '';`	Declara fechaPago y le asigna el valor o expresión mostrado para reutilizarlo.
22	`  const valor = typeof datos.valor === 'string' ? datos.valor.trim() : '';`	Declara valor y le asigna el valor o expresión mostrado para reutilizarlo.
23	`  const observacion =`	Declara observacion y le asigna el valor o expresión mostrado para reutilizarlo.
24	`    typeof datos.observacion === 'string' ? datos.observacion.trim() : '';`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
25	`  const fechaValida =`	Declara fechaValida y le asigna el valor o expresión mostrado para reutilizarlo.
26	`    fechaIso.test(fechaPago) &&`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
27	`    !Number.isNaN(Date.parse(`${fechaPago}T00:00:00Z`)) &&`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
28	`    new Date(`${fechaPago}T00:00:00Z`).toISOString().slice(0, 10) === fechaPago;`	Completa una instrucción JavaScript/JSX dentro del flujo actual.

N.º	Código original	Qué hace esta línea
29	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
30	`  if (!usuarioId) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
31	`    return { error: 'Selecciona un usuario válido como cliente.' };`	Devuelve un valor de la función o el elemento React que se renderizará.
32	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
33	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
34	`  if (!metodosPago.has(metodoPago)) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
35	`    return { error: 'Selecciona un método de pago válido.' };`	Devuelve un valor de la función o el elemento React que se renderizará.
36	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
37	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
38	`  if (metodoPago === 'transferencia') {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
39	`    if (!/^[0-9]{15}$/.test(cuentaBancaria)) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
40	`      return { error: 'La cuenta bancaria debe contener exactamente 15 dígitos numéricos.' };`	Devuelve un valor de la función o el elemento React que se renderizará.
41	`    }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
42	`  } else {`	Abre el bloque de código que contiene las instrucciones siguientes.
43	`    cuentaBancaria = null;`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
44	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
45	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
46	`  if (!fechaValida) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
47	`    return { error: 'Ingresa una fecha de pago válida.' };`	Devuelve un valor de la función o el elemento React que se renderizará.
48	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
49	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
50	`  if (!valorMonetario.test(valor) || Number(valor) <= 0) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
51	`    return { error: 'El valor debe ser mayor que cero y tener máximo dos decimales.' };`	Devuelve un valor de la función o el elemento React que se renderizará.
52	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
53	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
54	`  if (observacion.length > 10000) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
55	`    return { error: 'La observación no puede superar 10000 caracteres.' };`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
56	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
57	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
58	`  return {`	Devuelve un valor de la función o el elemento React que se renderizará.
59	`    datos: { usuarioId, cuentaBancaria, metodoPago, fechaPago, valor, observacion },`	Define el valor de la propiedad «datos» del objeto actual.
60	`  };`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.

N.º	Código original	Qué hace esta línea
61	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
62	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
63	`export function validarIngreso(req, res, next) {`	Exporta la función validarIngreso para que rutas u otros módulos puedan invocarla.
64	`  if (Array.isArray(req.body)) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
65	`    if (req.body.length === 0 || req.body.length > 100) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
66	`      return res.status(400).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
67	`        message: 'Envía entre 1 y 100 ingresos por solicitud.',`	Define el valor de la propiedad «message» del objeto actual.
68	`      });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
69	`    }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
70	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
71	`    const ingresos = [];`	Inicia en ingresos una estructura de datos que agrupa valores relacionados.
72	`    for (const [indice, datos] of req.body.entries()) {`	Inicia un ciclo para procesar elementos o repetir instrucciones.
73	`      if (!datos || typeof datos !== 'object' || Array.isArray(datos)) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
74	`        return res.status(400).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
75	`          message: `El ingreso en la posición ${indice + 1} debe ser un objeto JSON.`,`	Define el valor de la propiedad «message» del objeto actual.
76	`        });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
77	`      }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
78	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
79	`      const resultado = validarDatosIngreso(datos);`	Declara resultado y le asigna el valor o expresión mostrado para reutilizarlo.
80	`      if (resultado.error) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
81	`        return res.status(400).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
82	`          message: `Ingreso ${indice + 1}: ${resultado.error}`,`	Define el valor de la propiedad «message» del objeto actual.
83	`        });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
84	`      }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
85	`      ingresos.push(resultado.datos);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
86	`    }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
87	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
88	`    req.body = ingresos;`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
89	`    return next();`	Devuelve un valor de la función o el elemento React que se renderizará.
90	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
91	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
92	`  const resultado = validarDatosIngreso(req.body ?? {});`	Declara resultado y le asigna el valor o expresión mostrado para reutilizarlo.
93	`  if (resultado.error) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
94	`    return res.status(400).json({ message: resultado.error });`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.

N.º	Código original	Qué hace esta línea
95	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
96	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
97	`req.body = resultado.datos;`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
98	`return next();`	Devuelve un valor de la función o el elemento React que se renderizará.
99	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.

server/src/middleware/validateUser.js

Valida y normaliza nombre, apellido y correo antes de guardar usuarios.

47 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`export function validarUsuario(req, res, next) {`	Exporta la función validarUsuario para que rutas u otros módulos puedan invocarla.
2	`  const datos = req.body || {};`	Declara datos y le asigna el valor o expresión mostrado para reutilizarlo.
3	`  const name = typeof datos.name === 'string' ? datos.name.trim() : '';`	Declara name y le asigna el valor o expresión mostrado para reutilizarlo.
4	`  const lastName = typeof datos.lastName === 'string' ? datos.lastName.trim() : '';`	Declara lastName y le asigna el valor o expresión mostrado para reutilizarlo.
5	`  const email = typeof datos.email === 'string' ? datos.email.trim() : '';`	Declara email y le asigna el valor o expresión mostrado para reutilizarlo.
6	`  const soloLetras = /^[\p{L} ]+$/u;`	Declara soloLetras y le asigna el valor o expresión mostrado para reutilizarlo.
7	`  const emailValido = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;`	Declara emailValido y le asigna el valor o expresión mostrado para reutilizarlo.
8	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
9	`  if (!name) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
10	`    return res.status(400).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
11	`      message: 'El campo nombre no puede ir vacío',`	Define el valor de la propiedad «message» del objeto actual.
12	`    });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
13	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
14	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
15	`  if (name.length < 2 || name.length > 50 || !soloLetras.test(name)) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
16	`    return res.status(400).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
17	`      message: 'El nombre debe tener entre 2 y 50 caracteres y solo puede contener letras y espacios.',`	Define el valor de la propiedad «message» del objeto actual.
18	`    });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
19	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
20	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
21	`  if (!lastName) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
22	`    return res.status(400).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
23	`      message: 'El campo apellido no puede ir vacío',`	Define el valor de la propiedad «message» del objeto actual.
24	`    });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
25	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
26	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
27	`  if (lastName.length < 2 || lastName.length > 50 || !soloLetras.test(lastName)) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
28	`    return res.status(400).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
29	`      message: 'El apellido debe tener entre 2 y 50 caracteres y solo puede contener letras y espacios.',`	Define el valor de la propiedad «message» del objeto actual.
30	`    });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.

N.º	Código original	Qué hace esta línea
31	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
32	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
33	`  if (!email) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
34	`    return res.status(400).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
35	`      message: 'El campo correo electrónico no puede ir vacío',`	Define el valor de la propiedad «message» del objeto actual.
36	`    });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
37	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
38	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
39	`  if (email.length > 100 || !emailValido.test(email)) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
40	`    return res.status(400).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
41	`      message: 'Ingresa un correo electrónico válido de máximo 100 caracteres.',`	Define el valor de la propiedad «message» del objeto actual.
42	`    });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
43	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
44	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
45	`  req.body = { name, lastName, email };`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
46	`  next();`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
47	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.

server/src/routes/auth.js

Define login y logout, compara credenciales y delega la validación de sesión.

51 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { timingSafeEqual } from 'node:crypto';`	Importa { timingSafeEqual } desde node:crypto y conecta este archivo con ese recurso.
2	`import { Router } from 'express';`	Importa { Router } desde express y conecta este archivo con ese recurso.
3	`import { crearSesion, eliminarSesion } from '../authSessions.js';`	Importa { crearSesion, eliminarSesion } desde ../authSessions.js y conecta este archivo con ese recurso.
4	`import { requireAuth } from '../middleware/requireAuth.js';`	Importa { requireAuth } desde ../middleware/requireAuth.js y conecta este archivo con ese recurso.
5	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
6	`const router = Router();`	Declara router y le asigna el valor o expresión mostrado para reutilizarlo.
7	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
8	`function compararCredencial(proporcionada, esperada) {`	Declara la función compararCredencial; el bloque siguiente contiene las instrucciones que ejecuta.
9	`  const valorProporcionado = Buffer.from(proporcionada);`	Declara valorProporcionado y le asigna el valor o expresión mostrado para reutilizarlo.
10	`  const valorEsperado = Buffer.from(esperada);`	Declara valorEsperado y le asigna el valor o expresión mostrado para reutilizarlo.
11	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
12	`  return (`	Devuelve un valor de la función o el elemento React que se renderizará.
13	`    valorProporcionado.length === valorEsperado.length &&`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
14	`    timingSafeEqual(valorProporcionado, valorEsperado)`	Invoca timingSafeEqual para ejecutar la operación definida por esa función o biblioteca.
15	`  );`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
16	`}`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
17	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
18	`router.post('/login', (req, res) => {`	Registra una ruta POST de Express y la conecta a middleware o controlador.
19	`  const usuarioConfigurado = process.env.ADMIN_USERNAME;`	Declara usuarioConfigurado y le asigna el valor o expresión mostrado para reutilizarlo.
20	`  const claveConfigurada = process.env.ADMIN_PASSWORD;`	Declara claveConfigurada y le asigna el valor o expresión mostrado para reutilizarlo.
21	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
22	`  if (!usuarioConfigurado || !claveConfigurada) {`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.
23	`    console.error('Faltan ADMIN_USERNAME o ADMIN_PASSWORD en la configuración del servidor.');`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
24	`    return res.status(503).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
25	`      message: 'El inicio de sesión no está configurado en el servidor.',`	Define el valor de la propiedad «message» del objeto actual.
26	`    });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
27	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
28	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
29	`  const { username, password } = req.body ?? {};`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
30	`  if (`	Evalúa una condición y ejecuta el bloque siguiente solo si se cumple.

N.º	Código original	Qué hace esta línea
31	`    typeof username !== 'string' ||`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
32	`    typeof password !== 'string' ||`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
33	`    !compararCredencial(username, usuarioConfigurado) ||`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
34	`    !compararCredencial(password, claveConfigurada)`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
35	`  ) {`	Abre el bloque de código que contiene las instrucciones siguientes.
36	`    return res.status(401).json({ message: 'Usuario o clave incorrectos.' });`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
37	`  }`	Cierra el bloque, llamada, lista o elemento iniciado en líneas anteriores.
38	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
39	`  const token = crearSesion(usuarioConfigurado);`	Declara token y le asigna el valor o expresión mostrado para reutilizarlo.
40	`  return res.status(200).json({`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
41	`    token,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
42	`    user: { username: usuarioConfigurado },`	Define el valor de la propiedad «user» del objeto actual.
43	`  });`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
44	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
45	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
46	`router.post('/logout', requireAuth, (req, res) => {`	Registra una ruta POST de Express y la conecta a middleware o controlador.
47	`  eliminarSesion(req.authToken);`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
48	`  return res.status(204).end();`	Envía una respuesta HTTP desde el controlador Express y termina la solicitud.
49	`});`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
50	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
51	`export default router;`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.

server/src/routes/incomes.js

Conecta métodos HTTP de /api/ingresos con validaciones y controladores protegidos.

19 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { Router } from 'express';`	Importa { Router } desde express y conecta este archivo con ese recurso.
2	`import {`	Importa { desde el módulo indicado y conecta este archivo con ese recurso.
3	`  actualizarIngreso,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
4	`  crearIngreso,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
5	`  eliminarIngreso,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
6	`  obtenerIngresos,`	Completa o conecta la expresión actual; su significado depende del bloque que la rodea.
7	`} from '../controllers/incomesController.js';`	Completa una instrucción JavaScript/JSX dentro del flujo actual.
8	`import { requireAuth } from '../middleware/requireAuth.js';`	Importa { requireAuth } desde ../middleware/requireAuth.js y conecta este archivo con ese recurso.
9	`import { validarIdIngreso, validarIngreso } from '../middleware/validateIncome.js';`	Importa { validarIdIngreso, validarIngreso } desde ./middleware/validateIncome.js y conecta este archivo con ese recurso.
10	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
11	`const router = Router();`	Declara router y le asigna el valor o expresión mostrado para reutilizarlo.
12	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
13	`router.use(requireAuth);`	Registra una ruta USE de Express y la conecta a middleware o controlador.
14	`router.get('/', obtenerIngresos);`	Registra una ruta GET de Express y la conecta a middleware o controlador.
15	`router.post('/', validarIngreso, crearIngreso);`	Registra una ruta POST de Express y la conecta a middleware o controlador.
16	`router.put('/:id', validarIdIngreso, validarIngreso, actualizarIngreso);`	Registra una ruta PUT de Express y la conecta a middleware o controlador.
17	`router.delete('/:id', validarIdIngreso, eliminarIngreso);`	Registra una ruta DELETE de Express y la conecta a middleware o controlador.
18	`(línea vacía)`	Línea en blanco que separa instrucciones o bloques del código.
19	`export default router;`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.

server/src/routes/users.js

Conecta métodos HTTP de /api/users con validaciones y controladores protegidos.

18 líneas · La numeración coincide con el archivo fuente.

N.º	Código original	Qué hace esta línea
1	`import { Router } from 'express';`	Importa { Router } desde express y conecta este archivo con ese recurso.
2	`import { crearUsuario, obtenerUsuarios, actualizarUsuario, eliminarUsuario } from '../controllers/usersController.js';`	Importa { crearUsuario, obtenerUsuarios, actualizarUsuario, eliminarUsuario } desde ../controllers/usersController.js y conecta este archivo con ese recurso.
3	`import { validarUsuario } from '../middleware/validateUser.js';`	Importa { validarUsuario } desde ../middleware/validateUser.js y conecta este archivo con ese recurso.
4	`import { requireAuth } from '../middleware/requireAuth.js';`	Importa { requireAuth } desde ../middleware/requireAuth.js y conecta este archivo con ese recurso.
5	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
6	`const router = Router();`	Declara router y le asigna el valor o expresión mostrado para reutilizarlo.
7	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
8	`router.use(requireAuth);`	Registra una ruta USE de Express y la conecta a middleware o controlador.
9	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
10	`router.post('/', validarUsuario, crearUsuario); //endpoint para crear un usuario localhost:3000/api/users`	Registra una ruta POST de Express y la conecta a middleware o controlador.
11	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
12	`router.get('/', obtenerUsuarios); //endpoint para obtener todos los usuarios localhost:3000/api/users`	Registra una ruta GET de Express y la conecta a middleware o controlador.
13	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
14	`router.put('/:id', validarUsuario, actualizarUsuario); //endpoint para actualizar un usuario por id localhost:3000/api/users/:id`	Registra una ruta PUT de Express y la conecta a middleware o controlador.
15	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
16	`router.delete('/:id', eliminarUsuario); //endpoint para eliminar un usuario por id localhost:3000/api/users/:id`	Registra una ruta DELETE de Express y la conecta a middleware o controlador.
17	(línea vacía)	Línea en blanco que separa instrucciones o bloques del código.
18	`export default router;`	Exporta el valor principal del archivo para que otro módulo pueda importarlo.